

TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG
CẢNH BÁO THỜI GIAN THỰC
CHO CAMERA GIÁM SÁT AN
NINH DỰA TRÊN AI**

GIẢNG VIÊN HƯỚNG DẪN: **ThS. Nguyễn Thị Mỹ Hạnh**

SINH VIÊN THỰC HIỆN:

Nguyễn Trương Anh Tuấn – 23DH114873

Nguyễn Phương Nguyên – 23DH114782

TP. HỒ CHÍ MINH – THÁNG 9/2026

TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG
CẢNH BÁO THỜI GIAN THỰC
CHO CAMERA GIÁM SÁT AN
NINH DỰA TRÊN AI**

GIẢNG VIÊN HƯỚNG DẪN: ThS. Nguyễn Thị Mỹ Hạnh

SINH VIÊN THỰC HIỆN:

Nguyễn Trương Anh Tuấn – 23DH114873

Nguyễn Phương Nguyên – 23DH114782

TP. HỒ CHÍ MINH – THÁNG 9/2026

LỜI CẢM ƠN

Để hoàn thành báo cáo khóa luận với đề tài “Xây dựng hệ thống cảnh báo thời gian thực cho camera giám sát an ninh dựa trên AI”, nhóm chúng em đã nhận được rất nhiều sự hỗ trợ, hướng dẫn và động viên quý báu trong suốt quá trình thực hiện đề tài.

Trước hết, nhóm xin gửi lời cảm ơn chân thành đến cô Nguyễn Thị Mỹ Hạnh - giảng viên hướng dẫn học phần, người đã tận tình truyền đạt kiến thức nền tảng về thị giác máy tính, phát hiện và theo dõi đối tượng, đồng thời định hướng cho nhóm trong việc lựa chọn phương pháp và công cụ phù hợp để triển khai một hệ thống giám sát thời gian thực hoàn chỉnh.

Nhóm cũng xin gửi lời cảm ơn đến Khoa Công nghệ Thông tin và Trường Đại học Ngoại ngữ - Tin học Thành phố Hồ Chí Minh đã tạo điều kiện về cơ sở vật chất và môi trường học tập, giúp nhóm có cơ hội tiếp cận với các bài toán thực tiễn trong lĩnh vực trí tuệ nhân tạo ứng dụng.

Bên cạnh đó, nhóm xin cảm ơn cộng đồng mã nguồn mở và các tác giả của những công cụ, thư viện đã được sử dụng trong đề tài này, bao gồm Ultralytics YOLO, ByteTrack, InsightFace, OpenCV, FastAPI cùng các tài liệu nghiên cứu liên quan đến bài toán theo dõi đa đối tượng (Multi-Object Tracking) đã hỗ trợ rất nhiều trong quá trình tìm hiểu lý thuyết và triển khai hệ thống.

Mặc dù nhóm đã cố gắng hoàn thành báo cáo một cách tốt nhất, nhưng do kiến thức và kinh nghiệm thực tế còn hạn chế nên khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ thầy/cô để đề tài được hoàn thiện hơn.

Nhóm xin chân thành cảm ơn!

LỜI CAM ĐOAN

Chúng tôi xin cam đoan báo cáo khóa luận tốt nghiệp này là kết quả của quá trình nghiên cứu, phân tích, thiết kế và thực hiện của nhóm dưới sự hướng dẫn của giảng viên hướng dẫn.

Các nội dung, số liệu, hình ảnh và kết quả thực nghiệm được trình bày trong báo cáo là trung thực và phản ánh đúng quá trình triển khai đề tài. Những kiến thức, tài liệu và kết quả nghiên cứu của các tác giả khác được sử dụng trong báo cáo đều được trích dẫn và ghi rõ nguồn tham khảo.

Chúng tôi xin chịu hoàn toàn trách nhiệm về tính trung thực và tính nguyên bản của nội dung báo cáo này.

Thành phố Hồ Chí Minh, ngày tháng năm 2026

Nhóm sinh viên thực hiện

Sinh viên thực hiện 1

(Ký và ghi rõ họ tên)

Sinh viên thực hiện 2

(Ký và ghi rõ họ tên)

Nguyễn Trương Anh Tuấn

Nguyễn Phương Nguyên

MỤC LỤC

LỜI CẢM ƠN	i
LỜI CAM ĐOAN	ii
MỤC LỤC	i
DANH MỤC TỪ VIẾT TẮT	vi
DANH MỤC BẢNG	vii
DANH MỤC HÌNH ẢNH	viii
CHƯƠNG 1: TỔNG QUAN	1
1.1 Đặt vấn đề	1
1.2 Lý do chọn đề tài	2
1.3 Mục tiêu đề tài	2
1.3.1 Mục tiêu tổng quát	2
1.3.2 Mục tiêu cụ thể	3
1.4 Đối tượng và phạm vi nghiên cứu	3
1.5 Các công trình liên quan	4
1.6 Bố cục báo cáo	5
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	7
2.1 Tổng quan bài toán theo dõi đa đối tượng	7
2.2 Các mô hình phát hiện đối tượng YOLO	8
2.2.1 Vì sao chọn YOLO11s (biến thể Small) cho tầng phát hiện	10
2.3 Mô hình phát hiện đối tượng YOLOv11	10

2.4	Thuật toán theo dõi đa đối tượng ByteTrack.....	13
2.4.1	Hạn chế của các phương pháp MOT truyền thống	15
2.4.2	Vì sao chọn ByteTrack thay vì DeepSORT	15
2.4.3	Chiến lược liên kết hai bước BYTE	15
2.4.4	Mô hình chuyển động với bộ lọc Kalman.....	17
2.4.5	Cơ chế bù chuyển động camera (CMC).....	17
2.4.6	Quản lý trạng thái và ID đối tượng (Grace Frames)	18
2.4.7	Ưu điểm về hiệu suất	19
2.5	Thuật toán Hungarian (Kuhn–Munkres) trong bài toán gán ghép tối ưu.....	19
2.6	Nhận diện khuôn mặt dựa trên học biểu diễn (Representation Learning) với ArcFace	23
2.7	Cơ sở hình học tính toán trong phân tích hành vi.....	24
2.8	Mô hình hồi quy Logistic và chấm điểm rủi ro.....	26
2.9	Mô hình ước lượng tư thế (YOLOv11-Pose) trong pipeline phân tích hành vi.....	29
2.10	Các độ đo đánh giá hệ thống (Evaluation Metrics)	30
2.11	Cơ chế Object Detection: các khái niệm cốt lõi	32
2.12	Loss Function (hàm mất mát).....	34
2.13	Xử lý các vấn đề thực tế thường gặp.....	35
2.13.1	Người / vật bị che khuất (Occlusion)	36
2.13.2	Vật thể nhỏ / ở xa.....	38

2.13.3	<i>Ánh sáng yếu / ban đêm</i>	39
2.13.4	<i>Camera rung</i>	41
2.13.5	<i>ID Switch (nhằm đổi tượng A thành B)</i>	42
CHƯƠNG 3: TRIỂN KHAI ĐỀ TÀI		45
3.1	Kiến trúc tổng thể hệ thống	45
3.2	Môi trường và công cụ phát triển	46
3.3	Triển khai tầng phát hiện và theo dõi đối tượng	46
3.4	Triển khai tầng nhận diện danh tính	47
3.5	Triển khai tầng phân tích hành vi	48
3.6	Triển khai tầng học hành vi và chấm điểm rủi ro	50
3.7	Triển khai hệ thống điều phối cảnh báo và giao diện web	51
CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM		53
4.1	Kịch bản đo hiệu năng	53
4.2	Kết quả thông lượng và độ trễ	53
4.3	Độ trễ theo từng giai đoạn xử lý	54
4.4	Kiểm thử tự động hệ thống	57
4.5	Kiểm thử nghiệp vụ trên camera thực tế	71
4.5.1	<i>Trường hợp 1: Phát hiện người ngã và nằm bất động</i>	<i>71</i>
4.5.2	<i>Trường hợp 2: Ước lượng khung xương và phát hiện hành vi nghi trộm cắp</i>	<i>75</i>
4.5.3	<i>Trường hợp 3: Phát hiện tài sản bị di chuyển hoặc biến mất</i>	<i>77</i>

4.5.4	<i>Trường hợp 4: Phát hiện hành vi lãng vãng trong vùng ROI.....</i>	79
4.5.5	<i>Trường hợp 5: Phát hiện người lạ xuất hiện</i>	81
4.5.6	<i>Trường hợp 6: Phát hiện người lạ có hành vi đáng ngờ.....</i>	83
4.5.7	<i>Trường hợp 7: Phát hiện cắt vạch theo hướng vào</i>	85
4.6	Đánh giá pilot định lượng các hành vi.....	87
4.6.1	<i>Thiết kế thí nghiệm pilot.....</i>	87
4.6.2	<i>Tiêu chí đánh giá</i>	87
4.6.3	<i>Kết quả trên frozen test.....</i>	88
4.6.4	<i>Phân tích kết quả</i>	89
4.6.5	<i>Hạn chế của pilot.....</i>	90
4.7	Đánh giá pilot ngưỡng nhận diện khuôn mặt	90
4.7.1	<i>Thiết kế và giao thức pilot</i>	90
4.7.2	<i>Tiêu chí đánh giá</i>	90
4.7.3	<i>Kết quả ROC, EER và FAR/FRR.....</i>	91
4.7.4	<i>Hạn chế của pilot khuôn mặt.....</i>	93
4.8	Đánh giá độ ổn định hệ thống.....	93
4.9	Kết quả đạt được và hạn chế còn tồn tại	94
4.9.1	<i>Kết quả đạt được</i>	94
4.9.2	<i>Hạn chế còn tồn tại.....</i>	95
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		97
5.1	Kết luận.....	97
5.2	Ưu điểm và nhược điểm	98

5.2.1	<i>Ưu điểm</i>	98
5.2.2	<i>Nhược điểm</i>	98
5.3	<i>Hướng phát triển</i>	99
5.3.1	<i>Hướng phát triển ngắn hạn</i>	99
5.3.2	<i>Hướng phát triển dài hạn</i>	100
TÀI LIỆU THAM KHẢO		101

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh đầy đủ
AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface – Giao diện lập trình ứng dụng
CMC	Camera Motion Compensation – Bù chuyển động camera
CNN	Convolutional Neural Network – Mạng nơ-ron tích chập
FPS	Frames Per Second – Số khung hình trên giây
GPU	Graphics Processing Unit – Bộ xử lý đồ họa
HOTA	Higher Order Tracking Accuracy
IDF1	Identity F1 Score
IoU	Intersection over Union – Tỷ lệ giao trên hợp
JSON	JavaScript Object Notation
MOT	Multi-Object Tracking – Theo dõi đa đối tượng
MOTA	Multiple Object Tracking Accuracy
REST	Representational State Transfer
RSS	Resident Set Size – Bộ nhớ thường trú
RTSP	Real Time Streaming Protocol
YOLO	You Only Look Once

DANH MỤC BẢNG

BẢNG 2-1: SO SÁNH CÁC PHIÊN BẢN YOLO HIỆN ĐẠI (V8 – YOLO26) TRÊN CÁC TIÊU CHÍ QUYẾT ĐỊNH	8
BẢNG 2-2: MA TRẬN IOU GIỮA CÁC TRACK HIỆN CÓ VÀ CÁC DETECTION MỚI.....	21
BẢNG 2-3: SO SÁNH KẾT QUẢ GÁN GHÉP GIỮA CHIẾN LƯỢC GREEDY VÀ THUẬT TOÁN HUNGARIAN.....	22
BẢNG 2-4: TÓM TẮT CÁC ĐỘ ĐO ĐÁNH GIÁ HỆ THỐNG MOT	31
BẢNG 2-5: CÁC VẤN ĐỀ THỰC TẾ THƯỜNG GẶP VÀ CÁCH XỬ LÝ .	35
BẢNG 4-1: THÔNG SỐ HAI MÔI TRƯỜNG THỰC NGHIỆM.....	53
BẢNG 4-2: AI FPS TỔNG HỢP THEO SỐ LƯỢNG CAMERA TRONG PHÉP ĐO 10 PHÚT.....	53
BẢNG 4-3: ĐỘ TRỄ TRUNG BÌNH TỪNG GIAI ĐOẠN XỬ LÝ AI (TRƯỜNG HỢP MỘT CAMERA)	54
BẢNG 4-4: TỔNG HỢP CÁC NHÓM TESTCASE TỰ ĐỘNG THEO THÀNH PHẦN HỆ THỐNG.....	57
BẢNG 4-5: KẾT QUẢ TỪNG HÀNH VI TRÊN TẬP KIỂM THỬ ĐÓNG BẢNG	89
BẢNG 4-6: KẾT QUẢ PILOT NHẬN DIỆN KHUÔN MẶT TẠI BA NGƯỠNG KHẢO SÁT	92
BẢNG 4-7: KẾT QUẢ KIỂM THỬ ĐỘ ỔN ĐỊNH BỐN CAMERA TRONG 8 GIỜ.....	93

DANH MỤC HÌNH ẢNH

HÌNH 1: SƠ ĐỒ TỔNG QUÁT MULTI-OBJECT TRACKING	7
HÌNH 2: TRỰC QUAN HÓA BA TIÊU CHÍ QUYẾT ĐỊNH (ĐỘ CHÍNH XÁC, TỐC ĐỘ, ĐỘ TRƯỞNG THÀNH HỆ SINH THÁI) CHO TỪNG PHIÊN BẢN	9
HÌNH 3: KIẾN TRÚC BACKBONE – NECK – HEAD CỦA YOLOV11.....	11
HÌNH 4: CHIẾN LƯỢC LIÊN KẾT HAI BƯỚC BYTE	16
HÌNH 5: CHU TRÌNH DỰ ĐOÁN - CẬP NHẬT (PREDICT - UPDATE) CỦA BỘ LỌC KALMAN	17
HÌNH 6: BA BƯỚC CỦA CƠ CHẾ BÙ CHUYỂN ĐỘNG CAMERA (CAMERA MOTION COMPENSATION)	18
HÌNH 7: BÀI TOÁN GÁN GHÉP LƯỖNG PHÂN GIỮA TRACK VÀ DETECTION.....	22
HÌNH 8: SO SÁNH KHÔNG GIAN ĐẶC TRƯNG GIỮA SOFTMAX TRUYỀN THỐNG VÀ ARCFACE.....	23
HÌNH 9: HAI KỸ THUẬT HÌNH HỌC TÍNH TOÁN DÙNG TRONG PHÂN TÍCH HÀNH VI	25
HÌNH 10: HÀM SIGMOID ÁNH XẠ ĐIỂM SỐ TUYẾN TÍNH Z VỀ XÁC SUẤT RỦI RO TRONG KHOẢNG (0,1)	26
HÌNH 11: CÁC ĐIỂM KHỚP XƯƠNG (KEYPOINTS) ĐƯỢC YOLOV11-POSE ƯỚC LƯỢNG, DÙNG ĐỂ PHÂN TÍCH TƯ THẾ VÀ HÀNH VI.....	29
HÌNH 12: MINH HỌA IOU GIỮA HAI KHUNG BAO VÀ CƠ CHẾ LỌC TRÙNG LẬP NON-MAXIMUM SUPPRESSION (NMS)	32
HÌNH 13: CÁC THÀNH PHẦN HÌNH HỌC ĐƯỢC CIOU LOSS XEM XÉT: ĐỘ CHỒNG LẤP IOU	34

HÌNH 14: CONFIDENCE SCORE SỤT GIẢM TẠM THỜI KHI ĐỐI TƯỢNG BỊ CHE KHUẤT MỘT PHẦN, SAU ĐÓ PHỤC HỒI KHI ĐỐI TƯỢNG XUẤT HIỆN TRỞ LẠI.....	37
HÌNH 15: VẬT THỂ NHỎ HƠN MỘT Ô LƯỚI TRÊN FEATURE MAP CÓ STRIDE LỚN, DỄ BỊ BỎ SÓT.....	38
HÌNH 16: CÙNG MỘT ĐỐI TƯỢNG TRONG BA ĐIỀU KIỆN	40
HÌNH 17: BIỂU ĐỒ HISTOGRAM VÙNG TỐI SAU KHI ÁP DỤNG CLAHE, TĂNG ĐỘ TƯƠNG PHẢN GIÚP LỘ RÕ CHI TIẾT ĐỐI TƯỢNG	40
HÌNH 18: SO SÁNH IOU GIỮA HAI KHUNG LIÊN TIẾP KHI ẢNH RÕ NÉT SO VỚI KHI BỊ NHÒE DO RUNG	42
HÌNH 19: DIỄN BIẾN ID SWITCH KHI KHÔNG VÀ CÓ KẾT HỢP KALMAN + HUNGARIAN	43
HÌNH 20: KIẾN TRÚC PIPELINE SÁU TẦNG CỦA HỆ THỐNG	45
HÌNH 21: HAI NGUYÊN LÝ HÌNH HỌC NỀN TẢNG	48
HÌNH 22: SƠ ĐỒ MÔ HÌNH CHẤM ĐIỂM RỦI RO HÀNH VI BẰNG HỒI QUY LOGISTIC	50
HÌNH 23: KIẾN TRÚC ĐIỀU PHỐI CẢNH BÁO THEO MÔ HÌNH PRODUCER-CONSUMER	51
HÌNH 24: PHÁT HIỆN VÀ THEO DÕI	55
HÌNH 25: ƯỚC LƯỢNG TƯ THẾ	55
HÌNH 26: NHẬN DIỆN DANH TÍNH.....	56
HÌNH 27: PHÂN TÍCH HÀNH VI.....	56
HÌNH 28: KẾT QUẢ PHÁT HIỆN NGƯỜI NGÃ VÀ NẢM BẮT ĐỘNG KÈM CẢNH BÁO SOS TRÊN CAMERA THỰC TẾ	73

HÌNH 29: KẾT QUẢ PHÁT HIỆN HÀNH VI NGHI TRỘM CẤP VÀ GỬI CẢNH BÁO POSSIBLE THEFT QUA TELEGRAM.....	76
HÌNH 30: KẾT QUẢ PHÁT HIỆN XE MÁY BIẾN MẤT KHỎI VÙNG GIÁM SÁT VÀ GỬI CẢNH BÁO ASSET MISSING.....	78
HÌNH 31: KẾT QUẢ PHÁT HIỆN NGƯỜI LẠ LƯU LẠI TRONG VÙNG ROI QUÁ THỜI GIAN QUY ĐỊNH.....	80
HÌNH 32: KẾT QUẢ PHÁT HIỆN NGƯỜI LẠ TRÊN TOÀN KHUNG HÌNH VÀ GỬI CẢNH BÁO.....	82
HÌNH 33: KẾT QUẢ PHÁT HIỆN NGƯỜI LẠ CÓ HÀNH VI ĐỨNG LÂU ĐÁNG NGỜ.....	84
HÌNH 34: KẾT QUẢ PHÁT HIỆN NGƯỜI CẮT VẠCH THEO HƯỚNG VÀO VÀ CẬP NHẬT BỘ ĐẾM	86
HÌNH 35: KẾT QUẢ ĐÁNH GIÁ THỬ NGHIỆM HÀNH VI TRÊN TẬP KIỂM THỬ ĐÓNG BĂNG	88
HÌNH 36: ĐƯỜNG ROC CỦA PILOT NHẬN DIỆN KHUÔN MẶT HAI DANH TÍNH.....	91
HÌNH 37: FAR VÀ FRR THEO NGUỖNG COSINE SIMILARITY TRONG PILOT	92

CHƯƠNG 1: TỔNG QUAN

1.1 Đặt vấn đề

Camera an ninh (CCTV/IP camera) hiện là thiết bị phổ biến trong hộ gia đình, cửa hàng nhỏ và các cơ sở kinh doanh vừa và nhỏ nhờ chi phí phần cứng ngày càng giảm và các nền tảng lưu trữ đám mây dễ tiếp cận. Tuy nhiên, phần lớn hệ thống camera hiện nay vẫn dừng lại ở chức năng ghi hình liên tục hoặc ghi hình khi phát hiện chuyển động (motion-triggered recording), tức chỉ đóng vai trò lưu trữ bằng chứng sau sự việc chứ chưa thực sự hiểu được nội dung đang diễn ra trước camera.

Hệ quả trực tiếp của cách tiếp cận này là gánh nặng đặt hoàn toàn lên vai người vận hành: hoặc phải xem lại hàng giờ video sau sự cố, hoặc phải trực theo dõi màn hình liên tục - một công việc dễ gây mỏi mệt và bỏ sót thông tin theo thời gian. Đối với các hệ thống dùng cảm biến chuyển động đơn thuần, số lượng cảnh báo giả (false alarm) phát sinh từ gió thổi cây cối, ánh đèn xe đi ngang, bóng đổ thay đổi theo thời gian trong ngày hoặc vật nuôi di chuyển thường rất lớn, dẫn đến hiện tượng “mệt mỏi vì cảnh báo” (alert fatigue) khiến người dùng dần bỏ qua hoặc tắt hẳn tính năng thông báo. Ngoài ra, các hệ thống truyền thống không có khái niệm “danh tính” nên không thể tự động phân biệt thành viên quen thuộc với người lạ, cũng không có ngữ cảnh không gian và thời gian nên không thể phát hiện các hành vi bất thường kéo dài như lang thang gần một khu vực hay tương tác nghi vấn với tài sản.

Từ thực trạng trên, bài toán đặt ra là làm thế nào để chuyển đổi camera giám sát từ một thiết bị ghi hình thụ động thành một hệ thống có khả năng phát hiện, theo dõi, nhận diện và đánh giá hành vi theo thời gian thực, đồng thời tự động gửi cảnh báo có ngữ cảnh rõ ràng đến người vận hành, trong điều kiện phần cứng phổ thông (GPU máy tính xách tay, mini-PC) chứ không phải hạ tầng máy chủ chuyên dụng.

1.2 Lý do chọn đề tài

Việc đưa các mô hình thị giác máy tính hiện đại - phát hiện đối tượng (object detection), theo dõi đa đối tượng (multi-object tracking) và nhận diện khuôn mặt - vào một hệ thống giám sát thực tế mang lại giá trị ứng dụng cao nhưng cũng đặt ra nhiều thách thức kỹ thuật thú vị, phù hợp để nghiên cứu trong khuôn khổ khóa luận tốt nghiệp:

- **Tính thời gian thực:** mô hình phát hiện và theo dõi phải xử lý đủ nhanh để không làm trễ luồng video hiển thị, đòi hỏi phải hiểu rõ và tối ưu từng khâu trong pipeline xử lý ảnh.
- **Duy trì danh tính đối tượng qua thời gian:** thuật toán theo dõi phải đảm bảo một người không bị đổi mã định danh (ID) liên tục khi bị che khuất tạm thời, nếu không các luật hành vi dựa trên thời gian lưu trữ sẽ không hoạt động chính xác - đây là bài toán mở, có nhiều hướng cải tiến để khảo sát.
- **Cân bằng giữa độ nhạy và số lượng cảnh báo giả:** ngưỡng phát hiện càng thấp thì càng ít bỏ sót đối tượng nhưng càng dễ sinh cảnh báo giả, đòi hỏi kết hợp cả luật nghiệp vụ lẫn một lớp học máy giám sát bổ sung để giải quyết đánh đổi này.
- **Tính ứng dụng thực tiễn cao:** một hệ thống cảnh báo an ninh thông minh có thể áp dụng trực tiếp cho hộ gia đình, cửa hàng, bãi giữ xe hay khuôn viên trường học - phù hợp với xu hướng phát triển các sản phẩm AI ứng dụng gần gũi với đời sống.

1.3 Mục tiêu đề tài

1.3.1 Mục tiêu tổng quát

Nghiên cứu, thiết kế và hiện thực một hệ thống cảnh báo an ninh thời gian thực cho camera giám sát, có khả năng phát hiện, theo dõi, nhận diện danh tính và phân

Chương 1: Tổng quan

tích hành vi của người/phương tiện trong khung hình, từ đó tự động phát cảnh báo có nguy cơ đến người vận hành qua nhiều kênh thông báo.

1.3.2 Mục tiêu cụ thể

- Xây dựng pipeline xử lý video thời gian thực tích hợp mô hình phát hiện đối tượng YOLOv11 và thuật toán theo dõi đa đối tượng ByteTrack.
- Hiện thực module nhận diện danh tính bằng InsightFace để phân biệt người quen và người lạ.
- Thiết kế và hiện thực các luật phân tích hành vi (xâm nhập, lãng vãng, theo dõi tài sản, nghi vấn trộm cắp, đếm lượt qua vạch...) dựa trên vùng và vạch giám sát do người dùng tự cấu hình.
- Xây dựng một lớp học máy giám sát nhẹ (hồi quy logistic) nhằm chấm điểm rủi ro cho từng cảnh báo, giảm tỉ lệ cảnh báo giả theo thời gian.
- Xây dựng hệ thống điều phối và phân phối cảnh báo đa kênh (Telegram, Discord, còi báo động cục bộ) cùng giao diện web quản lý trực quan.
- Đo lường và đánh giá hiệu năng hệ thống (tốc độ khung hình, độ trễ xử lý) trên phần cứng phổ thông.

1.4 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là các phương pháp thị giác máy tính phục vụ bài toán giám sát an ninh, bao gồm mô hình phát hiện đối tượng một giai đoạn (YOLO), thuật toán theo dõi đa đối tượng theo hướng tracking-by-detection (ByteTrack), mô hình nhận diện khuôn mặt dựa trên vector nhúng (InsightFace/ArcFace), các phương pháp hình học phân tích không gian (point-in-polygon, tích có hướng) và mô hình học máy có giám sát cổ điển (hồi quy logistic).

Chương 1: Tổng quan

Phạm vi triển khai của đề tài giới hạn ở kịch bản giám sát trong nhà/ngoài trời quy mô nhỏ (hộ gia đình, cửa hàng, bãi giữ xe nhỏ) với một đến vài camera chạy trên một máy tính duy nhất có GPU phổ thông; đề tài không hướng đến các hệ thống giám sát quy mô thành phố với hàng trăm camera hay hạ tầng máy chủ chuyên dụng nhiều GPU. Nguồn video đầu vào có thể là webcam, camera IP qua giao thức RTSP, hoặc tệp video ghi sẵn dùng để kiểm thử lặp lại kết quả.

1.5 Các công trình liên quan

Bài toán theo dõi đa đối tượng thời gian thực đã được nghiên cứu rộng rãi trong những năm gần đây [6], [7]. SORT là một trong những phương pháp tracking-by-detection sớm và đơn giản, sử dụng bộ lọc Kalman để dự đoán chuyển động và thuật toán Hungarian để gán detection cho track [6], [11], [12]. DeepSORT mở rộng SORT bằng cách bổ sung vector đặc trưng ngoại hình (appearance embedding) để tái nhận diện đối tượng sau khi bị che khuất trong thời gian dài, giúp giảm số lần thay đổi ID nhưng làm tăng chi phí tính toán [7].

ByteTrack, được nhóm lựa chọn làm nền tảng cho tăng theo dõi trong đề tài này, xuất phát từ nhận định rằng phần lớn các phương pháp trước đó loại bỏ những khung phát hiện có độ tin cậy thấp, khiến các đối tượng bị che khuất một phần dễ bị bỏ sót. Thay vào đó, ByteTrack đề xuất chiến lược liên kết hai bước BYTE: các khung có độ tin cậy cao được gán cho track trước, sau đó các khung có độ tin cậy thấp còn lại được sử dụng để khôi phục những track chưa được gán. Cách tiếp cận này giúp cải thiện độ đo IDF1 trên nhiều bộ dữ liệu chuẩn mà không cần sử dụng một mạng nơ-ron riêng để trích xuất đặc trưng ngoại hình [1], [2].

Về phía mô hình phát hiện đối tượng, dòng mô hình YOLO (You Only Look Once) là lựa chọn phổ biến cho các ứng dụng thời gian thực nhờ kiến trúc một giai đoạn, không cần bước đề xuất vùng như Faster R-CNN [5], [14]. Các nghiên cứu gần đây về kiến trúc dựa trên Transformer như RT-DETR và DEIM cho thấy khả năng loại bỏ bước hậu xử lý Non-Maximum Suppression trong khi vẫn duy trì tốc độ xử lý

Chương 1: Tổng quan

cạnh tranh với YOLO, mở ra hướng phát triển tiềm năng cho các phiên bản tương lai của hệ thống [3], [4].

Về nhận diện danh tính, các phương pháp dựa trên vector nhúng khuôn mặt như ArcFace, sử dụng hàm mất mát góc cộng tính, đã trở thành một trong những cách tiếp cận phổ biến của bài toán nhận diện khuôn mặt hiện đại. Phương pháp này cho phép so sánh độ tương đồng cosine giữa khuôn mặt quan sát được và ảnh tham chiếu mà không cần huấn luyện lại mô hình cho từng cá nhân mới; đây cũng là cách tiếp cận được InsightFace hiện thực và được đề tài kế thừa trực tiếp [10].

Bên cạnh phát hiện và theo dõi đối tượng, các phương pháp học sâu cũng đã được nghiên cứu để nhận biết hành vi bất thường trực tiếp từ video giám sát thực tế [18].

1.6 Bộ cục báo cáo

Phần còn lại của báo cáo được tổ chức thành bốn chương tiếp theo và phần Tài liệu tham khảo như sau:

- **Chương 2 - Cơ sở lý thuyết:** trình bày các nền tảng lý thuyết được sử dụng trong đề tài, bao gồm bài toán theo dõi đa đối tượng, mô hình phát hiện đối tượng YOLOv11, thuật toán ByteTrack, nhận diện khuôn mặt bằng học sâu, các phép toán hình học phục vụ phân tích hành vi, mô hình hồi quy Logistic và các độ đo dùng để đánh giá hệ thống.
- **Chương 3 - Triển khai đề tài:** mô tả kiến trúc tổng thể hệ thống, môi trường và công cụ phát triển, cùng cách hiện thực từng tầng xử lý: phát hiện và theo dõi đối tượng, nhận diện danh tính, phân tích hành vi, học máy chấm điểm rủi ro và điều phối cảnh báo qua giao diện web.
- **Chương 4 - Kết quả thực nghiệm:** trình bày kịch bản và kết quả đo hiệu năng/scaling từ một đến bốn camera, độ trễ theo từng giai đoạn, bộ kiểm thử tự động, bày kịch bản E2E nghiệp vụ trên camera thực tế, pilot định lượng

Chương 1: Tổng quan

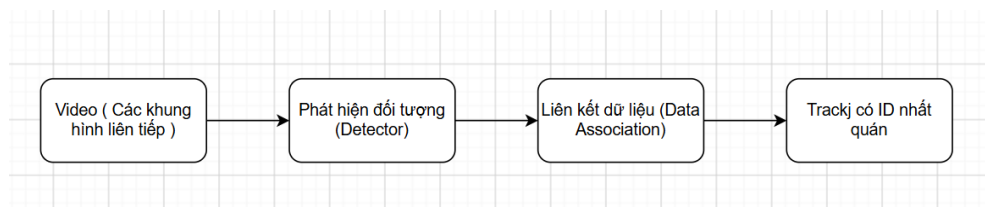
năm hành vi trên frozen test, pilot ngưỡng nhận diện khuôn mặt hai danh tính, hai phiên stability bốn camera trong 8 giờ, cùng các kết quả đạt được và hạn chế còn tồn tại.

- **Chương 5 - Kết luận và hướng phát triển:** tổng kết những kết quả chính đạt được, phân tích ưu điểm, nhược điểm của hệ thống và đề xuất hướng phát triển ngắn hạn, dài hạn.
- **Phần Tài liệu tham khảo:** liệt kê các tài liệu, bài báo và nguồn tham khảo được trích dẫn theo chuẩn IEEE trong suốt báo cáo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Tổng quan bài toán theo dõi đa đối tượng

Theo dõi đa đối tượng (Multi-Object Tracking - MOT) là bài toán xác định vị trí và duy trì danh tính nhất quán của nhiều đối tượng qua các khung hình liên tiếp trong một video. Hướng tiếp cận phổ biến nhất hiện nay là tracking-by-detection: ở mỗi khung hình, một mô hình phát hiện đối tượng (object detector) trả về tập hợp các khung bao (bounding box) ứng viên, sau đó một thuật toán liên kết dữ liệu (data association) gán mỗi khung bao cho một track đã tồn tại hoặc khởi tạo một track mới nếu không có track nào phù hợp.



Hình 2-1: Sơ đồ tổng quát Multi-Object Tracking

Hướng tiếp cận Tracking-by-Detection: đây là phương pháp phổ biến nhất hiện nay, chia bài toán thành hai giai đoạn: (1) mô hình phát hiện đối tượng trả về các khung bao (bounding box) ứng viên tại mỗi khung hình, và (2) thuật toán liên kết dữ liệu (data association) gán mỗi khung bao cho một track đã tồn tại.

Thách thức của MOT: các hệ thống thực tế thường đối mặt với vật thể bị che khuất (occlusion), thay đổi ánh sáng và hiện tượng “nhảy ID” (ID switches).

Hai thành phần cốt lõi của một hệ thống tracking-by-detection là: (1) mô hình chuyển động (motion model), thường dùng bộ lọc Kalman để dự đoán vị trí của track ở khung hình kế tiếp dựa trên vị trí và vận tốc ước lượng ở các khung hình trước; và (2) độ đo tương đồng (similarity metric) giữa khung bao dự đoán và khung bao phát hiện được, phổ biến nhất là độ giao trên hợp (Intersection over Union - IoU), được định nghĩa giữa hai khung bao A và B như sau [6], [11]:

$$IoU(A,B) = \frac{|A \cap B|}{|A \cup B|} \quad (1)$$

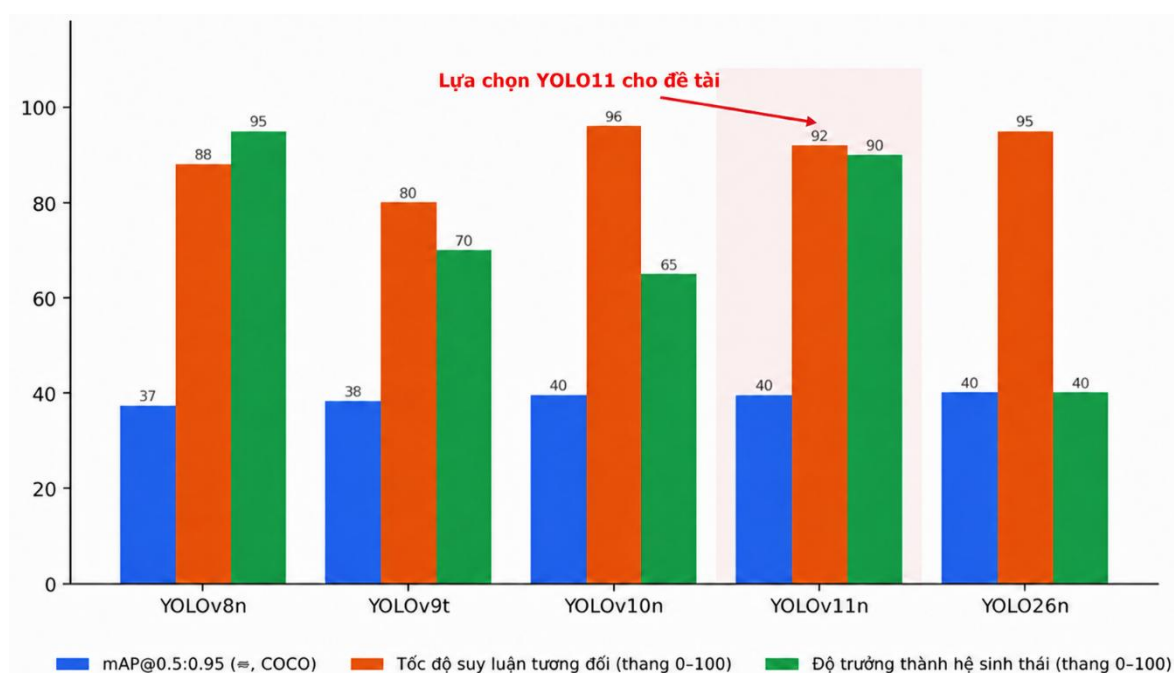
Sau khi tính ma trận tương đồng giữa tất cả các cặp track dự đoán và khung bao phát hiện, thuật toán Hungarian được sử dụng để tìm phép gán tối ưu toàn cục, tối đa hóa tổng độ tương đồng hoặc tối thiểu hóa tổng chi phí gán [12].

2.2 Các mô hình phát hiện đối tượng YOLO

Bảng 2-1: So sánh các phiên bản YOLO hiện tại trên các tiêu chí quyết định

Phiên bản	Kiến trúc / NMS	mAP@0.5:0.95	Tốc độ suy luận	Số tác vụ hỗ trợ	Độ trưởng thành hệ sinh thái
YOLOv8n (2023)	Anchor-free, C2f block; có dùng NMS	≈ 37,3%	Rất nhanh	(Detect/Seg/Class/Pose/OBB)	Rất cao - dùng rộng rãi, tài liệu/cộng đồng lớn nhất hiện nay
YOLOv9t (2024)	PGI + GELAN; có dùng NMS	≈ 38,3%	Khá nhanh	Chủ yếu Detect	Trung bình - ít checkpoint đa tác vụ hơn v8/v11
YOLOv10n (2024)	NMS-free trong suy luận (loại bỏ hậu xử lý NMS)	≈ 39,5%	Nhanh nhất nhóm	Chủ yếu Detect	Trung bình - công cụ hỗ trợ còn ít hơn v8/v11 tại thời điểm khảo sát
YOLOv11n (2024–2025)	Anchor-free, khối C3K2, SPPF cải tiến; có dùng NMS	≈ 39,5%	Nhanh, ổn định (20,76 FPS)	(Detect/Seg/Class/Pose/OBB)	Cao -checkpoint đã kiểm chứng cho mọi tác vụ, framework Ultralytics trưởng thành

Phiên bản	Kiến trúc / NMS	mAP@0.5:0.95	Tốc độ suy luận	Số tác vụ hỗ trợ	Độ trưởng thành hệ sinh thái
YOLO2 6n (1/2026)	NMS-free đầu-cuối, tối ưu cho edge	$\approx 40,0\%$	Nhanh, tối ưu edge	(thêm Semantic Seg. + Depth Est.)	Thấp - vừa phát hành, chưa có nhiều kiểm chứng thực tế dài hạn



Hình 2-2: Trực quan hóa ba tiêu chí quyết định (độ chính xác, tốc độ, độ trưởng thành hệ sinh thái) cho từng phiên bản

Nhìn tổng thể, YOLOv10n và YOLO26n có lợi thế về mặt kỹ thuật thuần túy (loại bỏ NMS, tốc độ cao), nhưng tại thời điểm triển khai đề tài, cả hai đều thiếu độ trưởng thành về checkpoint đa tác vụ và tài liệu hỗ trợ so với YOLOv11n. Trong khi đó, YOLOv8n tuy ổn định nhất nhưng có độ chính xác và một số cải tiến kiến trúc thấp hơn YOLOv11n. So sánh này là cơ sở để chọn họ YOLO11; trong cấu hình triển khai thực tế, detector sử dụng YOLO11s để cân bằng độ chính xác và tốc độ, còn

YOLO11n-pose đảm nhiệm ước lượng tư thế, như trình bày chi tiết ở mục kế tiếp [19].

2.2.1 Vì sao chọn YOLO11s (biến thể Small) cho tầng phát hiện

Tối ưu tốc độ và tài nguyên: phiên bản YOLO11s được lựa chọn để cân bằng giữa độ chính xác và tốc độ, trong khi YOLO11n-pose đảm nhiệm ước lượng tư thế. Thông lượng thực tế của cấu hình hiện tại được đo riêng trên hai nền tảng phần cứng và với từ một đến bốn camera, trình bày tại Chương 4.

Phù hợp phần cứng phổ thông: phục vụ mục tiêu triển khai trên GPU máy tính xách tay hoặc mini-PC thay vì hạ tầng máy chủ chuyên dụng đắt tiền; kích thước mô hình nhỏ và ít tham số giúp tiết kiệm VRAM, giảm chi phí triển khai mà vẫn ổn định.

Kiến trúc Anchor-free: giúp mô hình linh hoạt và chính xác hơn với các đối tượng có tỉ lệ khung hình bất thường (người ngồi/nằm) - thường gặp trong kịch bản cảnh báo ngã.

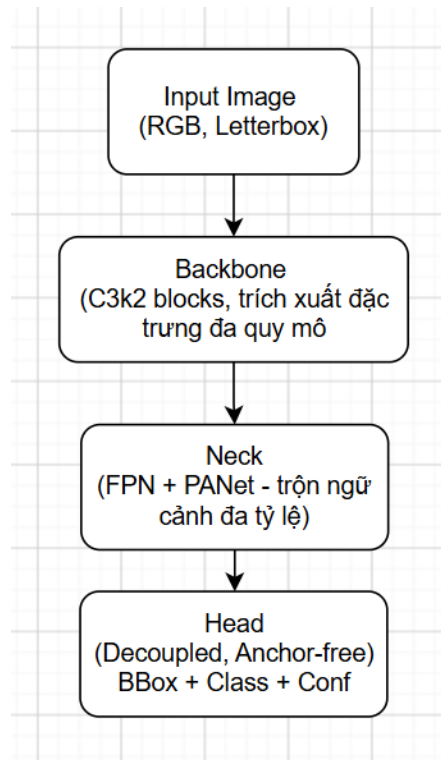
Độ nhạy cao: tầng phát hiện dùng ngưỡng tin cậy thấp (ví dụ 0,20 cho lớp người) để ưu tiên không bỏ sót đối tượng; kết hợp với bước lọc hình học và ByteTrack, hệ thống vẫn duy trì dấu vết ngay cả khi đối tượng bị che khuất một phần.

Chi phí tính toán thấp: dùng YOLO11s cho tầng phát hiện giúp dành tài nguyên cho các tầng xử lý khác (Pose, nhận diện khuôn mặt, phân tích hành vi hình học) mà không nghẽn toàn bộ luồng xử lý.

2.3 Mô hình phát hiện đối tượng YOLOv11

YOLO (You Only Look Once) là họ mô hình phát hiện đối tượng một giai đoạn (single-stage detector), khác với các mô hình hai giai đoạn như Faster R-CNN vốn phải tạo ra một tập vùng đề xuất (region proposal) trước khi phân loại. Thay vào đó, YOLO xem bài toán phát hiện đối tượng như một bài toán hồi quy duy nhất: ảnh đầu vào được đưa qua một mạng nơ-ron tích chập (CNN) duy nhất để đồng thời dự đoán tọa độ khung bao, độ tin cậy và xác suất lớp cho mọi đối tượng trong một lần truyền

xuôi giúp đạt tốc độ suy luận cao hơn nhiều so với các phương pháp hai giai đoạn [14], [20], [21].



Hình 2-3: Kiến trúc Backbone – Neck – Head của YOLOv11

- **Input:** ảnh RGB được chuẩn hóa (normalize pixel về $[0,1]$ hoặc theo mean/std) và resize/pad giữ tỉ lệ (letterbox, ví dụ ảnh gốc 1920×1080 được letterbox về 640×640 bằng viền đệm màu xám để tránh méo ảnh), kết hợp các kỹ thuật tăng cường dữ liệu (Mosaic, Mixup, Random Flip, Color Jitter) trước khi đưa vào mạng.

- **Backbone (xương sống):** chuỗi các lớp tích chập sâu dần, giảm kích thước không gian nhưng tăng số kênh đặc trưng, tạo ra các feature map ở nhiều tỉ lệ (stride 8/16/32 - ví dụ với ảnh 640×640 cho ra các feature map 80×80 cho vật thể nhỏ, 40×40 cho vật thể trung bình, 20×20 cho vật thể lớn). Công nghệ backbone tiêu biểu qua các thời kỳ: Darknet-53 (v3), CSPDarknet (v4/v5), RepVGG/EfficientRep (v6), E-ELAN (v7), C2f (v8), GELAN (v9), C3k2 (v11) [22], [25], [27].
- **Neck (cổ):** trộn các feature map từ nhiều độ phân giải khác nhau của Backbone để mô hình “nhìn” được cả vật thể lớn lẫn cực nhỏ, dùng FPN (Feature Pyramid Network - truyền ngữ cảnh từ tầng sâu xuống tầng nông) kết hợp PANet (Path Aggregation Network - truyền ngược thông tin định vị chi tiết từ tầng nông lên tầng sâu), tạo luồng hai chiều.
- **Detection Head:** nhận feature map đã tích hợp từ Neck để dự đoán cuối cùng. Có hai kiểu: - Coupled Head (YOLOv3–v5): dùng chung một lớp tích chập để dự đoán cả bounding box và class. - Decoupled Head (YOLOv6–v11 trở đi): tách riêng hai nhánh - một nhánh tính tọa độ box (thường qua DFL - Distribution Focal Loss, biểu diễn tọa độ dưới dạng phân bố xác suất để tăng độ chính xác sub-pixel), một nhánh phân loại class - giúp tăng tốc độ hội tụ và độ chính xác.
- **Output:** danh sách bounding box thô (có thể lên tới hàng nghìn box ứng viên) trước khi lọc. Sau khi áp ngưỡng confidence và NMS (hoặc cơ chế NMS-free của v10/YOLO26), thu được danh sách box cuối cùng dạng $[x, y, w, h, confidence, class_id]$.

Kiến trúc YOLO nói chung gồm ba thành phần: backbone trích xuất đặc trưng ảnh ở nhiều tỉ lệ (multi-scale feature extraction), neck kết hợp và tinh chỉnh các đặc trưng từ nhiều tầng khác nhau của backbone (thường theo dạng kim tự tháp đặc trưng - feature pyramid), và head dự đoán khung bao, độ tin cậy và lớp đối tượng dựa trên

các đặc trưng đã kết hợp. Các phiên bản YOLO gần đây, bao gồm YOLOv11 được sử dụng trong đề tài, sử dụng thiết kế không neo, nghĩa là mô hình dự đoán trực tiếp tâm và kích thước khung bao thay vì tinh chỉnh từ một tập khung neo cố định được định nghĩa trước, giúp đơn giản hóa quá trình huấn luyện và cải thiện độ chính xác trên các đối tượng có tỉ lệ khung hình bất thường [8], [9], [14]. Các kiến trúc mạng sâu hiện đại thường sử dụng kết nối tắt để hỗ trợ quá trình lan truyền gradient và huấn luyện ổn định, một ý tưởng được phổ biến bởi kiến trúc ResNet [15].

Thiết kế Anchor-free: đây là cải tiến quan trọng nhất của YOLOv11 so với các bản trước. Thay vì tinh chỉnh tọa độ từ các khung neo (anchor boxes) cố định, YOLOv11 dự đoán trực tiếp tâm và kích thước của đối tượng. Điều này giúp mô hình linh hoạt và chính xác hơn với các đối tượng có tỉ lệ hình học bất thường (ví dụ người đang ngồi hoặc nằm), đồng thời giảm độ phức tạp huấn luyện vì không cần chọn/tinh chỉnh anchor thủ công.

Đầu ra của mô hình sau bước hậu xử lý Non-Maximum Suppression (loại bỏ các khung bao trùng lặp có độ tin cậy thấp hơn) là danh sách các khung bao, mỗi khung bao gồm tọa độ $(x1, y1, x2, y2)$, độ tin cậy và mã lớp đối tượng, chính là dữ liệu đầu vào cho tầng theo dõi ở mục 2.4.

2.4 Thuật toán theo dõi đa đối tượng ByteTrack

ByteTrack là thuật toán theo dõi đa đối tượng được đề xuất nhằm giải quyết một hạn chế phổ biến của các phương pháp tracking-by-detection trước đó: việc chỉ giữ lại các khung phát hiện có độ tin cậy cao (thường trên ngưỡng 0.5) và loại bỏ hoàn toàn các khung có độ tin cậy thấp. Các tác giả của ByteTrack quan sát rằng nhiều khung phát hiện có độ tin cậy thấp thực chất tương ứng với các đối tượng thật sự nhưng bị che khuất một phần (occlusion), và việc loại bỏ chúng gây ra tình trạng mất track và gây quỹ đạo (fragmented trajectory) [2].

Để khai thác cả các khung phát hiện có độ tin cậy thấp mà không làm tăng số lượng nhận nhầm nền (background) thành đối tượng, ByteTrack đề xuất chiến lược

liên kết hai bước gọi là BYTE, với ý tưởng cốt lõi là mỗi khung phát hiện - dù điểm tin cậy cao hay thấp - đều là một đơn vị cơ bản của dữ liệu và cần được xem xét một cách có chọn lọc:

- **Bước 1 (liên kết cao):** Tại mỗi khung hình, các khung phát hiện được chia thành hai tập theo một ngưỡng tin cậy τ - tập điểm cao Dhigh và tập điểm thấp Dlow. Trước tiên, thuật toán dự đoán vị trí mới của mọi track hiện có bằng bộ lọc Kalman, sau đó liên kết các track với các khung phát hiện thuộc Dhigh dựa trên độ tương đồng IoU (hoặc đặc trưng ngoại hình nếu có), thông qua thuật toán Hungarian.
- **Bước 2 (liên kết thấp):** Các track chưa được gán ở bước 1 (thường do đối tượng bị che khuất khiến độ tin cậy phát hiện giảm xuống) được đem liên kết tiếp với các khung phát hiện thuộc tập điểm thấp Dlow, chỉ dựa trên độ tương đồng IoU vì đặc trưng ngoại hình ở các khung điểm thấp thường không đáng tin cậy. Những khung điểm thấp không được liên kết với track nào sẽ bị loại bỏ vì được xem là nền hoặc nhiễu.
- **Cập nhật track:** các track vẫn không được gán sau cả hai bước sẽ được chuyển sang trạng thái mất tạm thời (lost) và chỉ bị xóa hẳn sau khi vượt quá một số khung hình ân hạn cấu hình được; các khung phát hiện điểm cao còn dư sau bước 1 sẽ được dùng để khởi tạo track mới.

Nhờ chiến lược hai bước này, ByteTrack phục hồi được nhiều đối tượng bị che khuất một phần mà vẫn giữ được số lượng nhận nhầm ở mức thấp, đồng thời không yêu cầu một mạng trích xuất đặc trưng ngoại hình riêng biệt như DeepSORT, giúp thuật toán vừa chính xác vừa có tốc độ xử lý cao - phù hợp cho ứng dụng thời gian thực trong đề tài. Bên cạnh chiến lược BYTE nguyên bản, đề tài còn bổ sung thêm cơ chế bù chuyển động camera (Camera Motion Compensation) dựa trên luồng quang học thưa (sparse optical flow) để cải thiện độ chính xác của bước dự đoán Kalman khi camera bị rung nhẹ [13]. Nội dung triển khai cụ thể được trình bày ở Chương 3.

2.4.1 Hạn chế của các phương pháp MOT truyền thống

Các phương pháp tracking-by-detection truyền thống (SORT, DeepSORT) thường áp dụng một ngưỡng tin cậy cố định (thường 0.5-0.6) để lọc bỏ khung bao trước khi liên kết dữ liệu. Trong môi trường giám sát thực tế, đối tượng thường xuyên bị che khuất một phần, mờ do chuyển động hoặc có kích thước nhỏ, khiến điểm tin cậy của bộ phát hiện tụt xuống dưới ngưỡng. Loại bỏ ngay các khung hình này dẫn đến mất dấu đối tượng, gây quỹ đạo và làm tăng đáng kể hiện tượng nhảy ID (ID switches).

2.4.2 Vì sao chọn ByteTrack thay vì DeepSORT

1. Giải quyết triệt để vấn đề che khuất: SORT/DeepSORT chỉ giữ khung phát hiện điểm tin cậy cao (>0.5) và loại bỏ hoàn toàn khung điểm thấp. ByteTrack tận dụng cả các khung điểm thấp qua chiến lược liên kết hai bước BYTE để khôi phục quỹ đạo thay vì loại bỏ.

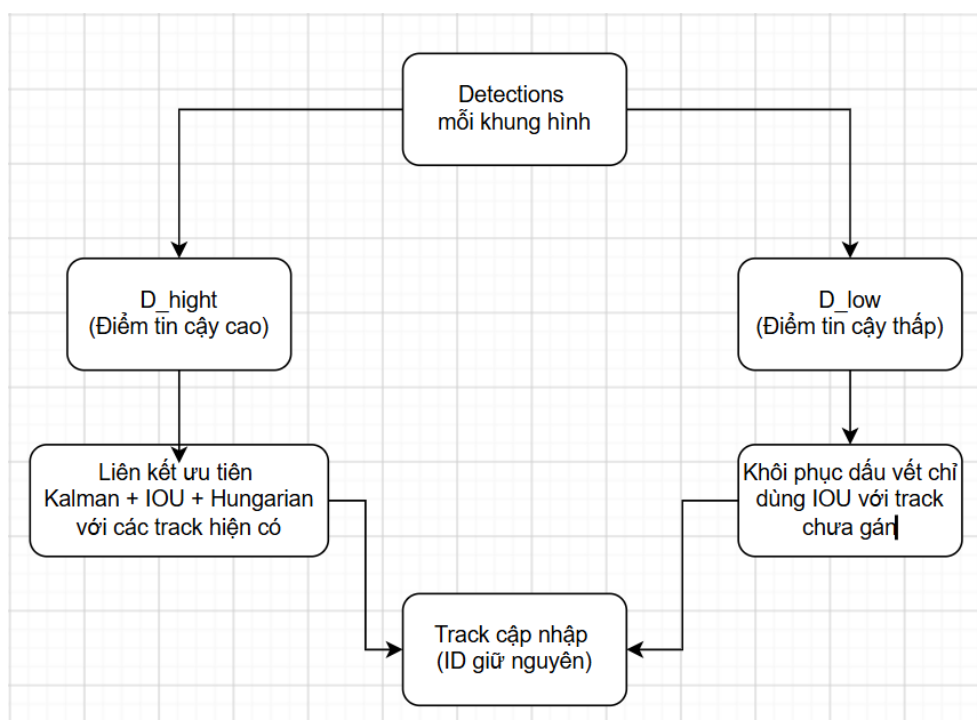
2. Tối ưu tốc độ và tài nguyên: DeepSORT cần một mạng Re-ID riêng để trích xuất đặc trưng ngoại hình cho mọi khung bao, làm tăng chi phí tính toán và độ trễ. ByteTrack chủ yếu dựa trên IoU và Kalman Filter nên giảm chi phí theo dõi; hiệu năng đầu cuối được đánh giá định lượng tại Chương 4.

3. Hệ sinh thái ổn định: đã qua kiểm chứng thực tế rộng rãi, tích hợp sẵn trong Ultralytics qua hàm `.track()`, giảm rủi ro khi triển khai production so với các bản tracker/detector còn ít case study hoặc quá mới

2.4.3 Chiến lược liên kết hai bước BYTE

ByteTrack không loại bỏ ngay các khung bao có độ tin cậy thấp mà coi mọi phát hiện - dù điểm cao hay thấp - là dữ liệu cần được xem xét có chọn lọc. Ở mỗi khung hình, các phát hiện được chia thành tập điểm cao (D_{high}) và tập điểm thấp (D_{low}) dựa trên ngưỡng tin cậy:

- Bước 1 – Liên kết ưu tiên (High-score association): hệ thống dùng bộ lọc Kalman dự đoán vị trí mới của các track hiện có, sau đó liên kết các track này với tập D_high dựa trên độ tương đồng IoU thông qua thuật toán Hungarian.
- Bước 2 – Khôi phục dấu vết (Low-score association): những track chưa được gán ở Bước 1 (thường do đối tượng vừa bị che khuất khiến điểm số giảm) sẽ được đối soát tiếp với tập D_low , chỉ dùng độ đo hình học IoU vì đặc trưng ngoại hình của các khung điểm thấp thường bị nhiễu, không còn ổn định. Các khung điểm thấp không được liên kết sau bước này sẽ bị loại bỏ hoàn toàn.



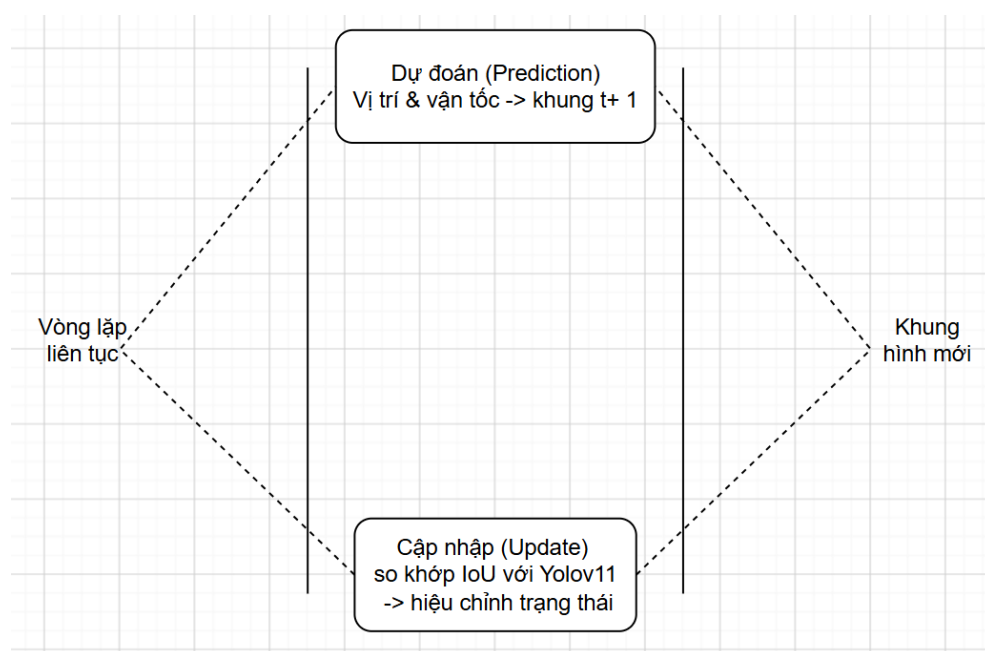
Hình 2-4: Chiến lược liên kết hai bước BYTE

Nhờ cơ chế này, ByteTrack phục hồi được quỹ đạo đối tượng ngay cả khi chúng chỉ lộ ra một phần nhỏ hoặc bị mờ, giúp giảm đáng kể hiện tượng nhảy ID mà không làm tăng số lượng nhận nhầm nhiều nền thành đối tượng.

2.4.4 Mô hình chuyển động với bộ lọc Kalman

Bộ lọc Kalman đóng vai trò “trí nhớ vật lý” giúp dự báo quỹ đạo: - *Dự đoán (Prediction)*: dựa trên vị trí và vận tốc ở các khung hình trước, bộ lọc tính toán vị trí dự kiến của đối tượng ở khung hình kế tiếp. - *Cập nhật (Update)*: khi có kết quả phát hiện mới từ YOLOv11, hệ thống so khớp (qua IoU) và điều chỉnh lại các tham số trạng thái của bộ lọc để giảm sai số, chuẩn bị cho chu kỳ dự báo tiếp theo.

Nhờ vậy, khi đối tượng bị che khuất khiến điểm tin cậy phát hiện giảm, bộ lọc Kalman vẫn tiếp tục dự báo quỹ đạo dựa trên quán tính, giúp hệ thống không mất dấu ID ngay lập tức; đồng thời cung cấp “vùng tìm kiếm” chính xác hơn cho cả hai bước liên kết của chiến lược BYTE.



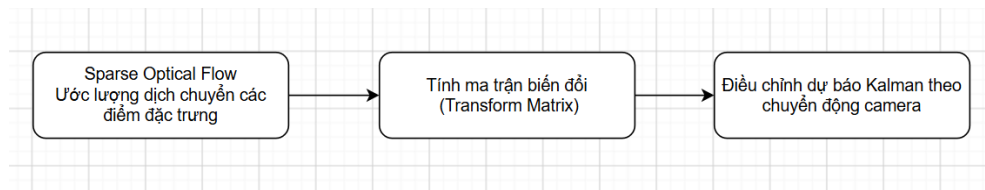
Hình 2-5: Chu trình Dự đoán - Cập nhật (Predict - Update) của bộ lọc Kalman

2.4.5 Cơ chế bù chuyển động camera (CMC)

Bộ lọc Kalman truyền thống giả định camera đứng yên tuyệt đối. Trong thực tế, camera có thể bị rung lắc do gió hoặc tác động ngoại lực, khiến tọa độ đối tượng trong khung hình thay đổi đột ngột không theo quy luật chuyển động thông thường, làm sai lệch dự báo của Kalman.

Cơ chế Bù chuyển động camera (Camera Motion Compensation) giải quyết vấn đề này qua các bước:

- Ước lượng chuyển động bằng luồng quang học thưa (Sparse Optical Flow): theo dõi các điểm đặc trưng giữa hai khung hình liên tiếp để xác định toàn bộ khung cảnh đang dịch chuyển như thế nào so với camera.
- Tính toán ma trận biến đổi: ánh xạ tọa độ từ khung hình cũ sang khung hình mới dựa trên các điểm dịch chuyển đã tìm được.
- Điều chỉnh dự báo Kalman: trước khi bộ lọc Kalman đưa ra dự báo vị trí tiếp theo, CMC áp dụng ma trận biến đổi để điều chỉnh tọa độ các track hiện tại, đảm bảo vị trí dự báo luôn khớp với thực tế dịch chuyển của camera, từ đó giúp bước tính IoU chính xác hơn.



Hình 2-6: Ba bước của cơ chế bù chuyển động camera (Camera Motion Compensation)

Cơ chế dự phòng an toàn: nếu bước ước lượng luồng quang học thất bại (môi trường quá phức tạp), hệ thống tự động trả về ma trận đồng nhất (Identity Transform, không thay đổi tọa độ) thay vì phát sinh ngoại lệ, đồng thời gọi hàm `reset_params()` để đặt lại trạng thái bộ ước lượng và tiếp tục xử lý ở khung hình kế tiếp.

2.4.6 Quản lý trạng thái và ID đối tượng (Grace Frames)

Một track không được gán sau cả hai bước liên kết sẽ không bị xóa ngay mà chuyển sang trạng thái “mất tạm thời” (lost), chỉ bị xóa hẳn sau một số khung hình ân hạn nhất định (grace frames, ví dụ 1 giây). Hệ thống lưu lại tâm điểm, tỉ lệ khung bao và vận tốc gần nhất của track trong khoảng thời gian này; nếu đối tượng xuất hiện trở lại gần vị trí dự báo của Kalman, ID cũ sẽ được tái sử dụng thay vì khởi tạo ID mới. Sau bước theo dõi, hệ thống còn thực hiện hậu xử lý loại bỏ các khung bao chồng

lớn quá mức (IoU vượt ngưỡng) cho cùng một đối tượng, làm sạch dữ liệu trước khi đưa vào tầng phân tích hành vi.

2.4.7 Ưu điểm về hiệu suất

Khác với DeepSORT - vốn cần một mạng nơ-ron Re-ID riêng biệt để trích xuất vector đặc trưng ngoại hình cho mọi khung bao - ByteTrack chủ yếu dựa trên các phép toán hình học IoU và bộ lọc Kalman. Cách tiếp cận này giảm chi phí tính toán của tầng theo dõi và tận dụng cả các khung phát hiện có điểm tin cậy thấp; thông lượng đầu cuối được đo trực tiếp theo số lượng camera và cấu hình phần cứng tại Chương 4.

Công thức IoU sử dụng xuyên suốt các bước liên kết dữ liệu của ByteTrack:

$$\text{IoU}(A, B) = |A \cap B| / |A \cup B|$$

Trong đó $|A \cap B|$ là diện tích phần giao nhau và $|A \cup B|$ là diện tích phần hợp giữa hai khung bao A, B. Thuật toán còn dùng IoU trong bước hậu xử lý để loại bỏ các khung bao trùng lặp thuộc cùng một lớp đối tượng khi tỉ lệ chồng lấn vượt ngưỡng quy định.

2.5 Thuật toán Hungarian (Kuhn–Munkres) trong bài toán gán ghép tối ưu

Ở Bước 1 của chiến lược BYTE, việc liên kết các track hiện có với tập D_{high} không được thực hiện một cách tùy tiện mà được quy về một bài toán tối ưu tổ hợp kinh điển: bài toán gán ghép (Assignment Problem). Cho hai tập hợp cùng kích thước n - track hiện có và detection mới - hệ thống xây dựng một ma trận chi phí (cost matrix) C kích thước $n \times n$, trong đó mỗi phần tử $c_{ij} = 1 - \text{IoU}(\text{track}_i, \text{detection}_j)$ thể hiện chi phí khi gán track i với detection j - giá trị càng nhỏ nghĩa là hai khung bao càng trùng khớp. Mục tiêu là tìm một phép gán ghép 1–1 hoàn chỉnh (mỗi track khớp đúng một detection và ngược lại) sao cho tổng chi phí trên toàn ma trận là nhỏ nhất - đây chính là bài toán mà thuật toán Hungarian (còn gọi là Kuhn–Munkres) giải quyết tối ưu trong thời gian đa thức.

Vì sao không thể gán ghép theo kiểu tham lam (greedy): một cách làm đơn giản là luôn chọn cặp track–detection có IoU cao nhất trước, rồi loại hàng/cột tương ứng và lặp lại. Cách này gọi là greedy matching. Vấn đề là greedy chỉ tối ưu cục bộ ở từng bước chứ không đảm bảo tổng chi phí toàn cục là nhỏ nhất: khi một detection tốt bị "tranh chấp" bởi nhiều track, việc gán nó cho track có IoU cao nhất ngay từ đầu có thể đẩy các track còn lại vào thế phải nhận những detection rất tệ, trong khi một phương án hoán đổi khác - dù từng cặp trông "kém tối ưu" hơn - lại cho tổng chất lượng gán ghép cao hơn hẳn. Thuật toán Hungarian tránh được bẫy này bằng cách xét đồng thời toàn bộ ma trận thay vì quyết định tuần tự.

Các bước chính của thuật toán (độ phức tạp $O(n^3)$):

- **Trừ hàng:** với mỗi hàng của ma trận chi phí, trừ toàn hàng cho giá trị nhỏ nhất của hàng đó (đảm bảo mỗi hàng có ít nhất một số 0).
- **Trừ cột:** tương tự, trừ mỗi cột cho giá trị nhỏ nhất của cột đó.
- **Phủ số 0:** dùng số đường thẳng (ngang/dọc) ít nhất có thể để phủ hết toàn bộ số 0 trong ma trận. Nếu số đường thẳng cần dùng bằng đúng n (kích thước ma trận) $\rightarrow$ đã có thể tìm được phép gán tối ưu, chuyển sang Bước 5.
- **Điều chỉnh:** nếu số đường thẳng $< n$, tìm giá trị nhỏ nhất trong các phần tử chưa bị phủ; trừ giá trị đó khỏi mọi phần tử chưa bị phủ, đồng thời cộng nó vào các phần tử nằm ở giao điểm của hai đường phủ; quay lại Bước 3.
- **Chọn gán ghép:** chọn ra n số 0 “độc lập” (mỗi hàng và mỗi cột chỉ chứa đúng một số 0 được chọn) - đây chính là phép gán ghép tối ưu ứng với ma trận chi phí gốc.

Trong thực nghiệm, các cài đặt hiện đại (ví dụ biến thể Jonker–Volgenant) đạt độ phức tạp $O(n^3)$, đủ nhanh để xử lý vài chục đến vài trăm track mỗi khung hình

trong thời gian thực. Mã nguồn hệ `scipy.optimize.linear_sum_assignment` của thư viện SciPy - hoặc hàm tương đương trong `lap/lapjv` - thường được dùng trực tiếp để giải bước liên kết này mà không cần cài đặt lại thuật toán từ đầu.

Ví dụ số minh họa: xét 3 track (A, B, C) đang được theo dõi và 3 detection mới (D1, D2, D3) ở khung hình hiện tại, với ma trận độ trùng khớp IoU như sau

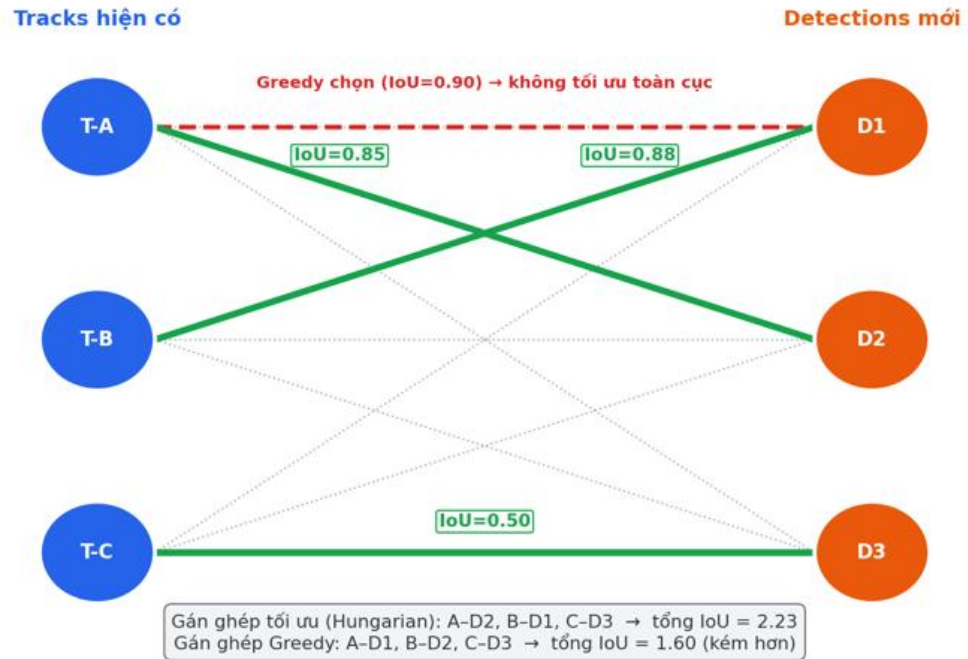
IoU	Det 1	Det 2	Det 3
Track A	0.90	0.85	0.10
Track B	0.88	0.20	0.10
Track C	0.10	0.10	0.50

Bảng 2-2: Ma trận IoU giữa các track hiện có và các detection mới

Nếu gán theo kiểu greedy (luôn chọn IoU cao nhất trước), hệ thống sẽ chọn ngay cặp A–D1 (IoU = 0.90), sau đó B chỉ còn có thể nhận D2 (IoU = 0.20) vì D1 đã bị chiếm, và C nhận D3 (IoU = 0.50) - tổng IoU đạt được là $0.90 + 0.20 + 0.50 = 1.60$. Trong khi đó, thuật toán Hungarian xét toàn bộ ma trận và tìm ra phương án A–D2 (0.85), B–D1 (0.88), C–D3 (0.50), cho tổng IoU = 2.23 - cao hơn hẳn so với greedy, dù từng cặp riêng lẻ (0.85 hay 0.88) không phải là giá trị lớn nhất theo hàng của nó. Bảng dưới đây tóm tắt sự khác biệt

Chiến lược	Phép gán	Tổng IoU
Greedy (tham lam)	A–D1, B–D2, C–D3	1.60
Hungarian (tối ưu)	A–D2, B–D1, C–D3	2.23

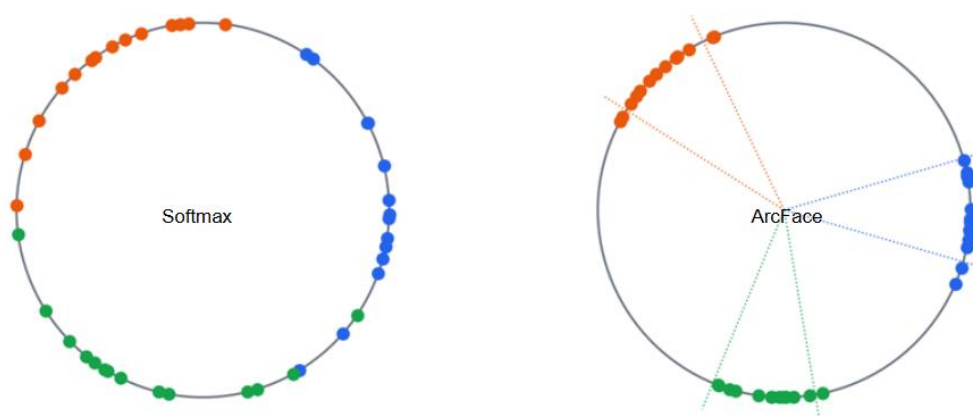
Bảng 2-3: So sánh kết quả gán ghép giữa chiến lược Greedy và thuật toán Hungarian



Hình 2-7: Bài toán gán ghép lưỡng phân giữa track và detection

Nhờ đảm bảo tính tối ưu toàn cục thay vì quyết định vội vàng theo từng cặp riêng lẻ, thuật toán Hungarian giúp giảm thiểu hiện tượng nhảy ID (ID switch) khi nhiều đối tượng di chuyển gần nhau hoặc che khuất lẫn nhau - đúng như vai trò của nó trong Bước 1 (High-score association) của chiến lược BYTE đã trình bày ở trên.

2.6 Nhận diện khuôn mặt dựa trên học biểu diễn (Representation Learning) với ArcFace



Hình 2-8: So sánh không gian đặc trưng giữa Softmax truyền thống và ArcFace

Thay vì phân loại danh tính trực tiếp vào một danh sách nhãn cố định (như Softmax truyền thống), hệ thống hiện đại học một hàm ánh xạ mỗi ảnh khuôn mặt thành một vector đặc trưng (embedding) trong không gian nhiều chiều - cách tiếp cận này cho phép nhận diện người mới chỉ bằng cách so sánh vector với ảnh tham chiếu, mà không cần huấn luyện lại toàn bộ mô hình khi có danh tính mới.

Hàm mất mát ArcFace (Additive Angular Margin Loss) là nền tảng của bộ công cụ InsightFace được hệ thống sử dụng. ArcFace đưa thêm một biên góc cộng tính (additive angular margin) vào quá trình huấn luyện, buộc mô hình tối đa hóa khoảng cách góc giữa các danh tính khác nhau trên một không gian đặc trưng đã chuẩn hóa (thường trên một mặt cầu). Kết quả là vector đặc trưng của cùng một người tụ lại rất chặt chẽ, trong khi vector của những người khác nhau bị đẩy ra xa nhau tối đa, tạo ra tính phân biệt (discriminative power) cao hơn hẳn so với Softmax.

Độ tương đồng Cosine: sau khi trích xuất vector đặc trưng quan sát và vector tham chiếu, hệ thống tính cosine similarity theo công thức (2), với giá trị trong khoảng $[-1, 1]$. Phiên bản được đánh giá dùng ngưỡng cầu hình 0,45; các operating point 0,35; 0,40; 0,45 được kiểm tra bằng pilot ở mục 4.7. Ngưỡng này không được xem là tối

ưu toàn cục và cần được xác thực lại khi thay đổi người dùng, camera hoặc điều kiện ánh sáng.

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|} \quad (2)$$

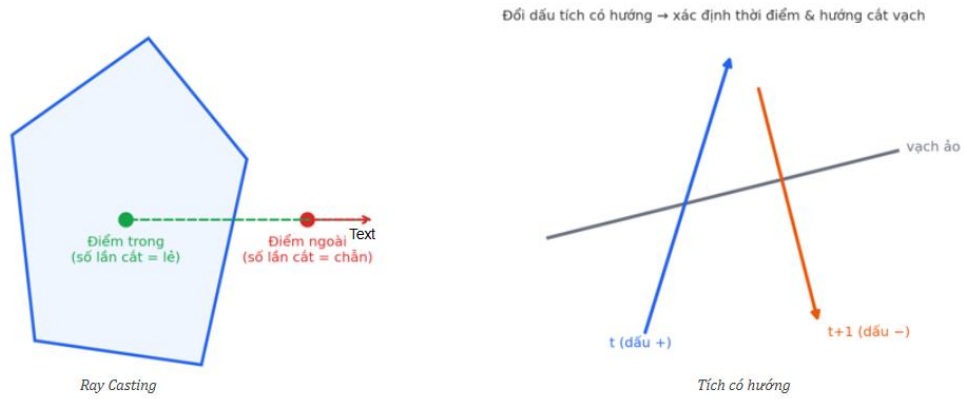
Vì sao ArcFace vượt trội hơn Softmax:

- Chuyển từ phân loại sang học biểu diễn: Softmax được thiết kế cho bài toán phân loại vào các nhãn cố định - khi có người mới, về lý thuyết mạng cần huấn luyện lại để thêm lớp mới. ArcFace học một không gian embedding chung nên không cần huấn luyện lại khi bổ sung danh tính mới.
- Tối đa hóa khoảng cách góc: Softmax chỉ tách biệt các lớp đủ để không nhầm lẫn, không bắt buộc vector cùng một người phải gần nhau tối đa hay vector của người khác phải xa nhau tối đa. ArcFace khắc phục điều này bằng biên góc cộng tính, tạo ra “siêu biên giới” phân biệt giữa các khuôn mặt.
- Hiệu quả trong môi trường thực tế: nhờ tính phân biệt cao, ArcFace giúp thiết lập một ngưỡng Cosine ổn định để phân biệt chính xác người quen/người lạ ngay cả khi ánh sáng thay đổi hoặc góc nhìn camera không thuận lợi, đồng thời xử lý tốt với số lượng người lớn.

Hiệu năng triển khai: trong kiến trúc pipeline nhiều tầng của hệ thống, tầng nhận diện danh tính dựa trên ArcFace (qua InsightFace) hoạt động với độ trễ rất thấp, khoảng 0,12 ms ở mức trung vị. Để tránh tranh chấp tài nguyên GPU với YOLOv11, module này thường được cấu hình chạy suy luận trên CPU (qua onnxruntime) mà vẫn đảm bảo tính thời gian thực.

2.7 Cơ sở hình học tính toán trong phân tích hành vi

Để biến các tọa độ thô thành “ngữ cảnh” hành vi, hệ thống không “đoán” mà áp dụng các nguyên lý hình học nền tảng, xác thực bằng toán học.



Hình 2-9: Hai kỹ thuật hình học tính toán dùng trong phân tích hành vi

Thuật toán Tia (Ray Casting) - kiểm tra điểm trong đa giác (Point-in-Polygon): dùng để xác định tâm đối tượng có nằm trong một vùng cấm/vùng giám sát (đa giác ROI) hay không. Từ điểm cần kiểm tra, hệ thống vẽ một tia giả tưởng kéo dài đến vô cực (thường song song trục hoành để tối ưu tính toán) và đếm số lần tia cắt các cạnh của đa giác: nếu số lần cắt là số lẻ, điểm nằm bên trong đa giác; nếu là số chẵn, điểm nằm bên ngoài. Trong mã nguồn hệ thống, thuật toán được hiện thực qua hàm `cv2.pointPolygonTest` của OpenCV. Đây là nền tảng kỹ thuật cho hai luật cảnh báo chính:

- Xâm nhập (Intrusion) - phát cảnh báo khi tâm đối tượng chuyển trạng thái từ “ngoài” vào “trong” vùng cấm.
- Lảng vảng (Loitering) - theo dõi thời gian đối tượng duy trì trạng thái “bên trong” đa giác vượt quá một ngưỡng quy định.

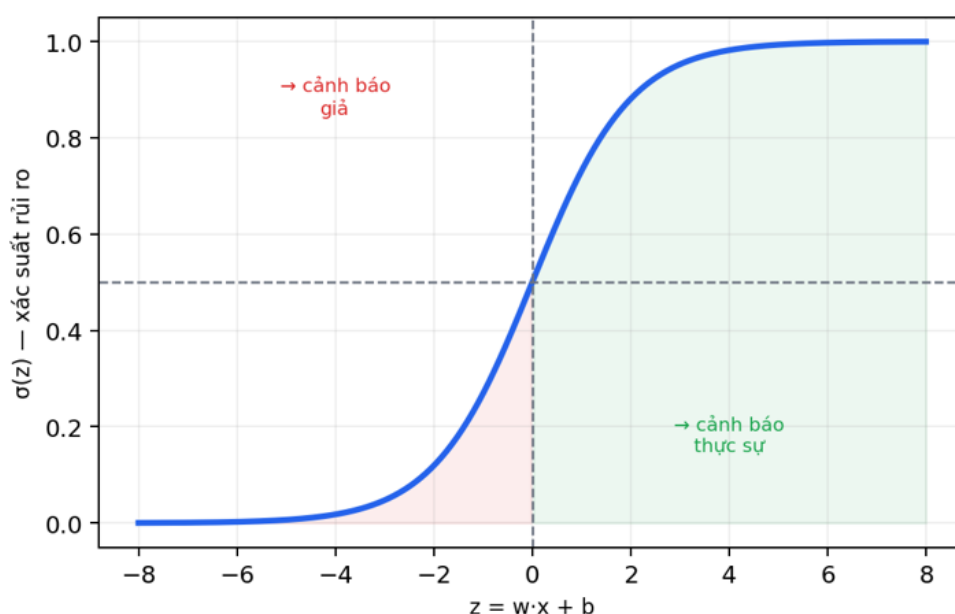
Tích có hướng (Signed Cross Product) - phát hiện cắt vạch: dùng để xác định hướng di chuyển (vào/ra) khi đối tượng đi qua một vạch ảo. Giá trị tích có hướng được tính theo công thức (3); hệ thống theo dõi dấu của giá trị này qua hai khung hình liên tiếp, trong đó sự đổi dấu cho biết thời điểm đối tượng cắt qua vạch và hướng di chuyển tương ứng.

$$side(p) = (x_{l2} - x_{l1})(y_p - y_{l1}) - (y_{l2} - y_{l1})(x_p - x_{l1}) \quad (3)$$

Hiệu suất: tăng phân tích hành vi dựa trên hình học hoạt động cực kỳ hiệu quả, chi phí tính toán gần như không đáng kể (tổng thời gian xử lý tầng Analytics chỉ khoảng 1,15 ms), đảm bảo tính thời gian thực cho toàn hệ thống.

2.8 Mô hình hồi quy Logistic và chấm điểm rủi ro

Để giảm thiểu cảnh báo giả (false alarms) - vấn đề lớn của camera giám sát truyền thống dựa trên luật cứng nhắc - hệ thống bổ sung một lớp học máy có giám sát cho bài toán phân loại nhị phân, giúp thoát khỏi các luật cố định và đưa ra quyết định có tính xác suất.



Hình 2-10: Hàm Sigmoid ánh xạ điểm số tuyến tính z về xác suất rủi ro trong khoảng $(0,1)$

Vì sao chọn Hồi quy Logistic: mô hình được ưu tiên hơn các mạng học sâu (deep learning) phức tạp nhờ ba đặc tính: (1) *đơn giản và nhẹ* - chỉ mất khoảng 1,15 ms để xử lý một cảnh báo, duy trì thông lượng thời gian thực; (2) *tính minh bạch/dễ hiểu được (interpretability)* - khác với các mô hình “hộp đen”, người vận hành có thể truy vết ngược để hiểu tại sao một hành vi bị coi là rủi ro dựa trên trọng số đóng góp của từng đặc trưng đầu vào (ví dụ đặc trưng “thời gian đứng yên” có trọng số cao sẽ giải thích vì sao hệ thống phát cảnh báo “người lạ khả nghi”); (3) *hiệu quả cho phân*

loại nhị phân - phù hợp để xác định một sự kiện là “cảnh báo đúng” hay “cảnh báo giả”.

Vector đặc trưng đầu vào (22 chiều): mỗi cảnh báo ứng viên từ tầng phân tích hành vi được chuyển thành một vector đặc trưng số gồm: - Tỷ lệ thời gian lưu trú và tỉ lệ quãng đường di chuyển; tổng độ dịch chuyển theo đường chéo khung hình. - Độ tin cậy phát hiện của YOLOv11 và tỉ lệ diện tích khung bao đối tượng. - Số lần đối tượng đi qua lại (pacing) trong vùng giám sát. - Các chỉ báo one-hot về lớp đối tượng (người/xe) và loại vùng/vạch đã cấu hình.

Công thức tính xác suất bằng hàm Sigmoid: với vector đặc trưng x , vector trọng số w và độ chệch (bias) b , mô hình ước lượng xác suất rủi ro theo công thức (4), ánh xạ một giá trị thực bất kỳ về khoảng $(0, 1)$. Giá trị càng gần 1, hành vi càng có khả năng là mối đe dọa thực sự.

$$p = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} \quad (4)$$

Huấn luyện và tối ưu hóa: các tham số w, b được tối ưu bằng cách tối thiểu hóa hàm mất mát entropy chéo nhị phân (Binary Cross-Entropy) kết hợp chính quy hóa L2 để chống quá khớp khi dữ liệu gán nhãn còn ít, theo công thức (5).

$$L(w, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log(1 - p_i)] + \frac{\lambda}{2} \|w\|^2 \quad (5)$$

Việc tối thiểu hóa hàm mất mát L được thực hiện bằng phương pháp hạ gradient (gradient descent), lặp lại việc cập nhật tham số theo hướng ngược chiều đạo hàm với một tốc độ học η cho đến khi hội tụ:

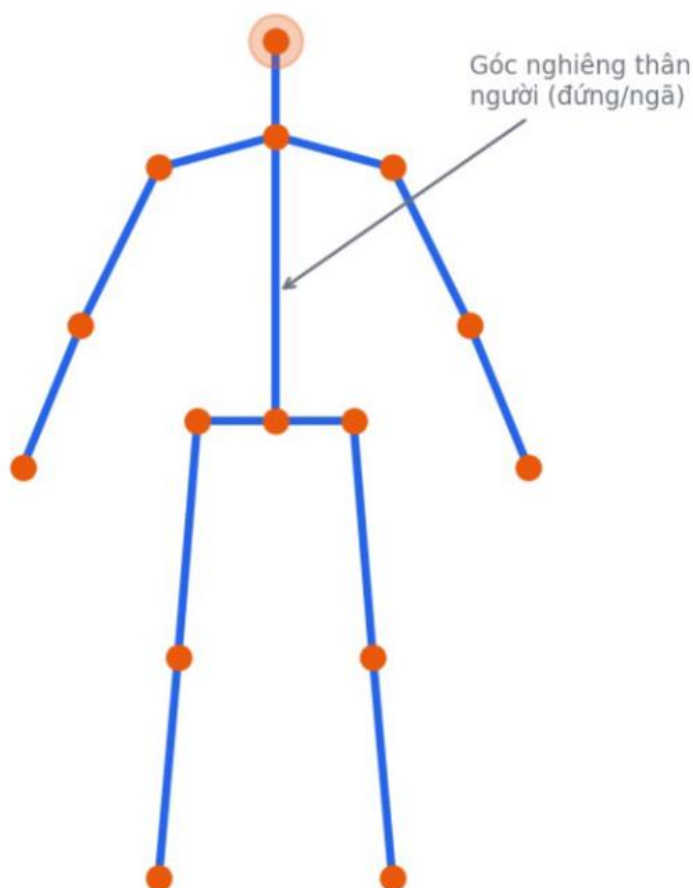
$$w \leftarrow w - \eta \nabla_w L, \quad b \leftarrow b - \eta \nabla_b L \quad (6)$$

Triển khai thực tế:

- **Chuẩn hóa:** các đặc trưng đầu vào được chuẩn hóa theo trung bình và độ lệch chuẩn trước khi tính toán.
- **Giảm cảnh báo giả:** người vận hành có thể thiết lập ngưỡng rủi ro tùy chỉnh; hệ thống tự động lọc bỏ hoặc chỉ hiển thị tham khảo các cảnh báo có điểm thấp hơn ngưỡng, giảm hiện tượng “mệt mỏi vì cảnh báo”.
- **Cập nhật linh hoạt (Hot-reload):** mô hình hỗ trợ nạp lại trọng số mới từ đĩa mà không cần khởi động lại toàn bộ hệ thống, cho phép học liên tục từ nhãn mới do người dùng phản hồi (dán nhãn thủ công qua CSV hoặc API).

Kết quả thực nghiệm cho thấy tầng xử lý này hoạt động cực kỳ hiệu quả, chỉ chiếm khoảng 1,15 ms trong tổng thể luồng AI của hệ thống, giúp biến các quy tắc hình học cứng nhắc thành các quyết định có tính toán xác suất, mang lại sự cân bằng giữa độ nhạy và độ chính xác cho cảnh báo an ninh.

2.9 Mô hình ước lượng tư thế (YOLOv11-Pose) trong pipeline phân tích hành vi



Hình 2-11: Các điểm khớp xương (keypoints) được YOLOv11-Pose ước lượng, dùng để phân tích tư thế và hành vi

Việc sử dụng mô hình Pose bên cạnh mô hình phát hiện đối tượng YOLO11s là cần thiết vì mỗi mô hình đảm nhận một vai trò khác nhau: YOLO11s đóng vai trò “mắt” để thấy đối tượng, còn Pose đóng vai trò “não bộ” để hiểu chi tiết hành động và tư thế - điều mà một khung bao (bounding box) đơn thuần không thể làm được.

Các vai trò cụ thể của tầng Pose: Phát hiện trạng thái nguy hiểm (ví dụ người ngã): trong khi YOLO11s chỉ xác định có người trong khung hình, mô hình Pose cung cấp các điểm khớp xương để tính toán góc nghiêng của thân người, giúp phân biệt

chính xác giữa việc người đó đang nằm/ngã với việc khung bao chỉ bị biến dạng do góc nhìn. - Xác định tương tác chi tiết với tài sản (phòng chống trộm cắp): kiểm tra xem tín hiệu tiếp xúc từ tư thế cổ tay hoặc khung xương người có chạm vào vùng phương tiện hay không (tín hiệu “contact”), tăng độ chính xác cho cảnh báo thay vì chỉ dựa vào việc người đứng gần. - Phân loại tư thế và trạng thái vận động: đánh giá các hành vi phức tạp như đứng yên lâu (standing still) hoặc đi qua lại (pacing); kết hợp thông tin quỹ đạo với tư thế giúp đưa ra lý do cảnh báo có ngữ cảnh rõ ràng hơn cho người vận hành.

Tối ưu hóa xử lý: Tầng ước lượng tư thế là một giai đoạn riêng biệt trong pipeline (chiếm khoảng 18,79 ms ở mức trung vị), chỉ chạy cho các đối tượng đã được xác định là “người” (person) sau khi qua tầng phát hiện và theo dõi. Việc tách biệt này giúp tiết kiệm tài nguyên tính toán nhưng vẫn đảm bảo khả năng phân tích hành vi chuyên sâu, đồng thời việc dùng một mô hình phát hiện YOLO11s cho phép dành tài nguyên cho các tầng Pose, nhận diện khuôn mặt và phân tích hành vi hình học mà không nghẽn toàn bộ luồng xử lý.

2.10 Các độ đo đánh giá hệ thống (Evaluation Metrics)

Để đánh giá tính hiệu quả một cách khách quan, hệ thống dựa trên các chuẩn mực quốc tế:

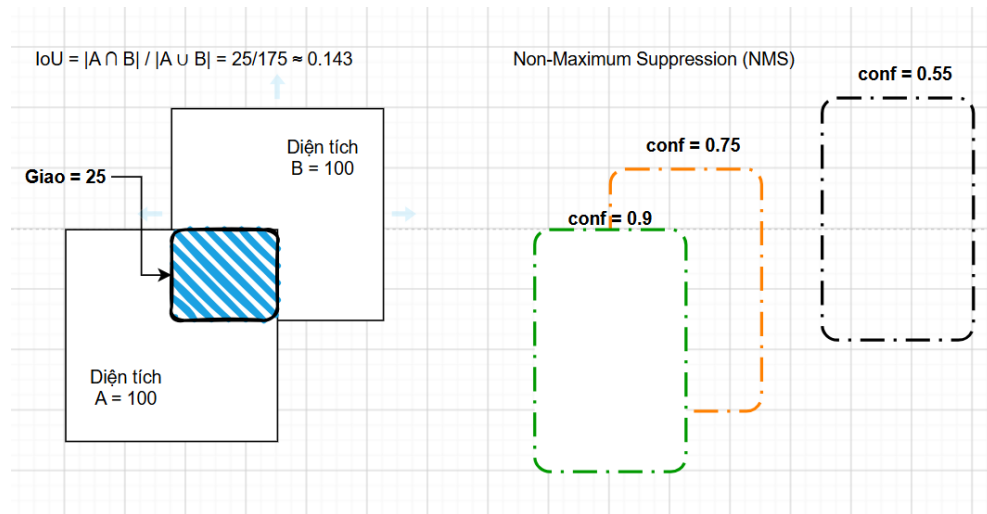
- MOTA (Multiple Object Tracking Accuracy): đo lường độ chính xác tổng thể của phát hiện và theo dõi, dựa trên số lượng nhận nhầm, bỏ sót và đổi ID.
- IDF1 (Identity F1 Score): tập trung vào khả năng duy trì danh tính nhất quán xuyên suốt video.
- HOTA (Higher Order Tracking Accuracy): độ đo hiện đại nhất, cân bằng giữa độ chính xác phát hiện, định vị và liên kết.

- FPS & Độ trễ (Latency, p50/p95/p99): phản ánh khả năng đáp ứng thời gian thực và độ ổn định xử lý của hệ thống trên phần cứng đầu cuối.

Bảng 2-4: Tóm tắt các độ đo đánh giá hệ thống MOT

Độ đo	Ý nghĩa
MOTA	Multiple Object Tracking Accuracy — đo độ chính xác tổng thể dựa trên số lượng nhận nhầm (FP), bỏ sót (FN) và đổi ID (ID switch); tập trung chủ yếu vào chất lượng phát hiện.
IDF1	Identity F1 Score — đo khả năng duy trì đúng danh tính của đối tượng xuyên suốt video, phản ánh chất lượng của bước liên kết dữ liệu hơn là chất lượng phát hiện.
HOTA	Higher Order Tracking Accuracy — cân bằng rõ ràng giữa độ chính xác phát hiện, liên kết và định vị, được đề xuất để khắc phục thiên lệch của MOTA về phía phát hiện.
FPS	Frames Per Second — số khung hình xử lý được trong một giây, phản ánh khả năng đáp ứng thời gian thực của toàn hệ thống.
Độ trễ (p50/p95/p99)	Thời gian xử lý một khung hình tại các mức phân vị 50%, 95%, 99%, giúp đánh giá không chỉ tốc độ trung bình mà cả các trường hợp xử lý chậm nhất.

2.11 Cơ chế Object Detection: các khái niệm cốt lõi



Hình 2-12: Minh họa IoU giữa hai khung bao và cơ chế lọc trùng lặp Non-Maximum Suppression (NMS)

Bounding Box: khung chữ nhật bao quanh vật thể, biểu diễn phổ biến nhất là $(x_center, y_center, width, height)$ - tọa độ tâm và kích thước, thường chuẩn hóa về khoảng $[0,1]$ theo kích thước ảnh. Ví dụ: box $(0.5, 0.5, 0.2, 0.3)$ trên ảnh 640×640 nghĩa là tâm tại pixel $(320, 320)$, rộng 128px, cao 192px.

Confidence Score và Class Probability: Confidence Score là độ tin cậy rằng box thực sự chứa một vật thể (objectness) nhân với xác suất lớp cao nhất - điểm số cuối dùng để lọc/threshold. Ví dụ: objectness = 0.95, xác suất lớp cao nhất (person) = 0.9 $\rightarrow$ confidence hiển thị ≈ 0.855 .

IoU (Intersection over Union): đo độ “trùng khớp” giữa hai box (1 là trùng hoàn toàn, 0 là không chạm nhau).

$$\text{IoU} = (\text{Diện tích giao nhau}) / (\text{Diện tích hợp nhau})$$

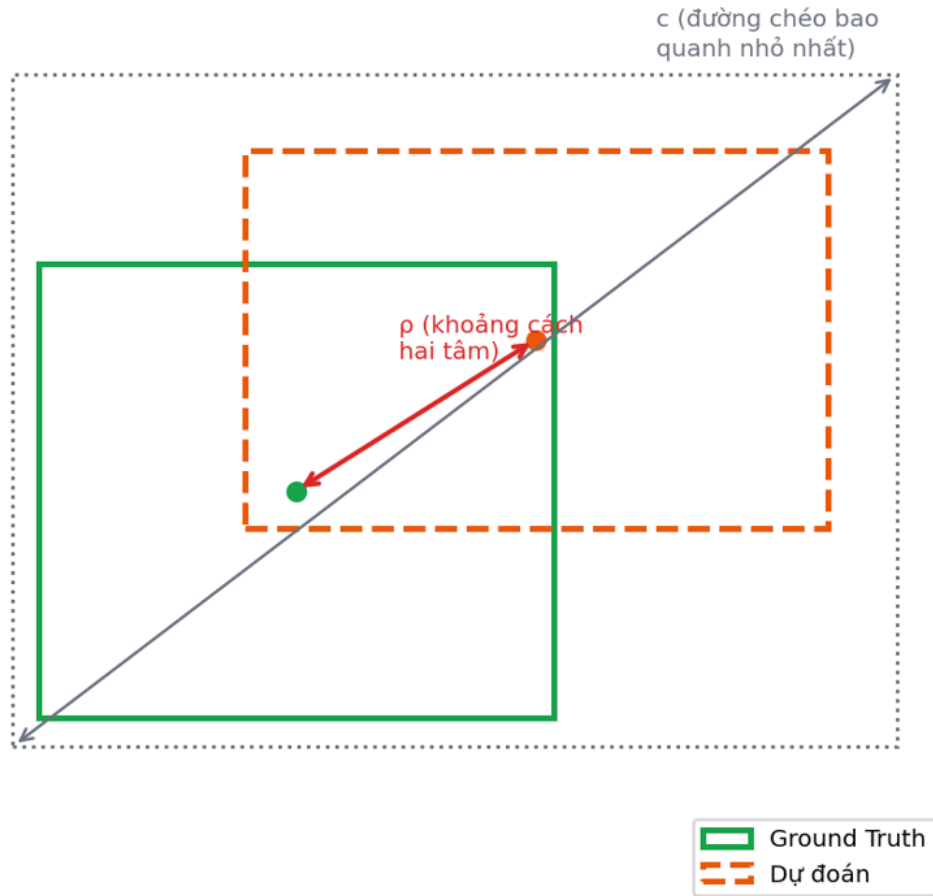
Ví dụ số: Box A $(0,0,10,10)$ diện tích 100; Box B $(5,5,15,15)$ diện tích 100; vùng giao nhau $(5,5)-(10,10) = 25$; vùng hợp $= 100+100-25 = 175 \rightarrow \text{IoU} = 25/175 \approx 0.143$.

NMS (Non-Maximum Suppression): khi mạng dự đoán nhiều box gần giống nhau cho cùng một vật thể, NMS chỉ giữ box “tốt nhất” (confidence cao nhất) và loại các box còn lại trùng lặp nhiều. Thuật toán: (1) lọc bỏ các box có confidence dưới ngưỡng (ví dụ 0.25); (2) chọn box confidence cao nhất còn lại, giữ vào danh sách kết quả; (3) tính IoU giữa box này với toàn bộ box còn lại cùng lớp, xóa box nào có IoU vượt ngưỡng (ví dụ 0.45); (4) lặp lại cho đến khi không còn box nào. Lưu ý: YOLOv10 và YOLO26 dùng chiến lược huấn luyện NMS-free (one-to-one assignment) để mỗi vật thể chỉ có đúng một box dự đoán ngay từ đầu, bỏ hẳn bước NMS ở suy luận, giảm độ trễ.

Anchor-based và Anchor-free: *Anchor-based (YOLOv2–v7):* định nghĩa trước tập khung mẫu (anchor) có tỉ lệ phổ biến (qua k-means clustering trên tập dữ liệu); mạng chỉ học độ lệch (offset) so với anchor. Nhược điểm: phụ thuộc việc chọn tham số anchor thủ công. - *Anchor-free (YOLOv6, v8–v11, YOLO26):* bỏ hoàn toàn khung mẫu, mỗi điểm trên feature map trực tiếp dự đoán khoảng cách đến 4 cạnh của vật thể (hoặc offset tâm). Ưu điểm: đơn giản hóa pipeline, giảm siêu tham số cần tinh chỉnh, tổng quát hóa tốt hơn với vật thể có tỉ lệ bất thường.

Multi-scale Detection: Head dự đoán trên nhiều feature map có stride khác nhau (ví dụ 8/16/32) - độ phân giải cao (stride nhỏ) phù hợp vật thể nhỏ, độ phân giải thấp (stride lớn, receptive field rộng) phù hợp vật thể lớn, giúp mạng không bỏ sót vật thể ở nhiều kích cỡ khác nhau [23].

2.12 Loss Function (hàm mất mát)



Hình 2-13: Các thành phần hình học được CIOU Loss xem xét: độ chồng lấp IoU

Loss function là “thước đo sai số”, cho mạng biết nó dự đoán sai bao nhiêu so với nhãn thật (ground truth); thuật toán tối ưu (SGD/Adam/AdamW) dùng gradient của loss để cập nhật trọng số qua lan truyền ngược (backpropagation), giảm dần loss về mức tối thiểu qua các epoch.

Tổng loss của các phiên bản YOLO hiện đại thường là tổ hợp có trọng số của các thành phần:

$$\text{Loss_total} = \lambda_{\text{box}} \cdot \text{Loss_box} + \lambda_{\text{cls}} \cdot \text{Loss_cls} + \lambda_{\text{obj}} \cdot \text{Loss_obj} (+ \lambda_{\text{dfl}} \cdot \text{Loss_DFL})$$

Localization / Box Loss: đo độ lệch vị trí giữa khung dự đoán và ground truth. YOLOv1–v3 dùng MSE (Mean Squared Error, không phản ánh đúng hình thái IoU);

từ YOLOv4 trở đi dùng CIoU (Complete IoU) hoặc các biến thể GIoU/DIoU/MPDIoU:

$$\text{Loss_CIoU} = 1 - \text{IoU} + \rho^2(b, b_{gt})/c^2 + \alpha \cdot v$$

Trong đó ρ^2 là khoảng cách Euclidean bình phương giữa hai tâm box, c là đường chéo của hình chữ nhật nhỏ nhất bao quanh cả hai box, v đo độ lệch tỉ lệ khung hình (aspect ratio), α là hệ số cân bằng. Ví dụ: nếu $\text{IoU}=0.7$, hai tâm gần trùng nhau ($\rho^2/c^2 \approx 0.02$), tỉ lệ khung hình gần giống nhau ($v \approx 0$) $\rightarrow \text{Loss_CIoU} \approx 0.32$.

Classification Loss: đo sai số phân loại vật thể, thường dùng Binary Cross-Entropy (BCE) hoặc Varifocal Loss:

$$\text{Loss_cls} = -\sum [y \cdot \log(p) + (1-y) \cdot \log(1-p)]$$

với y là nhãn thật (0/1), p là xác suất dự đoán. Ví dụ: nhãn thật “person” ($y=1$), mô hình dự đoán $p=0.8 \rightarrow \text{Loss} \approx 0.223$; nếu $p=0.99 \rightarrow \text{Loss} \approx 0.01$.

Objectness Loss: đo độ tin cậy rằng một vị trí/anchor thực sự chứa vật thể hay chỉ là nền, thường dùng BCE giữa objectness dự đoán và nhãn 0/1 (hoặc IoU giữa box dự đoán và ground truth làm target mềm).

DFL (Distribution Focal Loss) - dùng từ YOLOv8, giữ ở YOLOv11, loại bỏ ở YOLO26: thay vì hồi quy tọa độ như một số thực đơn lẻ, DFL biểu diễn mỗi cạnh box dưới dạng phân bố xác suất rời rạc trên tập giá trị khả dĩ, giúp mô hình học được độ “không chắc chắn” về vị trí biên, cải thiện độ chính xác với vật thể có biên mờ hoặc khó xác định.

2.13 Xử lý các vấn đề thực tế thường gặp

Bảng 2-5: Các vấn đề thực tế thường gặp và cách xử lý

Vấn đề	Cách cải thiện
Người/vật bị che khuất (Occlusion)	Dùng ByteTrack để tận dụng các box điểm thấp (low-score); khi huấn luyện, tăng cường dữ liệu mạnh bằng Mosaic/Mixup

Vấn đề	Cách cải thiện
Vật thể nhỏ / ở xa	Tăng độ phân giải đầu vào (ví dụ <code>imgsz=1280</code>); dùng kỹ thuật cắt ảnh SAHI (Slicing Aided Hyper Inference); thêm nhánh Detection Head riêng cho vật thể nhỏ ở feature map độ phân giải cao nhất
Ánh sáng yếu / ban đêm	Áp dụng tiền xử lý CLAHE (Contrast Limited Adaptive Histogram Equalization) hoặc mô hình tăng cường ảnh trước khi đưa qua YOLO; tăng cường dữ liệu huấn luyện bằng chỉnh độ sáng/độ tương phản/nhiều màu (Hue, Saturation, Value)
Camera rung	Dùng tracker có bù chuyển động như BoT-SORT (xem CMC ở mục 2.3.4)
ID Switch (nhầm đổi tượng A thành B)	Tăng Track Buffer hoặc dùng model Re-ID mạnh hơn

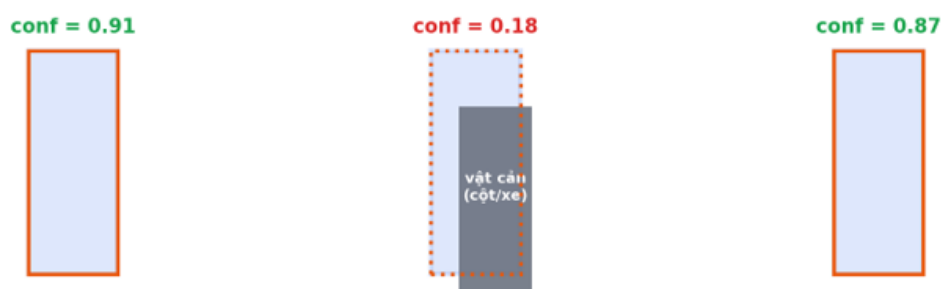
2.13.1 Người / vật bị che khuất (Occlusion)

Khái niệm: Occlusion (che khuất) xảy ra khi một phần hoặc toàn bộ đối tượng đang được theo dõi bị vật thể khác (cột đèn, xe cộ, đối tượng khác, khung cửa...) che lấp khỏi tầm nhìn của camera trong một hoặc nhiều khung hình liên tiếp. Đây là một trong những thách thức kinh điển và phổ biến nhất của bài toán MOT trong môi trường thực tế, vì camera giám sát hầu như luôn hoạt động trong không gian có nhiều vật cản (cây cối, phương tiện, đám đông).

Khi đối tượng bị che khuất một phần, các đặc trưng thị giác (hình dạng, màu sắc, cạnh viền) mà YOLO dùng để nhận diện bị mất đi đáng kể, khiến điểm tin cậy (confidence score) của khung bao sọt giảm mạnh - có thể từ mức trên 0.8–0.9 xuống dưới 0.2–0.3 chỉ trong một khung hình. Nếu hệ thống dùng một ngưỡng lọc cố định (ví dụ chỉ giữ box có điểm > 0.5 như SORT/DeepSORT truyền thống), toàn bộ các khung bao điểm thấp này sẽ bị loại bỏ ngay lập tức, khiến track "biến mất" khỏi hệ thống dù đối tượng vẫn còn hiện diện trong khung hình.

Ảnh hưởng: Hậu quả trực tiếp là hiện tượng nhảy ID (ID switch): khi đối tượng xuất hiện trở lại sau khi bị che khuất, hệ thống không còn liên kết được với track cũ

nên phải khởi tạo một ID hoàn toàn mới. Điều này làm gây quỹ đạo di chuyển, sai lệch các số liệu thống kê hành vi (thời gian lưu trú, quãng đường di chuyển), và trong trường hợp giám sát an ninh, có thể khiến hệ thống bỏ sót một đối tượng đáng ngờ chỉ vì ID của họ bị đổi giữa chừng



Hình 2-14: Confidence score sụt giảm tạm thời khi đối tượng bị che khuất một phần, sau đó phục hồi khi đối tượng xuất hiện trở lại

Ví dụ minh họa: một người đi bộ băng qua khu vực giám sát, đi ngang qua một cột đèn đường. Ở khung hình t , confidence = 0.91 (rõ ràng, dễ nhận diện); tại khung hình $t+1$, người này bị cột đèn che khuất khoảng 60% diện tích cơ thể, confidence tụt xuống chỉ còn 0.18 - thấp hơn nhiều so với ngưỡng lọc truyền thống (thường 0.5–0.6); đến khung hình $t+2$, người này bước ra khỏi vùng che khuất, confidence phục hồi về 0.87. Nếu hệ thống dùng ngưỡng cố định, box ở khung $t+1$ sẽ bị loại bỏ hoàn toàn, khiến track bị ngắt quãng ngay tại thời điểm quan trọng nhất.

Giải pháp: ByteTrack giải quyết vấn đề này bằng chiến lược liên kết hai bước BYTE (xem chi tiết mục 2.4.3 của báo cáo chính): thay vì loại bỏ ngay các box điểm thấp, hệ thống vẫn giữ lại chúng trong tập D_{low} và cố gắng liên kết chúng với các track chưa được gán ở bước ưu tiên, chỉ dựa trên độ đo hình học IoU (không cần đặc trưng ngoại hình). Nhờ vậy, track vẫn được duy trì xuyên suốt giai đoạn che khuất thay vì bị ngắt và cấp ID mới. Ngoài ra, trong giai đoạn huấn luyện, các kỹ thuật tăng cường dữ liệu như Mosaic (ghép 4 ảnh) và Mixup giúp mô hình "làm quen" với các tình huống vật thể bị chồng lấp/che khuất một phần, cải thiện khả năng tổng quát hóa.

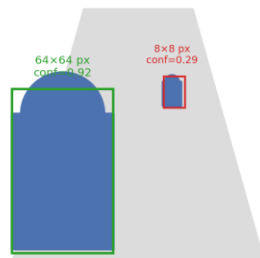
2.13.2 Vật thể nhỏ / ở xa

Khái niệm: Đây là thách thức xảy ra khi đối tượng cần phát hiện chiếm một diện tích rất nhỏ trong khung hình - thường do ở xa camera, có kích thước vật lý nhỏ, hoặc do góc quay rộng (wide-angle) khiến toàn cảnh bị thu nhỏ. Trong các hệ thống camera giám sát lắp ở vị trí cao (góc nhìn từ trên xuống, bao quát khu vực rộng), phần lớn đối tượng ở nửa trên khung hình thường chỉ chiếm vài chục đến vài trăm pixel - rất nhỏ so với độ phân giải ảnh đầu vào.

YOLO dự đoán trên nhiều feature map ở các tỉ lệ khác nhau (thường là stride 8, 16, 32 - tương ứng kích thước lưới 80×80 , 40×40 , 20×20 với ảnh đầu vào 640×640). Vật thể càng nhỏ thì càng cần được phát hiện ở feature map có độ phân giải cao (lưới mịn, stride nhỏ) vì ở lưới thô, một vật thể nhỏ có thể chỉ chiếm một phần rất nhỏ của một ô lưới - khiến mạng khó phân biệt vật thể với nhiễu nền, dẫn đến bỏ sót phát hiện (False Negative).

Ảnh hưởng : Vật thể nhỏ bị bỏ sót nghĩa là hệ thống hoàn toàn không biết đến sự tồn tại của đối tượng đó - không có track, không có cảnh báo, dù đối tượng có thể đang thực hiện hành vi bất thường. Đây là loại lỗi nguy hiểm hơn cả nhầm nhận diện, vì hệ thống "im lặng" thay vì báo sai.

Đối tượng gần (lớn) vs xa (nhỏ)



Ảnh hưởng của stride lớn trên feature map



Hình 2-15: Vật thể nhỏ hơn một ô lưới trên feature map có stride lớn, dễ bị bỏ sót

Ba giải pháp xử lý chính:

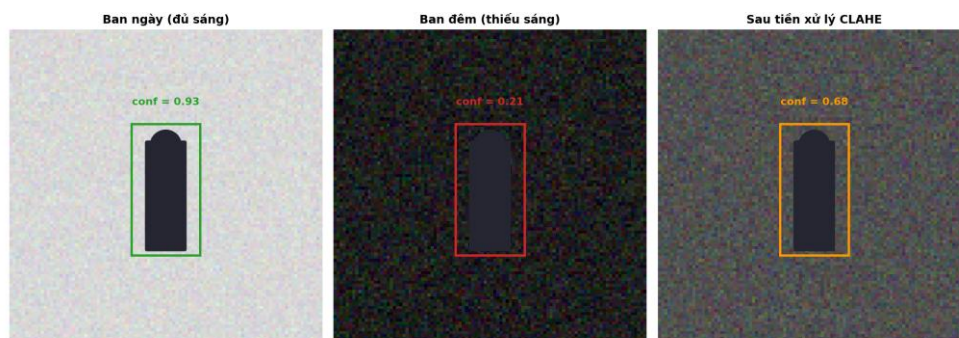
- Tăng độ phân giải ảnh đầu vào (ví dụ $\text{imgsz}=1280$ thay vì 640) để vật thể nhỏ chiếm nhiều pixel hơn, giữ được nhiều chi tiết hơn khi đi qua mạng;
- Dùng kỹ thuật SAHI (Slicing Aided Hyper Inference) - cắt ảnh gốc thành nhiều ô nhỏ chồng lấp, chạy suy luận riêng trên từng ô rồi ghép kết quả lại, giúp mỗi vật thể nhỏ chiếm tỉ lệ lớn hơn trong ảnh con;
- Tận dụng nhánh Detection Head ở feature map độ phân giải cao nhất (stride 8) - vốn được thiết kế chuyên biệt cho vật thể nhỏ trong kiến trúc YOLO.

2.13.3 Ánh sáng yếu / ban đêm

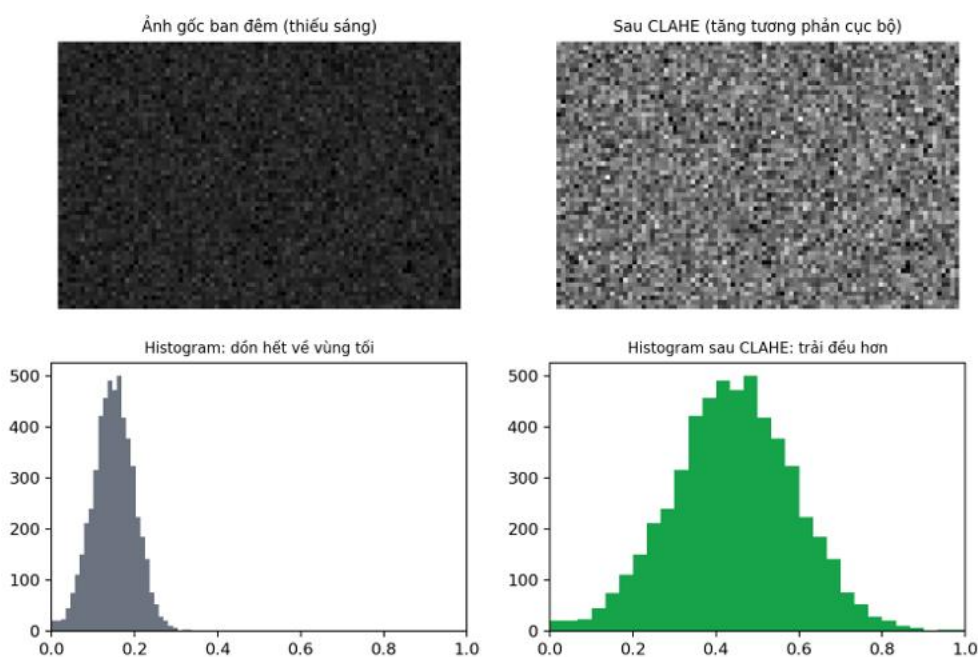
Khái niệm: Ánh sáng yếu là điều kiện phổ biến với camera giám sát hoạt động 24/7 - đặc biệt vào ban đêm hoặc trong khu vực thiếu chiếu sáng. Ảnh thu được trong điều kiện này có đặc trưng: phần lớn giá trị pixel dồn vào vùng tối (giá trị thấp), độ tương phản giữa đối tượng và nền rất kém, nhiễu cảm biến (sensor noise) tăng cao, và biên dạng vật thể trở nên mờ nhòe.

Các mô hình YOLO thường được huấn luyện chủ yếu trên dữ liệu ban ngày, ánh sáng đầy đủ (như bộ COCO). Khi phân bố độ sáng của ảnh đầu vào lúc suy luận khác biệt lớn so với phân bố lúc huấn luyện - hiện tượng gọi là domain shift - độ chính xác của mô hình giảm rõ rệt, dù bản thân đối tượng không hề thay đổi về hình dạng.

Ảnh hưởng: Camera an ninh hoạt động nhiều nhất và cần độ tin cậy cao nhất chính là vào ban đêm - thời điểm nguy cơ xâm nhập/trộm cắp cao nhất. Nếu hệ thống suy giảm hiệu năng đáng kể trong điều kiện thiếu sáng, đây lại chính là lúc hệ thống dễ bỏ sót hoặc phát hiện sai nhất - đúng vào lúc quan trọng nhất.



Hình 2-16: Cùng một đối tượng trong ba điều kiện



Hình 2-17: Biểu đồ histogram vùng tối sau khi áp dụng CLAHE, tăng độ tương phản giúp lộ rõ chi tiết đối tượng

Giải pháp: CLAHE (Contrast Limited Adaptive Histogram Equalization) được áp dụng như một bước tiền xử lý trước khi đưa ảnh vào YOLO: kỹ thuật này chia ảnh thành các vùng nhỏ (tile), cân bằng histogram cục bộ trong từng vùng (thay vì toàn ảnh) để tránh khuếch đại nhiễu quá mức, đồng thời giới hạn hệ số khuếch đại tương phản (clip limit) để không làm ảnh bị "cháy" chi tiết. Song song đó, trong giai đoạn huấn luyện, dữ liệu được tăng cường bằng cách chỉnh ngẫu nhiên độ sáng, độ tương phản và nhiều màu (Hue, Saturation, Value) để mô hình học được cách nhận diện đối tượng bất kể điều kiện ánh sáng.

2.13.4 Camera rung

Khái niệm: Camera rung (camera shake / camera motion) xảy ra khi bản thân camera bị dịch chuyển hoặc rung lắc - do gió tác động lên cột lắp đặt, rung động cơ học, hoặc với camera PTZ (Pan-Tilt-Zoom) đang chủ động xoay/zoom. Khi camera di chuyển, TOÀN BỘ khung hình dịch chuyển theo, kể cả các vật thể tĩnh như tường, cây cối, mặt đường - không chỉ riêng đối tượng đang được theo dõi.

Bộ lọc Kalman trong ByteTrack dự đoán vị trí tiếp theo của một track dựa trên giả định chuyển động diễn ra mượt mà, nhất quán trong hệ tọa độ ảnh (image coordinate). Khi camera rung, toàn bộ hệ tọa độ ảnh bị xô dịch đột ngột giữa hai khung hình liên tiếp - làm cho vị trí dự đoán của Kalman (dựa trên vận tốc ước lượng trước đó) lệch khỏi vị trí phát hiện thực tế, dù đối tượng trong thế giới thực gần như không di chuyển. Hệ quả là điểm IoU giữa box dự đoán và box phát hiện giảm mạnh, có thể khiến thuật toán liên kết dữ liệu gán nhầm hoặc bỏ lỡ track.

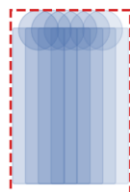
Ảnh hưởng: Rung camera gây ra hàng loạt lỗi liên kết sai trong thời gian ngắn, làm tăng đột biến số lượng ID switch và track bị đứt gãy - đặc biệt nghiêm trọng với camera ngoài trời, camera gắn trên cột cao, hoặc camera hoạt động trong điều kiện gió mạnh.

Khung hình ổn định vs bị rung (motion blur)

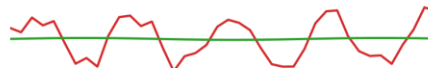
Rõ nét
IoU khung liên tiếp = 0.88



Mờ do rung
IoU khung liên tiếp = 0.31



Bù chuyển động camera (CMC)



— Vị trí box thô (chưa bù)
— Sau khi bù CMC (ổn định)

Hình 2-18: So sánh IoU giữa hai khung liên tiếp khi ảnh rõ nét so với khi bị nhòe do rung

Giải pháp: Cải tiến của ByteTrack tích hợp cơ chế CMC (Camera Motion Compensation - xem chi tiết mục 2.4.5 của báo cáo chính): hệ thống ước lượng chuyển động của TOÀN CẢNH bằng Sparse Optical Flow trên các điểm đặc trưng nền, tính ra một ma trận biến đổi (transform matrix) đại diện cho chuyển động chung của camera, rồi dùng ma trận này để "bù trừ" lại trước khi so khớp dự đoán Kalman với detection thực tế - giúp tách bạch chuyển động do camera rung khỏi chuyển động thật của đối tượng.

2.13.5 ID Switch (nhằm đổi tượng A thành B)

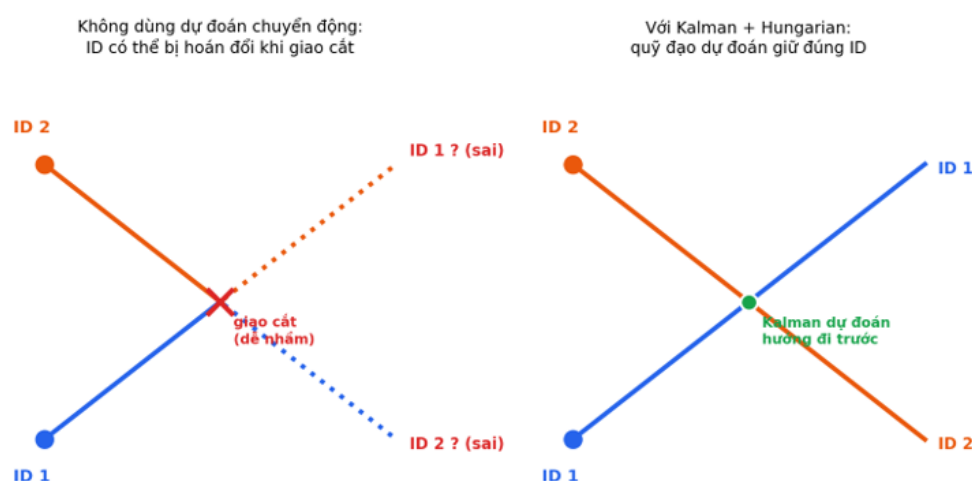
Khái niệm: ID Switch là hiện tượng hệ thống theo dõi gán nhầm danh tính (ID) giữa hai đối tượng khác nhau - ví dụ đối tượng A đang mang ID=1 bỗng bị đổi thành ID=2 (vốn thuộc đối tượng B) sau một số khung hình, dù trên thực tế cả hai vẫn là hai người/vật thể riêng biệt, không hề đổi chỗ cho nhau. Đây là một trong những chỉ số quan trọng nhất để đánh giá chất lượng tracker (được đo trực tiếp trong độ đo IDF1 và là một thành phần của MOTA - xem Bảng 2-4).

ID switch thường xảy ra trong ba tình huống:

- Hai đối tượng di chuyển giao cắt quỹ đạo hoặc đi sát nhau (đặc biệt khi vận tốc, hướng đi và kích thước tương đồng, khiến độ đo IoU/vị trí không đủ để phân biệt)

- Một đối tượng bị che khuất đủ lâu khiến track cũ hết hạn (bị xóa khỏi bộ nhớ), sau đó một đối tượng KHÁC xuất hiện gần vị trí đó và vô tình được gán nhầm vào track vừa hết hạn
- Thiếu đặc trưng ngoại hình (Re-ID) đủ mạnh để phân biệt hai đối tượng có hình dáng/trang phục tương tự khi độ đo hình học không còn đáng tin cậy.

Ảnh hưởng: ID switch làm sai lệch toàn bộ dữ liệu thống kê hành vi được xây dựng dựa trên từng ID (thời gian lưu trú, quãng đường di chuyển, số lần đi qua lại - xem mục 2.8 về vector đặc trưng đầu vào cho mô hình chấm điểm rủi ro). Một track bị đổi ID giữa chừng sẽ khiến hệ thống hiểu nhầm rằng có hai đối tượng lưu trú ngắn thay vì một đối tượng lưu trú dài - có thể làm bỏ sót các cảnh báo về hành vi "lảng vảng" (loitering) cần đủ thời gian tích lũy để kích hoạt.



Hình 2-19: Diễn biến ID Switch khi không và có kết hợp Kalman + Hungarian

Giải pháp: Ba cơ chế phối hợp để giảm thiểu ID switch:

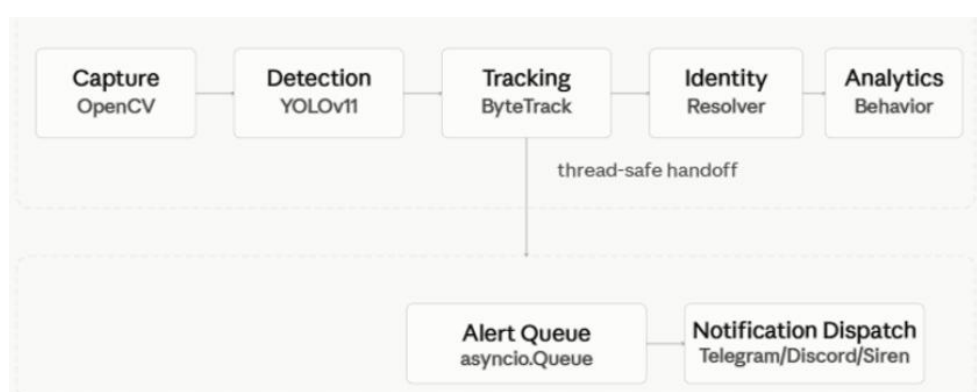
- Bộ lọc Kalman dự đoán quỹ đạo dựa trên vận tốc trước đó, giúp phân biệt hai track giao cắt như minh họa ở trên.
- Thuật toán Hungarian tìm phép gán tối ưu toàn cục thay vì tham lam theo từng cặp riêng lẻ (xem chi tiết mục 2.5 của báo cáo chính).

- Tăng thời gian Track Buffer (khoảng thời gian một track được giữ lại trong bộ nhớ dù chưa được gán detection) để track không bị xóa quá sớm trong lúc đối tượng bị che khuất tạm thời; kết hợp với mô hình Re-ID (như trong DeepSORT/BoT-SORT) để dùng đặc trưng ngoại hình làm căn cứ bổ sung khi độ đo hình học không đủ tin cậy.

CHƯƠNG 3: TRIỂN KHAI ĐỀ TÀI

3.1 Kiến trúc tổng thể hệ thống

Hệ thống được thiết kế theo nguyên tắc phân tầng theo luồng xử lý dữ liệu (data-flow layering), trong đó dữ liệu hình ảnh được biến đổi tuần tự qua sáu tầng chức năng độc lập, giúp mỗi thành phần có thể được kiểm thử, thay thế hoặc nâng cấp riêng lẻ mà không ảnh hưởng đến toàn hệ thống. Hình 20 minh họa kiến trúc tổng thể của hệ thống.



Hình 3-1: Kiến trúc pipeline sáu tầng của hệ thống

Như thể hiện trong Hình 20, mỗi camera được xử lý trên một luồng (thread) đồng bộ riêng biệt, thực hiện tuần tự năm tầng đầu: Capture (thu nhận khung hình thô từ OpenCV), Detection (phát hiện đối tượng bằng YOLOv11, xem lý thuyết ở mục 2.3), Tracking (gán danh tính tạm thời cho từng đối tượng qua các khung hình bằng ByteTrack, xem lý thuyết ở mục 2.4), Identity (phân giải danh tính người bằng InsightFace, xem lý thuyết ở mục 2.6) và Analytics (đánh giá các luật hành vi dựa trên nền tảng hình học ở mục 2.7). Kết quả từ tầng Analytics - bao gồm danh sách cảnh báo phát sinh - được chuyển giao an toàn giữa các luồng (thread-safe handoff) sang một vòng lặp sự kiện bất đồng bộ (asynchronous event loop) chạy song song, nơi cảnh báo được đưa vào hàng đợi (Alert Queue) trước khi được phân phối đến các kênh thông báo và luồng MJPEG hiển thị trên dashboard.

Việc tách riêng luồng xử lý AI đồng bộ khỏi luồng mạng bất đồng bộ mang lại hai lợi ích chính: một tác vụ suy luận AI chậm ở một camera không làm nghẽn việc gửi cảnh báo hoặc hiển thị của các camera khác, và việc gửi thông báo qua mạng (vốn có độ trễ I/O không thể dự đoán trước) không làm chậm lại tốc độ xử lý khung hình.

3.2 Môi trường và công cụ phát triển

Hệ thống được phát triển bằng ngôn ngữ Python, sử dụng các thư viện và framework mã nguồn mở sau, chia theo ba nhóm chức năng:

- **Nhóm AI/Deep Learning:** ultralytics (mô hình YOLOv11 và YOLOv11-Pose, tích hợp sẵn ByteTrack), torch/torchvision (nền tảng suy luận trên GPU/CPU), insightface (trích xuất vector đặc trưng khuôn mặt), onnxruntime (chạy suy luận InsightFace trên CPU để không tranh chấp GPU với YOLO).
- **Nhóm tính toán số và hình học:** opencv-python (đọc luồng video, xoay/mã hóa khung hình, kiểm tra điểm trong đa giác), numpy (biểu diễn ma trận và huấn luyện mô hình hồi quy logistic bằng gradient descent thuần, không dùng framework học máy nặng).
- **Nhóm dịch vụ web và giao tiếp mạng:** fastapi/uvicorn (REST API và luồng MJPEG bất đồng bộ), jinja2 (dựng giao diện HTML động), httpx (gửi thông báo Telegram/Discord có cơ chế thử lại), PyYAML (đọc/ghi cấu hình hệ thống và từng camera).

3.3 Triển khai tầng phát hiện và theo dõi đối tượng

Tầng phát hiện sử dụng mô hình YOLO11s (bản Small, cân bằng độ chính xác và tốc độ) với ngưỡng độ tin cậy có thể cấu hình riêng cho từng lớp đối tượng thay vì dùng một ngưỡng chung, ví dụ ngưỡng mặc định cho lớp person là 0.20 - thấp hơn hẳn ngưỡng mặc định 0.4 dùng cho các lớp khác - nhằm ưu tiên không bỏ sót người xuất hiện trong khung hình. Hệ thống áp dụng thêm một bước lọc hình học: các khung

bao của lớp person có tỉ lệ khung hình vượt ngưỡng cho phép hoặc diện tích quá nhỏ sẽ bị loại bỏ, giúp giảm các trường hợp nhận nhầm rèm cửa, cột nhà thành người.

Tầng theo dõi hiện thực chiến lược BYTE đã trình bày ở mục 2.4 thông qua cài đặt ByteTrack có sẵn của thư viện Ultralytics [26], được mở rộng thêm ba cơ chế phù hợp với bài toán giám sát dài hạn:

- **Bộ nhớ track bị mất tạm thời:** khi một track biến mất khỏi kết quả theo dõi trong một khoảng thời gian ngắn (do che khuất vượt quá số khung ân hạn mà bước liên kết thấp của BYTE xử lý), hệ thống lưu lại tâm điểm, tỉ lệ khung bao và lớp đối tượng gần nhất; nếu một phát hiện mới xuất hiện đủ gần về vị trí và kích thước, track cũ được gán lại ID thay vì tạo track mới.
- **Bù chuyển động camera:** sử dụng luồng quang học thưa để ước lượng và bù trừ rung lắc/xoay nhẹ của camera trước khi dự đoán vị trí track bằng bộ lọc Kalman, có cơ chế dự phòng an toàn trả về phép biến đổi đồng nhất nếu bước ước lượng chuyển động thất bại.
- **Loại bỏ khung bao trùng lặp:** sau bước gán track, các khung bao cùng lớp đối tượng có IoU vượt một ngưỡng cấu hình được xem là trùng lặp và bị loại bớt, hiện tượng thường gặp khi mô hình phát hiện không dùng cơ chế neo phát sinh hai khung chồng lấn cho cùng một đối tượng khi ảnh bị mờ chuyển động.

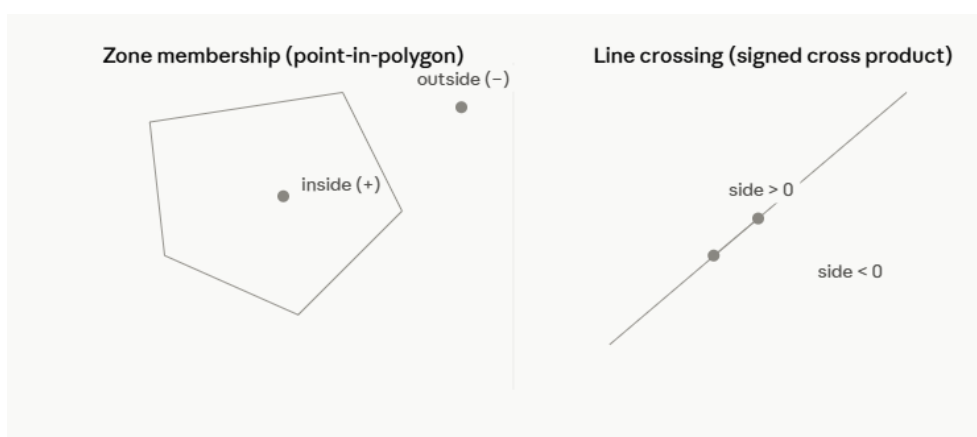
3.4 Triển khai tầng nhận diện danh tính

Mỗi track thuộc lớp person được trích xuất vector đặc trưng khuôn mặt bằng InsightFace và so sánh với các vector tham chiếu đã thu thập sẵn theo công thức cosine similarity ở mục 2.6. Một track được gán nhãn `known_person` khi score lớn nhất đạt ngưỡng cấu hình 0,45, được gán `stranger` sau khi đã thử đủ số lần xác nhận mà không khớp và ở trạng thái `pending_person` trong thời gian chờ. Mục 4.7 đánh giá riêng 0,35; 0,40; 0,45 trên một pilot nhỏ.

Để giảm tình trạng nhấp nháy nhãn danh tính khi ByteTrack vô tình cấp lại một ID mới cho cùng một người, hệ thống duy trì một bộ nhớ ngắn hạn cho phép một track mới xuất hiện gần vị trí và có kích thước tương đồng với một danh tính vừa được xác nhận trước đó kế thừa lại nhãn đó mà không cần chạy lại toàn bộ quá trình nhận diện khuôn mặt.

3.5 Triển khai tầng phân tích hành vi

Dựa trên hai nguyên lý hình học đã trình bày ở mục 2.7, bảy luật hành vi độc lập được hiện thực và có thể bật/tắt/cấu hình riêng cho từng camera. Hình 21 minh họa trực quan hai nguyên lý hình học nền tảng được sử dụng xuyên suốt tầng này.



Hình 3-2: Hai nguyên lý hình học nền tảng

- **Intrusion (Xâm nhập):** phát cảnh báo một lần duy nhất khi có người bước vào vùng cấm, có cơ chế đếm một số khung hình trống trước khi coi vùng đã trống hẳn, tránh cảnh báo lặp lại do phát hiện chập chờn.
- **Loitering (Lảng vảng):** đo thời gian một người lưu lại liên tục trong một vùng giám sát bằng bộ đếm thời gian riêng cho từng track, so sánh với ngưỡng thời gian cấu hình theo từng vùng.
- **Unknown Person (Người lạ):** phát cảnh báo khi một track được xác nhận là người lạ.

- **Suspicious Stranger (Người lạ khả nghi):** bổ sung điều kiện về kiểu di chuyển - đứng yên gần như bất động hoặc đi lại qua lại nhiều lần trong một khu vực nhỏ - trước khi cảnh báo về người lạ lưu lại quá lâu.
- **Asset Watch (Theo dõi tài sản):** theo dõi xe đạp, xe máy, ba lô, va li... trong một vùng được bảo vệ, cảnh báo khi tài sản rời khỏi vùng hoặc biến mất trong khi trước đó có người lạ ở gần.
- **Suspicious Theft Behavior (Hành vi nghi trộm cắp):** tích lũy điểm số dựa trên nhiều tín hiệu độc lập - thời gian đứng gần phương tiện, số lần đi qua lại, phương tiện bắt đầu dịch chuyển, hướng di chuyển tương quan giữa người và phương tiện, tín hiệu tiếp xúc từ tư thế cổ tay - chỉ cảnh báo khi đạt đủ số lượng tín hiệu độc lập cấu hình trước.
- **Line Counter (Đếm cắt vạch):** đếm số lượt đối tượng đi vào/đi ra theo từng hướng qua một vạch được cấu hình.

Ngoài bảy luật ROI/line ở trên, pipeline còn tích hợp FallDetector theo hướng safety-critical. Hệ thống không kết luận ngã từ một khung hình đơn lẻ mà yêu cầu quan sát chuyển tiếp từ trạng thái thẳng (đứng/đi/chạy) sang lying hoặc changing_to_lying. Sau 10 giây vẫn nằm, hệ thống phát possible_fall; khi tổng thời gian đạt 45 giây, cảnh báo được nâng thành possible_unresponsive. Các cảnh báo này không bị Risk gate chặn và chỉ mang ý nghĩa cảnh báo sớm, không phải chẩn đoán bất tỉnh, đột quỵ hoặc tình trạng y tế.

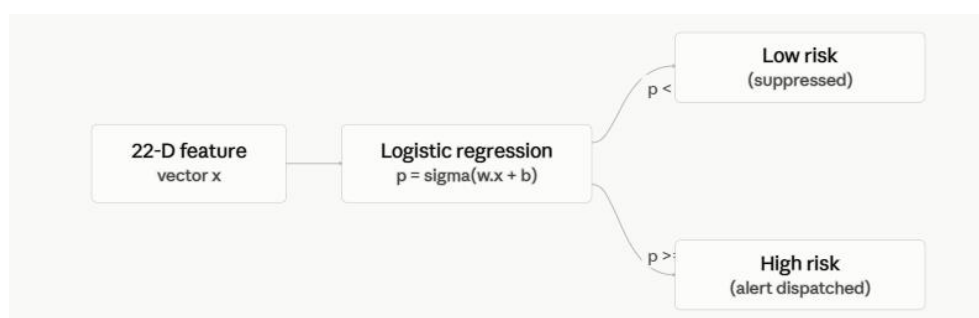
Sau khi đã quan sát thấy người chuyển sang nằm, nếu bbox hoặc pose tạm mất do che khuất, hệ thống giữ fall candidate tối đa 60 giây, lưu last_known_bbox và vẽ khung nét đứt LAST SEEN tại vị trí cuối cùng. Nội dung cảnh báo ghi rõ giới hạn tầm nhìn thay vì giả định người vẫn đang nằm tại vị trí đó. Nếu chưa từng quan sát thấy tư thế nằm, việc một track đơn thuần biến mất không tạo cảnh báo ngã.

Hồi phục được xác nhận khi người xuất hiện lại và duy trì tư thế ngồi, đứng, đi hoặc chạy ổn định trong 3 giây. FallDetector phát sự kiện fall_recovery nội bộ để

pipeline xóa lớp phủ POSSIBLE FALL/POSSIBLE EMERGENCY; lịch sử pose và fall candidate cũng được xóa khi camera khởi động lại để tránh kế thừa trạng thái cũ.

3.6 Triển khai tầng học hành vi và chấm điểm rủi ro

Mỗi cảnh báo ứng viên do các luật ở mục 3.5 sinh ra được chuyển thành một vector đặc trưng số gồm 22 chiều (tỉ lệ thời gian lưu trú, tỉ lệ quãng đường di chuyển và độ dịch chuyển tổng theo đường chéo khung hình, độ tin cậy phát hiện, tỉ lệ diện tích khung bao, số lần đi qua lại, cùng các chỉ báo one-hot về lớp đối tượng và loại vùng cấu hình) và được ghi vào một tệp nhật ký JSONL kèm một mã sự kiện cố định. Người vận hành có thể gán nhãn (cảnh báo đúng/cảnh báo giả) cho các sự kiện này thông qua tệp CSV hoặc một API REST nhỏ. Mô hình hồi quy logistic đã trình bày lý thuyết ở mục 2.8 được huấn luyện trên tập sự kiện đã gán nhãn này. Hình 22 minh họa luồng xử lý của mô hình chấm điểm rủi ro.

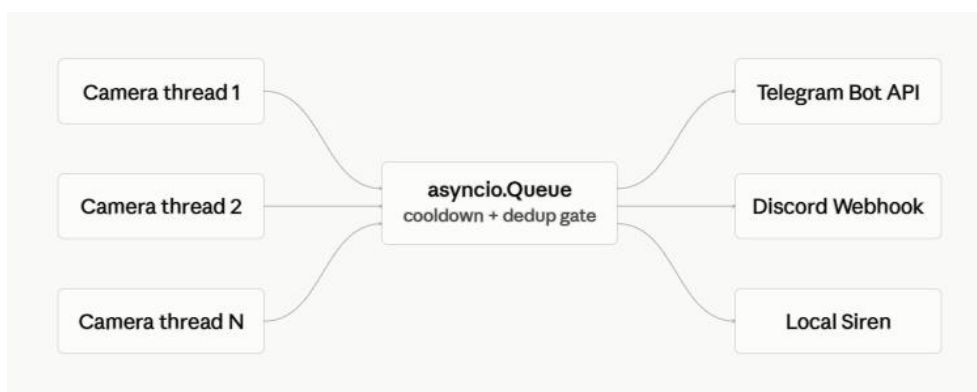


Hình 3-3: Sơ đồ mô hình chấm điểm rủi ro hành vi bằng hồi quy logistic

Mô hình sau huấn luyện (vector trọng số, độ chệch, và các tham số chuẩn hóa trung bình/độ lệch chuẩn của từng đặc trưng) được nạp vào bộ nhớ khi hệ thống khởi động và tự động nạp lại (hot-reload) mỗi khi phát hiện tệp mô hình trên đĩa có thời gian sửa đổi mới hơn, cho phép huấn luyện lại và cập nhật mô hình mà không cần khởi động lại toàn bộ hệ thống. Theo mặc định, mô hình chỉ gán thêm điểm rủi ro vào cảnh báo để tham khảo; người vận hành có thể bật thêm tùy chọn để tự động giữ lại các cảnh báo có điểm rủi ro thấp hơn một ngưỡng cấu hình được, qua đó đánh đổi một phần độ nhạy để giảm tỉ lệ cảnh báo giả khi đã có đủ dữ liệu gán nhãn.

3.7 Triển khai hệ thống điều phối cảnh báo và giao diện web

Tầng cuối cùng của hệ thống chịu trách nhiệm chuyển các cảnh báo đã được phân tích thành hành động cụ thể: gửi thông báo đến người vận hành và hiển thị luồng video trực quan. Hình 23 minh họa kiến trúc điều phối cảnh báo theo mô hình Producer-Consumer.



Hình 3-4: Kiến trúc điều phối cảnh báo theo mô hình Producer-Consumer

Mỗi luồng xử lý camera đóng vai trò là một producer, đẩy cảnh báo vào một hàng đợi bất đồng bộ dùng chung (dung lượng giới hạn 1000 phần tử, cảnh báo mức cao khi đạt 800 phần tử) thông qua cơ chế chuyển giao an toàn giữa các luồng. Một worker bất đồng bộ duy nhất đóng vai trò consumer, đọc lần lượt từng cảnh báo và áp dụng cơ chế thời gian chờ tối thiểu (cooldown) theo tổ hợp khóa (camera, loại cảnh báo, vùng/vạch/track) trước khi thực sự gửi đi, nhằm tránh việc gửi trùng lặp liên tục cùng một cảnh báo trong khoảng thời gian ngắn.

Các kênh gửi thông báo được hỗ trợ bao gồm Telegram Bot API, Discord Webhook (cả hai đều gửi kèm ảnh chụp khung hình tại thời điểm cảnh báo, có cơ chế thử lại khi gặp lỗi mạng) và còi báo động cục bộ chạy trên luồng riêng để không làm nghẽn vòng lặp sự kiện chính. Song song đó, một ứng dụng FastAPI cung cấp luồng video MJPEG cho từng camera (giới hạn tối đa mười người xem đồng thời mỗi luồng), REST API để quản lý camera/vùng/vạch, và một dashboard cho phép người

dùng vẽ trực tiếp vùng giám sát dạng đa giác hoặc vạch đếm lên trên hình ảnh trực tiếp của camera thông qua canvas HTML5.

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM

4.1 Kịch bản đo hiệu năng

Để đánh giá hiệu năng và khả năng mở rộng, hệ thống được kiểm thử trên hai cấu hình phần cứng ở Bảng 4-1 với cùng bốn nguồn video cục bộ và cùng cấu hình pipeline. Phép đo scaling lần lượt bật 1, 2, 3 và 4 camera; mỗi mức chạy 600 giây và ghi một mẫu sau mỗi 10 giây. Phép đo độ ổn định bật đồng thời 4 camera trong 28.800 giây (8 giờ), cũng với chu kỳ ghi 10 giây. Các camera và mô hình được khởi động trước khi bắt đầu ghi để loại ảnh hưởng nạp mô hình lần đầu.

Bảng 4-1: Thông số hai môi trường thực nghiệm

Thành phần	Laptop	GPU VM
Hệ điều hành	Windows 11, 64-bit	Windows 10 Pro, build 19045
CPU	AMD Ryzen 7 6800H	Intel Core i7-12700K (theo cấu hình thuê)
RAM	16 GB	28 GB
GPU	RTX 3050 Laptop, 4 GB VRAM	RTX 5060 Ti, 16 GB VRAM
Nền tảng	Python 3.10.11; PyTorch CUDA	Python 3.10.11; PyTorch 2.12.1+cu130
Mô hình	YOLO11s; YOLO11n-pose; InsightFace buffalo_sc	Cùng cấu hình mô hình
Dữ liệu đo	Cùng 4 video cục bộ	Cùng 4 video cục bộ

Script `monitor_runtime.py` tự động ghi thời gian, trạng thái camera, AI FPS tổng hợp, CPU, RAM, VRAM, mức sử dụng GPU, nhiệt độ GPU và số khung bị bỏ. Vì nguồn video cục bộ có thể phát lại nhanh hơn thời gian thực, các kết luận hiệu năng chỉ dùng `aggregate_ai_fps`; chỉ số `aggregate_fps` của vòng đọc video không được dùng.

4.2 Kết quả thông lượng và độ trễ

Bảng 4-2: AI FPS tổng hợp theo số lượng camera trong phép đo 10 phút

Cấu hình	1 camera	2 camera	3 camera	4 camera	Hiệu suất 4 camera
RTX 3050 Laptop	7,56	16,35	21,34	16,40	54,2%
RTX 5060 Ti	7,60	15,31	21,97	29,83	98,2%

Mỗi ô FPS trong Bảng 4-2 là trung bình của 60–61 mẫu. Hiệu suất scaling tại 4 camera được tính bằng thông lượng thực tế chia cho bốn lần thông lượng một camera.

Chương 4: Kết quả thực nghiệm

Trên RTX 3050 Laptop, thông lượng tăng đến 21,34 AI FPS ở ba camera nhưng giảm còn 16,40 AI FPS ở bốn camera; CPU trung bình khi đó đạt 406,7%, RAM 2.585,1 MB, VRAM 334 MB và nhiệt độ cực đại 59 °C. Trên RTX 5060 Ti, thông lượng tăng gần tuyến tính từ 7,60 lên 29,83 AI FPS, tương ứng hiệu suất scaling 98,2%; ở bốn camera, CPU trung bình là 289,6%, GPU 13,8%, RAM 2.761,4 MB, VRAM 1.488,7 MB và nhiệt độ cực đại 44 °C. Tại bốn camera, RTX 5060 Ti cao hơn RTX 3050 Laptop 81,9% về AI FPS. Kết quả cho thấy điểm nghẽn của laptop ở mức bốn camera không chỉ phụ thuộc năng lực GPU mà còn liên quan đến CPU và cơ chế khóa suy luận dùng chung. Trên Windows, CPU% là tổng mức sử dụng trên các lõi luận lý nên có thể lớn hơn 100%.

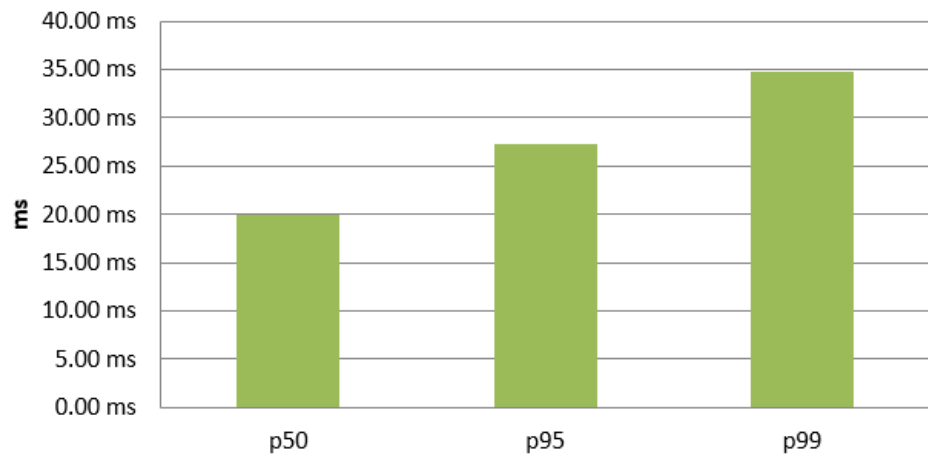
4.3 Độ trễ theo từng giai đoạn xử lý

Bảng 4-3: Độ trễ trung bình từng giai đoạn xử lý AI (trường hợp một camera)

Giai đoạn	p50	p95	p99
Phát hiện và Theo dõi	19,97 ms	27,21 ms	34,81 ms
Ước lượng tư thế	18,79 ms	25,91 ms	32,17 ms
Nhận diện danh tính	0,12 ms	0,17 ms	0,22 ms
Phân tích hành vi	1,15 ms	1,70 ms	2,45 ms

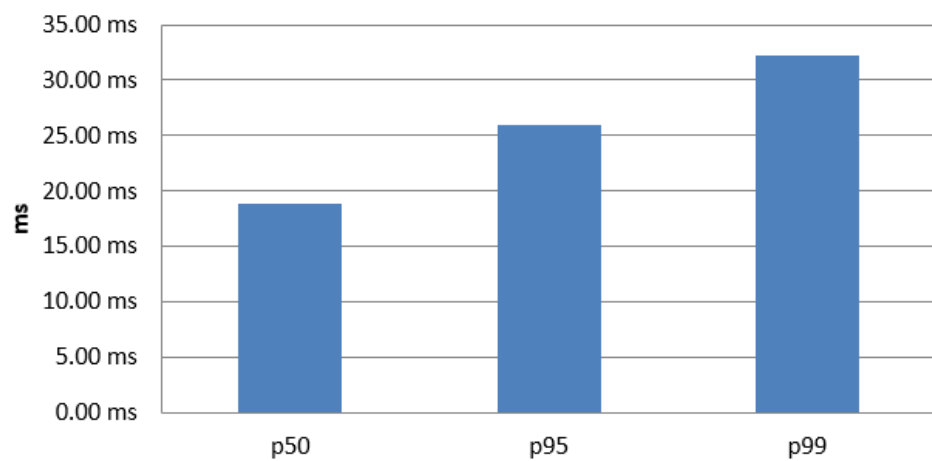
Chương 4: Kết quả thực nghiệm

Giai đoạn : Phát hiện và Theo dõi

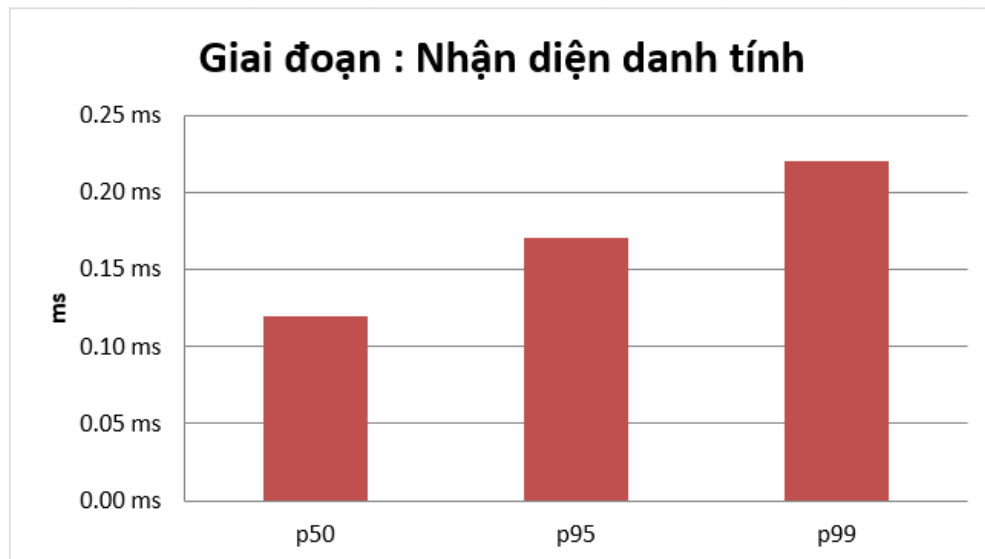


Hình 4-1: Phát hiện và theo dõi

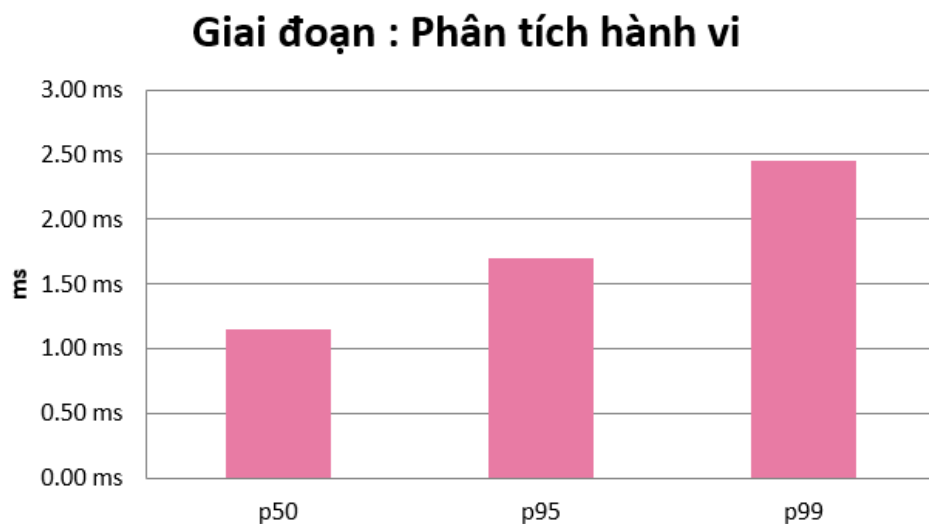
Giai đoạn : Ước lượng tư thế



Hình 4-2: Ước lượng tư thế



Hình 4-3: Nhận diện danh tính



Hình 4-4: Phân tích hành vi

Bảng 4-3 và các hình cho thấy hai giai đoạn tốn nhiều thời gian nhất là Phát hiện + Theo dõi và Ước lượng tư thế, mỗi giai đoạn chiếm xấp xỉ 19–20 ms ở mức trung vị, trong khi tăng Nhận diện danh tính và Phân tích hành vi gần như không đáng kể (dưới 1,2 ms cộng lại). Phép đo tách giai đoạn này bổ sung cho phép đo scaling đầu cuối ở Bảng 4-2 và cho thấy phần lớn chi phí nằm ở các mô hình thị giác.

Chương 4: Kết quả thực nghiệm

Cần lưu ý rằng độ trễ của tăng nhận diện danh tính trong bảng trên không phản ánh đầy đủ chi phí thực của việc so khớp khuôn mặt trực tiếp trên camera sống, do kịch bản benchmark sử dụng video cục bộ với chính sách gán nhãn người lạ mặc định (không chạy đường xử lý so khớp khuôn mặt đầy đủ). Việc đo hiệu năng riêng cho module nhận diện khuôn mặt trên dữ liệu có sự đồng thuận của người tham gia được xác định là một hướng đánh giá bổ sung trong tương lai.

4.4 Kiểm thử tự động hệ thống

Song song với việc đo hiệu năng ở các mục 4.1–4.3, tại phiên bản final ngày 20/08/2026, một bộ kiểm thử tự động (automated unit tests) được xây dựng bằng framework pytest nhằm xác nhận tính đúng đắn của từng thành phần trong hệ thống ở mức đơn vị (unit test), độc lập với việc phải chạy camera thật. Bảng 4-4 tổng hợp số lượng testcase theo từng nhóm chức năng.

Bảng 4-4: Tổng hợp các nhóm testcase tự động theo thành phần hệ thống

STT	Tập kiểm thử	Số testcase	Thành phần được kiểm thử
1	test_asset_watch.py	6	Theo dõi tài sản, cảnh báo mất và chống phát cảnh báo trùng lặp
2	test_auth.py	5	Xác thực API, dashboard và quản lý phiên đăng nhập
3	test_database.py	3	Khởi tạo, di trú, rollback và lưu dữ liệu cảnh báo
4	test_detector.py	6	Ngưỡng tin cậy và bộ lọc hình dạng khung bao
5	test_drawing.py	15	Nhãn, vùng ROI và lớp phủ cảnh báo trên khung hình
6	test_fall_detection.py	24	Phát hiện té ngã, xác nhận, nhắc lại và phục hồi
7	test_frame_buffer.py	2	Snapshot và số liệu độ trễ của bộ đệm khung hình

Chương 4: Kết quả thực nghiệm

STT	Tập kiểm thử	Số testcase	Thành phần được kiểm thử
8	test_intrusion.py	1	Điều kiện kích hoạt cảnh báo xâm nhập
9	test_line_counter.py	5	Phát hiện cắt vạch, đếm lượt và hướng di chuyển
10	test_monitor_runtime.py	2	Phân tích log GPU/runtime và số liệu camera khi benchmark
11	test_person_alerts.py	29	Cảnh báo người lạ, lãng vãng, vùng ROI và cooldown
12	test_person_identity.py	17	So khớp khuôn mặt, cache và kế thừa danh tính
13	test_pipeline.py	20	Nhịp AI, worker bất đồng bộ, xoay khung và dữ liệu cũ
14	test_pose.py	2	Ghép kết quả ước lượng tư thế với khung bao người
15	test_pose_classifier.py	13	Phân loại tư thế và trạng thái vận động
16	test_priority_behavior_experiment.py	6	Đánh giá pilot hành vi, ghép sự kiện và frozen test
17	test_revision_experiments.py	6	Tách dữ liệu, metric và biểu đồ cho thực nghiệm revision
18	test_runtime_settings.py	11	Cấu hình runtime, camera, mô hình và quá trình khởi động
19	test_theft_behavior.py	3	Phát hiện hành vi nghi trộm cắp và chống cảnh báo trùng
20	test_tracker_cmc.py	23	ByteTrack, CMC, track ID, thời gian ân hạn và chuyển lớp
21	test_web_responsiveness.py	7	Dashboard, MJPEG streaming và khả năng đáp ứng web

Chương 4: Kết quả thực nghiệm

STT	Tập kiểm thử	Số testcase	Thành phần được kiểm thử
	Tổng cộng	206	Toàn bộ testcase đạt trạng thái PASSED

Toàn bộ 206 testcase của phiên bản final ngày 20/08/2026 được chạy tự động bằng pytest và đều đạt trạng thái PASSED trong 17,08 giây; pytest ghi nhận thêm 3 cảnh báo không làm thất bại kiểm thử. Kết quả thực thi tổng thể được trình bày dưới đây.

```
PS E:\SCT_Camera> .\venv\Scripts\python.exe -m pytest -q tests --tb=no --disable-warnings
206 passed, 3 warnings in 17.08s
```

Dưới đây là 11 testcase tiêu biểu đại diện cho các thành phần quan trọng nhất của hệ thống

Chương 4: Kết quả thực nghiệm

Testcase 1: test_bootstrap_and_migrations_are_idempotent

Mã testcase	TC-DB-01
Mục đích kiểm thử	Kiểm tra cơ chế di trú cơ sở dữ liệu (database migration) khởi tạo đúng schema và có tính idempotent — chạy lại nhiều lần không gây lỗi hoặc trùng lặp.
Điều kiện tiên đề	Tập cơ sở dữ liệu SQLite tạm thời chưa tồn tại; các tệp di trú 001, 002, 003 hợp lệ trong thư mục migrations.
Các bước thực hiện	1. Gọi DatabaseManager.bootstrap() để khởi tạo bảng metadata. 2. Gọi run_migrations() lần đầu. 3. Gọi run_migrations() lần thứ hai. 4. Kiểm tra bảng schema_migrations và danh sách bảng dữ liệu đã tạo.
Kết quả mong đợi	Lần chạy đầu áp dụng đúng 3 phiên bản di trú [1, 2, 3]; lần chạy thứ hai trả về danh sách rỗng (không áp dụng lại); đầy đủ các bảng alerts, notification_deliveries, behavior_events, behavior_labels, video_clips, alert_clips được tạo.

- **Kết quả thực tế:** PASSED; 1 passed, 2 deselected; thời gian 0,07 giây.

```
PS E:\SCT_Camera> .\venv\Scripts\python.exe -m pytest -v
tests\test_database.py -k "test_bootstrap_and_migrations_are_idempotent"
```

```
collected 3 items / 2 deselected / 1 selected
```

```
test_bootstrap_and_migrations_are_idempotent PASSED [100%]
```

```
1 passed, 2 deselected in 0.07s
```

Chương 4: Kết quả thực nghiệm

Testcase 2: test_failed_migration_rolls_back_schema_and_version

Mã testcase	TC-DB-02
Mục đích kiểm thử	Kiểm tra một tệp di trú bị lỗi cú pháp SQL sẽ được cuộn ngược (rollback) hoàn toàn, không để lại bảng hay bản ghi phiên bản dở dang.
Điều kiện tiên đề	Một tệp di trú 001_broken.sql chứa lệnh INSERT vào một bảng không tồn tại, được đặt cố ý để gây lỗi.
Các bước thực hiện	1. Gọi run_migrations() với tệp di trú lỗi. 2. Bắt ngoại lệ sqlite3.OperationalError phát sinh. 3. Kiểm tra lại bảng should_rollback và bảng schema_migrations trong cơ sở dữ liệu.
Kết quả mong đợi	Ngoại lệ OperationalError được ném ra đúng như kỳ vọng; bảng should_rollback không tồn tại trong cơ sở dữ liệu; bảng schema_migrations không có bản ghi nào — xác nhận toàn bộ giao dịch đã được cuộn ngược.

- **Kết quả thực tế:** PASSED; 1 passed, 2 deselected; thời gian 0,06 giây.

```
PS E:\SCT_Camera> .\venv\Scripts\python.exe -m pytest -v
tests\test_database.py -k "test_failed_migration_rolls_back_schema_and_version"
```

```
collected 3 items / 2 deselected / 1 selected
```

```
test_failed_migration_rolls_back_schema_and_version PASSED [100%]
```

```
1 passed, 2 deselected in 0.06s
```

Chương 4: Kết quả thực nghiệm

Testcase 3: test_rejects_extremely_thin_person_boxes

Mã testcase	TC-DET-01
Mục đích kiểm thử	Kiểm tra bộ lọc hình dạng khung bao của tăng phát hiện loại bỏ đúng các khung bao lớp person quá mảnh (tỉ lệ khung hình vượt ngưỡng), tránh nhận nhầm cột nhà/rèm cửa thành người.
Điều kiện tiên đề	Một khung bao person có tọa độ (460, 320, 494, 720) — chiều rộng 34px, chiều cao 400px; ngưỡng tỉ lệ khung hình tối đa cấu hình là 4.0.
Các bước thực hiện	1. Gọi hàm <code>_valid_detection_shape()</code> với khung bao và ngưỡng nêu trên. 2. Kiểm tra giá trị trả về.
Kết quả mong đợi	Hàm trả về False (khung bao bị loại bỏ) vì tỉ lệ chiều cao/chiều rộng (≈ 11.8) vượt xa ngưỡng cho phép (4.0).

- **Kết quả thực tế:** PASSED; 1 passed, 5 deselected; thời gian 0,16 giây.

```
PS E:\SCT_Camera> .\venv\Scripts\python.exe -m pytest -v
tests\test_detector.py -k "test_rejects_extremely_thin_person_boxes"
```

```
collected 6 items / 5 deselected / 1 selected
```

```
test_rejects_extremely_thin_person_boxes PASSED [100%]
```

```
1 passed, 5 deselected in 0.16s
```

Chương 4: Kết quả thực nghiệm

Testcase 4: test_allows_wide_sitting_or_lying_person_boxes

Mã testcase	TC-DET-02
Mục đích kiểm thử	Kiểm tra bộ lọc hình dạng không loại bỏ nhầm các khung bao person hợp lệ khi người ở tư thế ngồi hoặc nằm (khung bao rộng hơn cao).
Điều kiện tiên đề	Một khung bao person có tọa độ (610, 360, 1030, 540) — chiều rộng 420px, chiều cao 180px; ngưỡng tỉ lệ khung hình tối đa cấu hình là 4.0.
Các bước thực hiện	1. Gọi hàm <code>_valid_detection_shape()</code> với khung bao và ngưỡng nêu trên. 2. Kiểm tra giá trị trả về.
Kết quả mong đợi	Hàm trả về True (khung bao được giữ lại) vì tỉ lệ khung hình (≈ 2.3) vẫn nằm trong ngưỡng cho phép, không bị loại bỏ nhầm.

- **Kết quả thực tế:** PASSED; 1 passed, 5 deselected; thời gian 0,14 giây.

```
PS E:\SCT_Camera> .\venv\Scripts\python.exe -m pytest -v
tests\test_detector.py -k "test_allows_wide_sitting_or_lying_person_boxes"
```

```
collected 6 items / 5 deselected / 1 selected
```

```
test_allows_wide_sitting_or_lying_person_boxes PASSED [100%]
```

```
1 passed, 5 deselected in 0.14s
```

Chương 4: Kết quả thực nghiệm

Testcase 5: test_cmc_failure_falls_back_to_identity_transform

Mã testcase	TC-TRK-01
Mục đích kiểm thử	Kiểm tra cơ chế dự phòng an toàn của module bù chuyển động camera (CMC): khi bước ước lượng chuyển động bằng luồng quang học thất bại, hệ thống phải trả về phép biến đổi đồng nhất thay vì làm sập chương trình.
Điều kiện tiên đề	Đối tượng _SafeGMC được khởi tạo với phương thức sparseOptFlow; hàm apply() nội bộ được giả lập (mock) để chủ động ném ngoại lệ RuntimeError.
Các bước thực hiện	1. Gọi gmc.apply() với một khung hình bất kỳ. 2. So sánh ma trận trả về với ma trận đồng nhất 2x3. 3. Kiểm tra hàm reset_params() có được gọi hay không.
Kết quả mong đợi	Kết quả trả về đúng bằng ma trận đồng nhất np.eye(2, 3); hàm reset_params() được gọi đúng một lần để đặt lại trạng thái bộ ước lượng, hệ thống không phát sinh ngoại lệ ra bên ngoài.

- **Kết quả thực tế:** PASSED; 1 passed, 22 deselected; thời gian 3,18 giây.

```
PS E:\SCT_Camera> .\venv\Scripts\python.exe -m pytest -v
tests\test_tracker_cmc.py -k "test_cmc_failure_falls_back_to_identity_transform"
```

```
collected 23 items / 22 deselected / 1 selected
```

```
test_cmc_failure_falls_back_to_identity_transform PASSED [100%]
```

```
1 passed, 22 deselected in 3.18s
```

Chương 4: Kết quả thực nghiệm

Testcase 6: test_stale_track_expires_after_grace_frames

Mã testcase	TC-TRK-02
Mục đích kiểm thử	Kiểm tra một track không còn được ByteTrack trả về sẽ tồn tại thêm đúng số khung hình ân hạn (track_grace_frames) cấu hình được trước khi bị xóa hẳn khỏi danh sách theo dõi.
Điều kiện tiên đề	Bộ theo dõi ByteTrackTracker được cấu hình track_grace_frames = 1; một track ID 7 đã xuất hiện ở khung hình đầu tiên.
Các bước thực hiện	1. Chạy track() ở khung hình đầu tiên với một phát hiện hợp lệ. 2. Chạy track() ở khung hình kế tiếp với danh sách phát hiện rỗng. 3. Chạy track() ở khung hình thứ ba, vẫn với danh sách phát hiện rỗng.
Kết quả mong đợi	Ở khung hình thứ hai, track ID 7 vẫn còn được trả về (đang trong thời gian ân hạn); ở khung hình thứ ba, danh sách trả về rỗng — track đã bị xóa đúng sau khi hết số khung hình ân hạn cấu hình.

- **Kết quả thực tế:** PASSED; 1 passed, 22 deselected; thời gian 3,07 giây.

```
PS E:\SCT_Camera> .\venv\Scripts\python.exe -m pytest -v
tests\test_tracker_cmc.py -k "test_stale_track_expires_after_grace_frames"
```

```
collected 23 items / 22 deselected / 1 selected
```

```
test_stale_track_expires_after_grace_frames PASSED [100%]
```

```
1 passed, 22 deselected in 3.07s
```

Chương 4: Kết quả thực nghiệm

Testcase 7: test_rejects_embedding_below_similarity_threshold

Mã testcase	TC-ID-01
Mục đích kiểm thử	Kiểm tra tăng nhận diện danh tính gán nhãn Stranger cho một khuôn mặt có độ tương đồng cosine với ảnh tham chiếu thấp hơn ngưỡng cấu hình.
Điều kiện tiên đề	Một PersonIdentityResolver có một ảnh tham chiếu "Alice" với vector đặc trưng [1.0, 0.0]; khuôn mặt quan sát được có vector đặc trưng [0.0, 1.0] (trực giao, độ tương đồng cosine = 0).
Các bước thực hiện	1. Gọi label_objects() với track chứa khuôn mặt nêu trên. 2. Kiểm tra nhãn identity_label và identity_kind trả về.
Kết quả mong đợi	Track được gán nhãn identity_label = "Stranger" và identity_kind = "stranger"; điểm identity_score trả về là None do không có ảnh tham chiếu nào vượt ngưỡng tương đồng.

- **Kết quả thực tế:** PASSED; 1 passed, 16 deselected; thời gian 0,19 giây.

```
PS E:\SCT_Camera> .\venv\Scripts\python.exe -m pytest -v
tests\test_person_identity.py -k "test_rejects_embedding_below_similarity_threshold"
```

```
collected 17 items / 16 deselected / 1 selected
```

```
test_rejects_embedding_below_similarity_threshold PASSED [100%]
```

```
1 passed, 16 deselected in 0.19s
```

Chương 4: Kết quả thực nghiệm

Testcase 8: test_nearby_new_track_inherits_recent_known_identity

Mã testcase	TC-ID-02
Mục đích kiểm thử	Kiểm tra cơ chế bộ nhớ ngắn hạn: một track mới xuất hiện gần vị trí một danh tính vừa được xác nhận trước đó sẽ kế thừa lại nhãn danh tính đó mà không cần chạy lại nhận diện khuôn mặt.
Điều kiện tiên đề	Track ID 16 đã được xác nhận là "Alice" ở khung hình trước; ở khung hình kế tiếp, bộ nhận diện khuôn mặt trả về danh sách rỗng (giả lập mất khuôn mặt tạm thời), track ID mới (17) xuất hiện với khung bao gần vị trí cũ.
Các bước thực hiện	1. Gọi label_objects() cho track 16 với khuôn mặt khớp "Alice". 2. Thay bộ nhận diện khuôn mặt bằng bộ giả không trả về khuôn mặt nào. 3. Gọi label_objects() cho track 17 (ID mới, vị trí gần track 16).
Kết quả mong đợi	Track 17 được gán nhãn identity_label = "Alice" và identity_kind = "known_person", kế thừa đúng danh tính đã xác nhận gần đó thay vì chuyển về trạng thái chờ xác nhận (pending).

- **Kết quả thực tế:** PASSED; 1 passed, 16 deselected; thời gian 0,19 giây.

```
PS E:\SCT_Camera> .\venv\Scripts\python.exe -m pytest -v
tests\test_person_identity.py -k "test_nearby_new_track_inherits_recent_known_identity"
```

```
collected 17 items / 16 deselected / 1 selected
```

```
test_nearby_new_track_inherits_recent_known_identity PASSED [100%]
```

```
1 passed, 16 deselected in 0.19s
```

Chương 4: Kết quả thực nghiệm

Testcase 9: test_loitering_uses_roi_zone_and_exposes_timer

Mã testcase	TC-BHV-01
Mục đích kiểm thử	Kiểm tra luật Loitering đo đúng thời gian lưu trú của một người trong vùng giám sát và chỉ phát cảnh báo khi vượt ngưỡng thời gian cấu hình.
Điều kiện tiên đề	LoiteringDetector với ngưỡng mặc định 30 giây; một vùng "Porch" kiểu loitering bao phủ toàn khung hình; một track person xuất hiện tại thời điểm mô phỏng $t = 100s$.
Các bước thực hiện	1. Gọi <code>analyze()</code> lần đầu tại $t = 100s$ — chưa đủ thời gian, kỳ vọng không có cảnh báo. 2. Kiểm tra trạng thái timer đang chạy (<code>get_active_states</code>) tại $t = 110s$ — đã lưu trú 10 giây. 3. Gọi <code>analyze()</code> lần thứ hai tại $t = 131s$ — đã lưu trú 31 giây, vượt ngưỡng 30 giây.
Kết quả mong đợi	Không có cảnh báo nào phát sinh ở bước 1; trạng thái timer tại bước 2 hiển thị đúng <code>duration ≈ 10.0</code> giây và <code>threshold_seconds = 30.0</code> ; ở bước 3, đúng một cảnh báo loại "loitering" được phát ra với <code>zone_id = "porch"</code> .

- **Kết quả thực tế:** PASSED; 1 passed, 26 deselected; thời gian 0,26 giây.

```
PS E:\SCT_Camera> .\venv\Scripts\python.exe -m pytest -v
tests\test_person_alerts.py -k "test_loitering_uses_roi_zone_and_exposes_timer"
```

```
collected 27 items / 26 deselected / 1 selected
```

```
test_loitering_uses_roi_zone_and_exposes_timer PASSED [100%]
```

```
1 passed, 26 deselected in 0.26s
```

Chương 4: Kết quả thực nghiệm

Testcase 10: test_theft_alert_dedupes_when_vehicle_track_id_changes

Mã testcase	TC-BHV-02
Mục đích kiểm thử	Kiểm tra luật Suspicious Theft Behavior không phát cảnh báo trùng lặp cho cùng một phương tiện khi ByteTrack vô tình đổi track ID của phương tiện đó (ví dụ do bị che khuất tạm thời rồi xuất hiện lại với ID mới).
Điều kiện tiên đề	SuspiciousTheftDetector với proximity_seconds = 1, score_threshold = 2; một người lạ đứng gần một xe đạp trong vùng bảo vệ; kịch bản mô phỏng xe đạp đổi từ track ID 10 sang track ID 11 giữa hai lượt phân tích.
Các bước thực hiện	1. Phân tích cặp (người, xe đạp track 10) tại t = 0s và t = 2s — xe đạp có dấu hiệu dịch chuyển, đủ điều kiện phát cảnh báo lần đầu. 2. Phân tích cặp (người, xe đạp track 11 — ID mới) tại t = 4s và t = 6s với cùng kịch bản dịch chuyển.
Kết quả mong đợi	Đúng một cảnh báo "suspicious_theft_behavior" được phát ra ở lượt đầu tiên (track 10); lượt thứ hai (track 11, cùng loại phương tiện, cùng vùng) không phát sinh cảnh báo mới nhờ cơ chế khử trùng lặp theo chữ ký (camera, vùng, lớp phương tiện).

- **Kết quả thực tế:** PASSED; 1 passed, 2 deselected; thời gian 0,20 giây.

```
PS E:\SCT_Camera> .\venv\Scripts\python.exe -m pytest -v
tests\test_theft_behavior.py
"test_theft_alert_dedupes_when_vehicle_track_id_changes"
```

-k

collected 3 items / 2 deselected / 1 selected

test_theft_alert_dedupes_when_vehicle_track_id_changes PASSED [100%]

1 passed, 2 deselected in 0.20s

Chương 4: Kết quả thực nghiệm

Testcase 11: test_new_camera_automatically_applies_max_quality_profile

Mã testcase	TC-CFG-01
Mục đích kiểm thử	Kiểm tra khi thêm một camera mới vào hệ thống, cấu hình runtime toàn cục tự động được nâng lên hồ sơ chất lượng cao nhất (max_quality_realtime).
Điều kiện tiên đề	RuntimeState đang ở cấu hình mặc định (stream_max_height = 720); chưa có camera nào được cấu hình.
Các bước thực hiện	1. Gọi runtime.upsert_camera() với thông tin một camera mới (tên, nguồn RTSP). 2. Kiểm tra runtime.update_settings() có được gọi với bộ tham số MAX_QUALITY_RUNTIME_SETTINGS hay không. 3. Kiểm tra trường vision_profile trong kết quả trả về.
Kết quả mong đợi	Hàm update_settings() được gọi đúng một lần với đúng bộ tham số MAX_QUALITY_RUNTIME_SETTINGS; pipeline được khởi động lại (restart) đúng một lần; camera vừa thêm có vision_profile = "max_quality_realtime".

- **Kết quả thực tế:** PASSED; 1 passed, 10 deselected; thời gian 0,59 giây.

```
PS E:\SCT_Camera> .\venv\Scripts\python.exe -m pytest -v
tests\test_runtime_settings.py
"test_new_camera_automatically_applies_max_quality_profile"
```

-k

collected 11 items / 10 deselected / 1 selected

test_new_camera_automatically_applies_max_quality_profile PASSED [100%]

1 passed, 10 deselected in 0.59s

Danh sách trên trình bày 11 trong tổng số 206 testcase để minh họa mức độ chi tiết của bộ kiểm thử. Các testcase còn lại có cấu trúc tương tự; toàn bộ mã nguồn được lưu trong thư mục tests/ của dự án và có thể chạy lại bằng lệnh pytest -v để tái tạo kết quả.

4.5 Kiểm thử nghiệp vụ trên camera thực tế

Bên cạnh 206 testcase tự động ở mục trước, hệ thống được đối chiếu theo bảy nhóm cảnh báo nghiệp vụ mà BehaviorEngine thực sự gọi trong pipeline: ngã/bất tỉnh, nghi trộm cắp, tài sản bị di chuyển hoặc biến mất, lãng vãng trong ROI, phát hiện người lạ, người lạ có hành vi đáng ngờ và cắt vạch theo hướng vào. Bảy trường hợp dưới đây là kịch bản kiểm thử tích hợp đầu cuối nên không được cộng thêm vào tổng số 206 testcase pytest. Cả bảy trường hợp đều đã có ảnh camera thực tế đặt ngay sau bảng testcase tương ứng. Các testcase tại mục 4.5 xác nhận chức năng đầu-cuối trong các kịch bản cụ thể, trong khi pilot tại mục 4.6 đánh giá định lượng trên các positive/negative clip đóng băng và do đó có thể bộc lộ thêm false alert, miss hoặc duplicate event.

4.5.1 Trường hợp 1: Phát hiện người ngã và nằm bất động

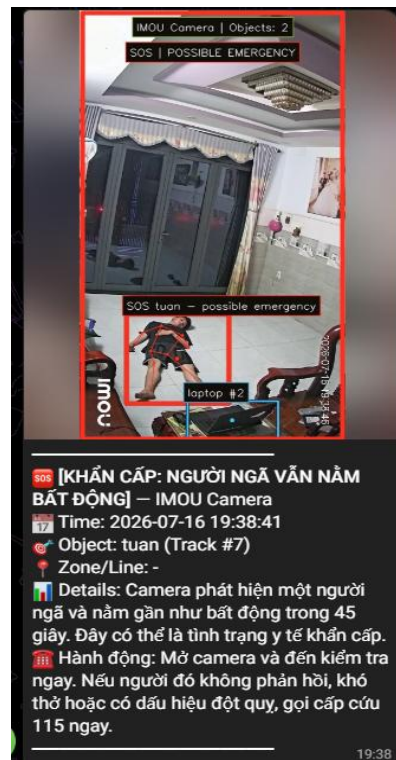
Mã testcase	TC-E2E-01
Mục đích kiểm thử	Xác nhận chuỗi xử lý đầu cuối phát hiện chuyển tiếp từ tư thế thẳng sang nằm, phát cảnh báo theo hai mức, duy trì trạng thái khi người bị che khuất sau khi đã quan sát thấy tư thế nằm và xóa cảnh báo khi xác nhận hồi phục.
Điều kiện tiên đề	Camera đã kết nối; mô hình phát hiện người và ước lượng tư thế đang hoạt động; fall detection bật với lying_alert_seconds=10, escalation_seconds=45, occlusion_grace_seconds=60 và recovery_confirm_seconds=3.
Các bước thực hiện	1. Khởi động camera và pipeline AI. 2. Tạo chuyển tiếp từ đứng/đi sang nằm và duy trì tư thế. 3. Quan sát cảnh báo POSSIBLE FALL sau ngưỡng 10 giây và POSSIBLE EMERGENCY sau ngưỡng 45 giây. 4. Sau khi hệ thống đã thấy tư thế nằm, che khuất người hoặc làm mất bbox trong thời gian ngắn và quan sát chỉ báo LAST SEEN. 5. Cho người xuất hiện lại, ngồi/đứng ổn định ít nhất 3 giây và kiểm tra cảnh báo trực quan được xóa.

Chương 4: Kết quả thực nghiệm

Mã testcase	TC-E2E-01
Kết quả mong đợi	Hệ thống chỉ tạo candidate khi đã quan sát thấy chuyển tiếp sang nằm; phát POSSIBLE FALL rồi nâng mức POSSIBLE EMERGENCY theo ngưỡng cấu hình. Khi mất bbox sau trạng thái nằm đã xác nhận, hệ thống giữ vị trí cuối bằng khung nét đứt LAST SEEN tối đa 60 giây và ghi rõ giới hạn tầm nhìn, không coi đó là phát hiện hiện tại. Khi ngồi/đứng/đi lại ổn định 3 giây, sự kiện fall_recovery xóa lớp phủ cảnh báo cũ.

PASSED đối với kịch bản camera nhìn thấy liên tục: hệ thống đánh dấu người bằng khung bao và khung xương, phát POSSIBLE FALL sau khoảng 10 giây, chuyển sang “SOS / POSSIBLE EMERGENCY” và gửi thông báo khẩn cấp khi thời gian nằm đạt khoảng 45 giây. Các nhánh che khuất/LAST SEEN, hồi phục, xóa cảnh báo và reset trạng thái camera được xác nhận bằng nhóm 72 testcase hồi quy fall/pose/pipeline/drawing, là một tập con của bộ 206 testcase toàn hệ thống; 72/72 testcase đạt trong lần chạy ngày 20/08/2026.

Chương 4: Kết quả thực nghiệm



Hình 4-5: Kết quả phát hiện người ngã và nằm bất động kèm cảnh báo SOS trên camera thực tế

Cần lưu ý: LAST SEEN biểu diễn vị trí nhìn thấy cuối cùng trong thời gian che khuất, không phải bbox phát hiện hiện tại và không chứng minh người bất tỉnh.

Chương 4: Kết quả thực nghiệm

- Thuật toán phát hiện người ngã và nằm bất động

`handle_fall_detection(track):`

1. Xác nhận trạng thái thẳng:

- Track phải ở trạng thái đứng yên, đi chậm hoặc chạy liên tục tối thiểu `min_upright_seconds`.

2. Phát hiện chuyển tiếp sang tư thế nằm:

- Trong vòng `max_transition_seconds`, nếu track chuyển sang `lying/changing_to_lying` và có độ hạ tâm đủ lớn, tạo fall candidate. Không cảnh báo nếu người chỉ biến mất mà chưa từng được thấy nằm.

3. Xác nhận nguy cơ ngã:

- Nếu trạng thái nằm kéo dài ít nhất `lying_alert_seconds` (10 giây), phát `possible_fall`.
- Hiển thị lớp phủ “SOS | POSSIBLE FALL”.

4. Xử lý trường hợp bị che khuất hoặc mất dấu:

- Sau khi đã xác nhận nằm, giữ fall candidate tối đa `occlusion_grace_seconds` (60 giây).
- Lưu `last_known_bbox` và hiển thị khung nét đứt “LAST SEEN”.
- Thông báo ghi rõ người đang bị che khuất hoặc mất khỏi tầm nhìn.

5. Nâng cấp mức cảnh báo:

- Khi tổng thời gian nằm đạt `escalation_seconds` (45 giây), phát `possible_unresponsive` và hiển thị “POSSIBLE EMERGENCY”.
- Có thể gửi lại cảnh báo theo chu kỳ `urgent_reminder_seconds`.

6. Xác nhận hồi phục:

- Khi track trở lại trạng thái ngồi, đứng, đi hoặc chạy ổn định tối thiểu `recovery_confirm_seconds` (3 giây), phát `fall_recovery` nội bộ.
- Xóa lớp phủ “POSSIBLE FALL” hoặc “POSSIBLE EMERGENCY”.

7. Đặt lại trạng thái:

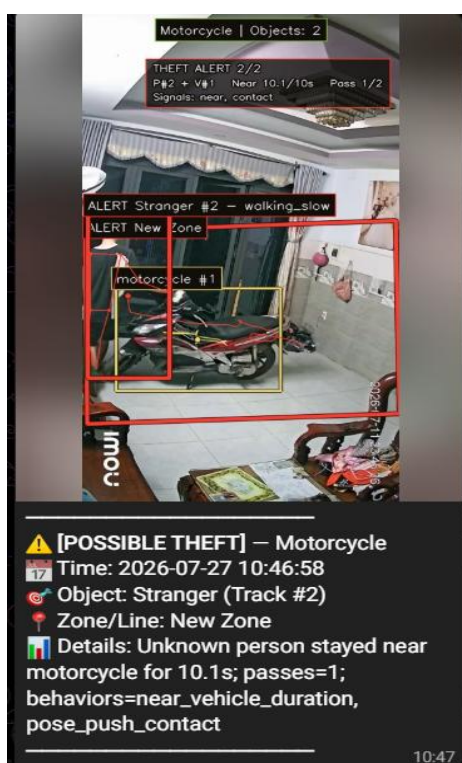
- Khi nguồn camera khởi động lại, xóa lịch sử pose và fall candidate của camera để tránh kế thừa trạng thái cũ

4.5.2 Trường hợp 2: Ước lượng khung xương và phát hiện hành vi nghi trộm cắp

Mã testcase	TC-E2E-02
Mục đích kiểm thử	Xác nhận hệ thống kết hợp phát hiện người, ước lượng khung xương, theo dõi phương tiện và vùng ROI để nhận diện hành vi nghi trộm cắp.
Điều kiện tiên đề	Vùng bảo vệ đã được cấu hình; camera nhìn thấy người và xe máy; các module ByteTrack, Pose và Suspicious Theft Behavior đang hoạt động.
Các bước thực hiện	1. Khởi động camera và bật vùng bảo vệ. 2. Cho một người lạ tiếp cận xe máy trong vùng. 3. Duy trì trạng thái ở gần, tiếp xúc và làm xe dịch chuyển. 4. Quan sát khung xương, track người-phương tiện, điểm tín hiệu hành vi và cảnh báo đầu ra.
Kết quả mong đợi	Hệ thống gắn nhãn người lạ và xe máy, hiển thị khung xương cùng vùng giám sát, tích lũy các tín hiệu near/contact/moved và phát cảnh báo THEFT ALERT khi điểm rủi ro vượt ngưỡng.

PASSED - hệ thống hiển thị vùng cảnh báo, gắn nhãn Stranger và motorcycle, vẽ khung xương người, đồng thời phát THEFT ALERT với các tín hiệu near, contact và gửi cảnh báo POSSIBLE THEFT qua Telegram

Chương 4: Kết quả thực nghiệm



Hình 4-6: Kết quả phát hiện hành vi nghi trộm cắp và gửi cảnh báo Possible Theft qua Telegram

- Thuật toán phát hiện hành vi nghi trộm cắp

handle_theft_behavior(person_track, vehicle_track):

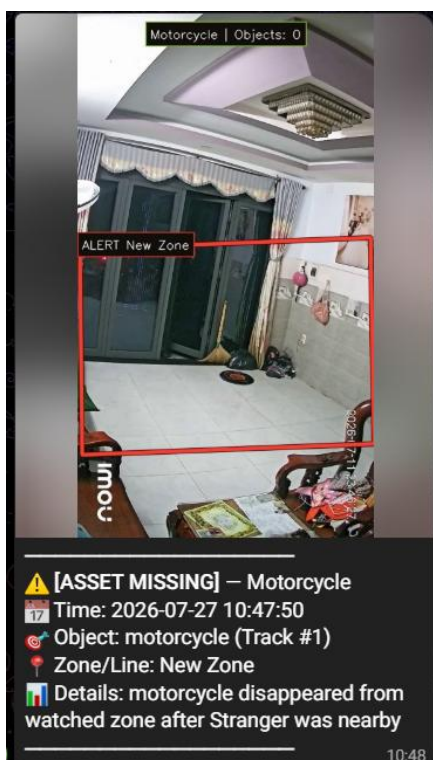
1. Nếu person_track và vehicle_track cùng nằm trong vùng bảo vệ:
 - Tính khoảng cách giữa tâm khung bao người và phương tiện.
2. Nếu khoảng cách nhỏ hơn ngưỡng gần (near):
 - Cộng điểm tín hiệu "near" vào điểm rủi ro của cặp track.
3. Nếu khung xương người chạm vào vùng phương tiện (contact):
 - Cộng điểm tín hiệu "contact".
4. Nếu vị trí phương tiện lệch đáng kể so với vị trí ban đầu (moved):
 - Cộng điểm tín hiệu "moved".
5. Nếu tổng điểm rủi ro vượt ngưỡng THEFT_THRESHOLD và chưa cảnh báo:
 - Phát cảnh báo "THEFT ALERT" kèm các tín hiệu same-dir/near/pacing/contact/moved.
 - Đánh dấu cặp track đã cảnh báo.

4.5.3 Trường hợp 3: Phát hiện tài sản bị di chuyển hoặc biến mất

Mã testcase	TC-E2E-03
Mục đích kiểm thử	Xác nhận hệ thống giám sát tài sản trong vùng asset_watch và phát cảnh báo khi tài sản bị đưa ra khỏi vùng hoặc biến mất sau khi có người ở gần.
Điều kiện tiên đề	Vùng asset_watch đã được cấu hình; lớp tài sản cần bảo vệ nằm trong danh sách theo dõi; camera quan sát được tài sản và người tiếp cận; ngưỡng mất tài sản đã được thiết lập.
Các bước thực hiện	1. Khởi động camera và xác nhận tài sản đang nằm trong vùng bảo vệ. 2. Cho một người đến gần tài sản. 3. Di chuyển tài sản ra khỏi vùng hoặc che/đưa tài sản khỏi khung hình. 4. Duy trì quá ngưỡng và quan sát cảnh báo.
Kết quả mong đợi	Hệ thống liên kết người tiếp cận với tài sản, phát asset_removed khi tài sản đi ra khỏi vùng hoặc asset_missing khi tài sản biến mất, kèm camera, vùng, loại tài sản, người liên quan và còi cảnh báo; không gửi trùng khi track tài sản thay đổi.

PASSED (kiểm thử tự động và camera thực tế) - Hệ thống phát hiện xe máy biến mất khỏi vùng giám sát, liên kết sự kiện với người ở gần tài sản và gửi cảnh báo ASSET MISSING (asset_missing) thành công; cơ chế chống gửi trùng khi track tài sản thay đổi cũng hoạt động đúng. Kết quả thực tế được minh họa tại Hình 30.

Chương 4: Kết quả thực nghiệm



Hình 4-7: Kết quả phát hiện xe máy biến mất khỏi vùng giám sát và gửi cảnh báo Asset Missing

- Thuật toán phát hiện tài sản bị di chuyển hoặc biến mất

handle_asset_watch(asset_track, region):

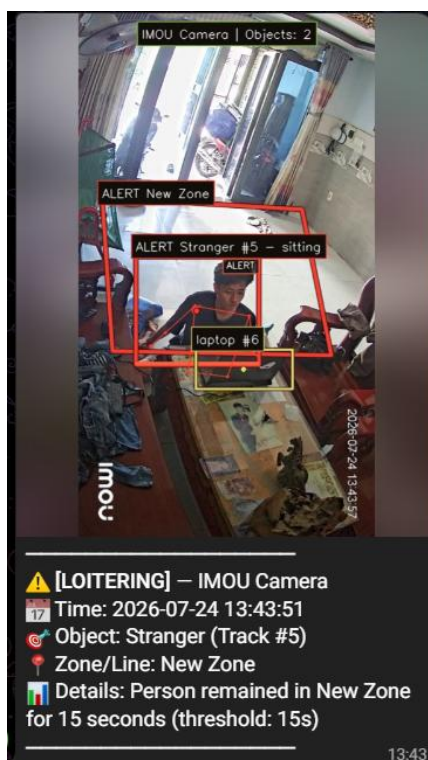
1. Nếu asset_track thuộc danh sách tài sản cần bảo vệ và đang trong vùng asset_watch:
 - Nếu có người xuất hiện gần asset_track trong cửa sổ thời gian gần đây: đánh dấu related_person.
2. Nếu asset_track di chuyển ra khỏi region:
 - Phát cảnh báo "asset_removed" kèm camera, vùng, loại tài sản, related_person, còi cảnh báo.
3. Ngược lại, nếu asset_track biến mất khỏi khung hình quá ngưỡng thời gian:
 - Phát cảnh báo "asset_missing" kèm thông tin tương tự.
4. Sau khi đã cảnh báo:
 - Đánh dấu để không gửi trùng khi track của tài sản thay đổi (ví dụ do mất và tái phát hiện).

4.5.4 Trường hợp 4: Phát hiện hành vi lảng vảng trong vùng ROI

Mã testcase	TC-E2E-04
Mục đích kiểm thử	Xác nhận hệ thống theo dõi thời gian một người hiện diện trong vùng loitering ROI và phát cảnh báo khi thời gian lưu lại vượt ngưỡng.
Điều kiện tiên đề	Camera và ByteTrack đang hoạt động; vùng ROI được cấu hình với loại loitering; ngưỡng thời gian lảng vảng đã được thiết lập.
Các bước thực hiện	1. Khởi động camera và hiển thị vùng loitering ROI. 2. Cho một người đi vào và ở lại trong vùng. 3. Quan sát bộ đếm thời gian gắn với track người. 4. Duy trì sự hiện diện quá ngưỡng và kiểm tra cảnh báo đầu ra.
Kết quả mong đợi	Hệ thống giữ đúng track, chỉ tính thời gian khi người nằm trong ROI, hiển thị thời gian lưu lại và phát cảnh báo loitering có camera, vùng, track và thời lượng; không phát lặp lại trong cùng một phiên.

PASSED (kiểm thử tự động và camera thực tế) - Hệ thống phát hiện người lảng vảng trong vùng ROI vượt quá ngưỡng thời gian cấu hình và gửi cảnh báo lảng vảng thành công. Kết quả thực tế được minh họa tại Hình 31.

Chương 4: Kết quả thực nghiệm



Hình 4-8: Kết quả phát hiện người lạ lưu lại trong vùng ROI quá thời gian quy định

- Thuật toán phát hiện lảng vảng trong vùng ROI

handle_loitering(track, roi):

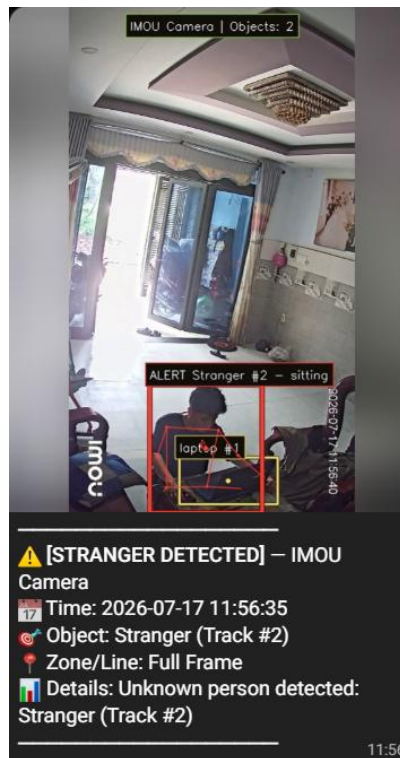
1. Nếu tâm khung bao của track nằm trong ROI loại loitering:
 - Nếu track chưa có bộ đếm: khởi tạo loiter_timer = 0.
 - Tăng loiter_timer theo thời gian track ở lại trong ROI.
2. Nếu loiter_timer $\geq$ ngưỡng lảng vảng và chưa cảnh báo trong phiên hiện tại:
 - Phát cảnh báo loitering kèm camera, vùng, track, thời lượng lưu lại.
 - Đánh dấu track đã cảnh báo trong phiên.
3. Nếu track ra khỏi ROI:
 - Nếu thời gian mất track ngắn (trong khoảng ân hạn): giữ nguyên loiter_timer.
 - Ngược lại: đặt lại loiter_timer và trạng thái cảnh báo về 0.

4.5.5 Trường hợp 5: Phát hiện người lạ xuất hiện

Mã testcase	TC-E2E-05
Mục đích kiểm thử	Xác nhận hệ thống nhận diện một track người không khớp danh tính đã đăng ký và phát cảnh báo người lạ ngay khi danh tính được xác nhận.
Điều kiện tiên đề	Nhận diện khuôn mặt đang hoạt động hoặc camera được cấu hình chính sách coi người chưa biết là Stranger; người kiểm thử chưa có trong cơ sở dữ liệu danh tính.
Các bước thực hiện	1. Khởi động camera và pipeline nhận diện danh tính. 2. Cho người chưa đăng ký xuất hiện rõ mặt trong khung hình. 3. Chờ hệ thống xác nhận nhãn Stranger. 4. Quan sát cảnh báo và tiếp tục đứng trong khung hình để kiểm tra chống gửi lặp.
Kết quả mong đợi	Track được gán nhãn Stranger và phát một cảnh báo stranger_detected cho lần xuất hiện, kèm camera, track, nhãn danh tính và thời gian; không cảnh báo lặp liên tục khi cùng track vẫn còn hiện diện.

PASSED - hệ thống gán nhãn Stranger (Track #2) trên toàn khung hình và gửi cảnh báo STRANGER DETECTED kèm camera, thời gian và thông tin đối tượng.

Chương 4: Kết quả thực nghiệm



Hình 4-9: Kết quả phát hiện người lạ trên toàn khung hình và gửi cảnh báo

- Thuật toán phát hiện người lạ xuất hiện

handle_stranger_detection(track):

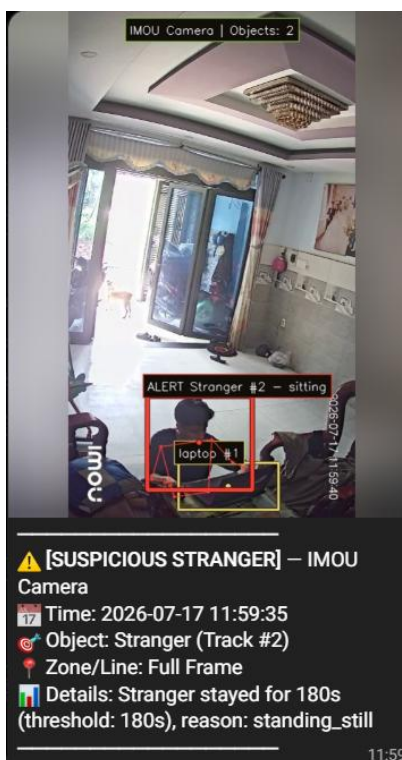
1. Nếu module nhận diện khuôn mặt không khớp với danh tính đã đăng ký:
 - Gắn nhãn track là "Stranger".
2. Nếu track vừa được gắn nhãn Stranger và chưa từng phát cảnh báo:
 - Phát cảnh báo "STRANGER DETECTED" kèm camera, track, nhãn danh tính, thời gian.
 - Đánh dấu track đã cảnh báo.
3. Nếu track vẫn đang hiện diện và đã được cảnh báo trước đó:
 - Không gửi cảnh báo lặp lại cho cùng track.

4.5.6 Trường hợp 6: Phát hiện người lạ có hành vi đáng ngờ

Mã testcase	TC-E2E-06
Mục đích kiểm thử	Xác nhận hệ thống kết hợp danh tính, thời gian lưu lại và quỹ đạo chuyển động để phát hiện người lạ đứng lâu hoặc đi qua lại gần khu vực được giám sát.
Điều kiện tiên đề	Vùng stranger_watch hoặc vùng toàn khung hình tự động đang hoạt động; đối tượng đã được xác nhận là Stranger; ngưỡng thời gian và lịch sử quỹ đạo đủ để đánh giá hành vi.
Các bước thực hiện	1. Khởi động camera và vùng giám sát người lạ. 2. Cho người lạ đứng yên lâu hoặc đi qua lại trong vùng. 3. Duy trì hành vi vượt ngưỡng thời gian cấu hình. 4. Quan sát lý do hành vi và cảnh báo gửi tới người giám sát.
Kết quả mong đợi	Hệ thống ghi nhận thời lượng, phân loại lý do standing_still hoặc pacing_near_area và phát cảnh báo suspicious_stranger có còi/các kênh thông báo theo cấu hình; khoảng mất track ngắn không làm mất bộ đếm.

PASSED - hệ thống duy trì bộ đếm người lạ trong 180 giây, xác định lý do standing_still và phát cảnh báo SUSPICIOUS STRANGER cho Track #2.

Chương 4: Kết quả thực nghiệm



Hình 4-10: Kết quả phát hiện người lạ có hành vi đứng lâu đáng ngờ

- Thuật toán phát hiện người lạ có hành vi đáng ngờ

handle_suspicious_stranger(track):

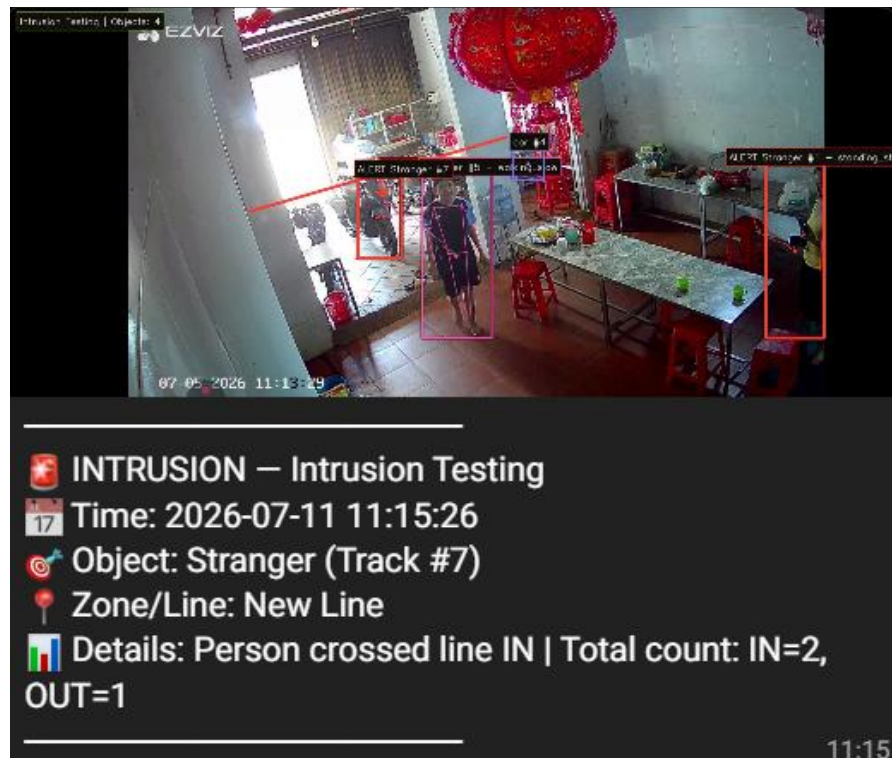
1. Nếu track được gắn nhãn Stranger và nằm trong vùng giám sát người lạ:
 - Cập nhật lịch sử quỹ đạo (vị trí theo thời gian) của track.
 - Tính thời gian đứng yên liên tục và số lần đi qua lại gần khu vực.
2. Nếu thời gian đứng yên $\geq$ ngưỡng standing_still:
 - Đặt lý do = "standing_still".
3. Ngược lại, nếu số lần đi qua lại $\geq$ ngưỡng pacing:
 - Đặt lý do = "pacing_near_area".
4. Nếu xác định được lý do và chưa cảnh báo trong phiên hiện tại:
 - Phát cảnh báo "SUSPICIOUS STRANGER" kèm lý do và còi/kênh thông báo theo cấu hình.
5. Nếu mất track trong thời gian ngắn (trong khoảng ân hận):
 - Không đặt lại các bộ đếm thời gian và quỹ đạo.

4.5.7 Trường hợp 7: Phát hiện cắt vạch theo hướng vào

Mã testcase	TC-E2E-07
Mục đích kiểm thử	Xác nhận hệ thống phát hiện một người cắt vạch đếm theo hướng IN, cập nhật bộ đếm vào/ra và tạo cảnh báo xâm nhập.
Điều kiện tiên đề	Camera và ByteTrack đang hoạt động; vạch đếm cùng hướng forward/reverse đã được cấu hình; track người có lịch sử tâm khung bao liên tục.
Các bước thực hiện	1. Khởi động camera và hiển thị vạch đếm. 2. Cho một người đi qua vạch theo hướng OUT và kiểm tra chỉ tăng bộ đếm. 3. Cho người đi qua vạch theo hướng IN. 4. Quan sát hướng, tổng đếm và cảnh báo đầu ra.
Kết quả mong đợi	Lượt OUT chỉ cập nhật bộ đếm; lượt IN tăng bộ đếm và phát cảnh báo intrusion kèm line_id, hướng IN, track và tổng IN/OUT. Việc chạm vạch không tạo nhiều cảnh báo liên tiếp.

PASSED - hệ thống phát hiện Stranger (Track #7) cắt vạch New Line theo hướng IN, phát cảnh báo INTRUSION và cập nhật bộ đếm IN=2, OUT=1.

Chương 4: Kết quả thực nghiệm



Hình 4-11: Kết quả phát hiện người cắt vạch theo hướng vào và cập nhật bộ đếm

- Thuật toán phát hiện cắt vạch theo hướng vào

handle_line_crossing(track, line):

1. Lấy vị trí tâm khung bao hiện tại và trước đó của track.
2. Nếu đoạn di chuyển của track cắt qua vạch line (dùng tích có hướng để xác định phía):
 - Xác định hướng cắt là IN hoặc OUT.
3. Nếu hướng cắt là OUT:
 - Tăng bộ đếm OUT của line.
4. Nếu hướng cắt là IN:
 - Tăng bộ đếm IN của line.
 - Phát cảnh báo "INTRUSION" kèm line_id, hướng IN, track, tổng IN/OUT.
5. Đánh dấu track đã xử lý cho lượt cắt vạch hiện tại:
 - Tránh đếm hoặc cảnh báo trùng khi track chạm/rung sát vạch nhiều lần.

Mục 4.5 đã bao quát đủ bảy nhóm cảnh báo nghiệp vụ của hệ thống. Cả bảy trường hợp đều có bằng chứng trực quan trên camera thật, bao gồm cảnh báo Possible

Chương 4: Kết quả thực nghiệm

Theft qua Telegram, cảnh báo xe máy biến mất khỏi vùng asset_watch và cảnh báo lãng vãng trong ROI. Tất cả ảnh minh chứng được đặt ngay sau bảng tương ứng và nằm ngoài bảng.

4.6 Đánh giá pilot định lượng các hành vi

4.6.1 Thiết kế thí nghiệm pilot

Thí nghiệm được thiết kế dưới dạng pilot với một người tham gia (P01), gồm 20 clip cho năm hành vi: intrusion (xâm nhập), loitering (lãng vãng), suspicious behavior (hành vi đáng ngờ), theft (nghỉ trộm cắp) và line crossing (cắt vạch). Mỗi hành vi có hai lần lặp; mỗi lần lặp gồm một trường hợp dương tính và một trường hợp âm tính. Lần lặp r1 tạo 10 clip phát triển, còn r2 tạo 10 clip kiểm thử đóng băng.

Tập dữ liệu cân bằng theo hai điều kiện ánh sáng daylight/low-light và hai góc quan sát frontal-oblique/side-oblique. Thời lượng tập phát triển là 666,296 giây, thời lượng frozen test là 677,944 giây và tổng thời lượng xử lý là 1.344,240 giây. Hai tập tách theo lần lặp và nhóm phiên ghi, nhưng không participant-disjoint vì P01 xuất hiện ở cả hai tập. Video, cấu hình và ngưỡng được đóng băng trước khi đánh giá; không chạy lại hoặc retune sau khi xem kết quả test.

4.6.2 Tiêu chí đánh giá

Các metric được tính riêng cho từng hành vi trên tập frozen test theo các định nghĩa sau:

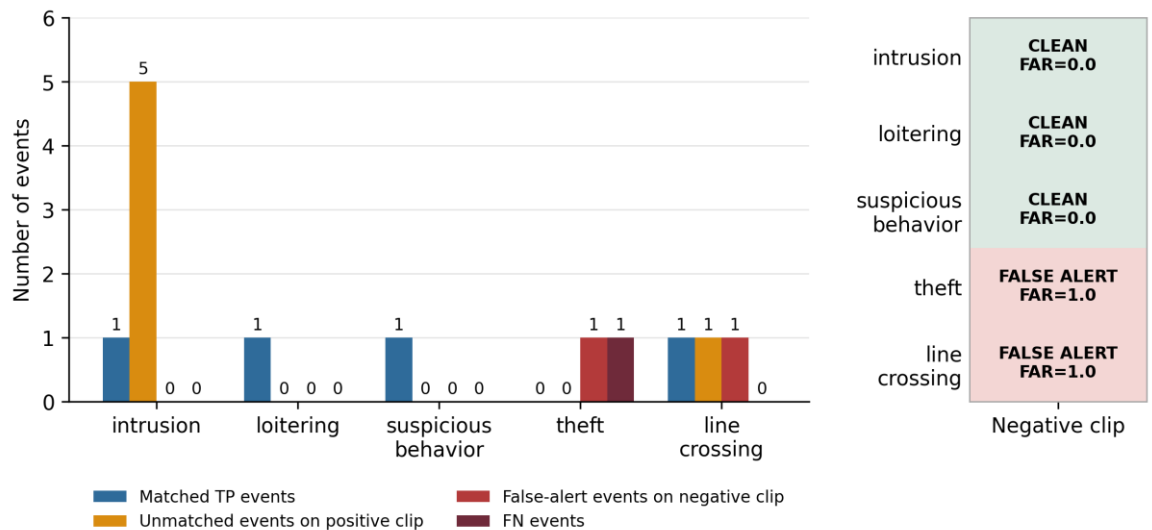
- True Positive (TP): sự kiện cảnh báo khớp với cửa sổ ground truth của clip dương tính.
- False Positive (FP): mọi sự kiện được hệ thống phát ra nhưng không khớp ground truth, bao gồm cả sự kiện duplicate hoặc unmatched trên clip dương tính.
- False Negative (FN): sự kiện dương tính trong ground truth nhưng hệ thống không phát hiện được.

Chương 4: Kết quả thực nghiệm

- True Negative (TN): clip âm tính không phát sinh cảnh báo sai cho hành vi đang xét.
- Detection Rate (DR) = $TP/(TP+FN)$, phản ánh tỷ lệ sự kiện dương tính được phát hiện.
- False Alarm Rate (FAR): tỷ lệ clip âm tính có ít nhất một false alert trên tổng số clip âm tính.
- Precision = $TP/(TP+FP)$; Recall = $TP/(TP+FN)$; F1-score là trung bình điều hòa của Precision và Recall.
- Detection latency: thời gian từ thời điểm hành vi đủ điều kiện theo ground truth đến TP đầu tiên; báo cáo trung vị theo từng hành vi.

4.6.3 Kết quả trên frozen test

Hình 35 tách số sự kiện TP, FP và FN khỏi trạng thái báo giả ở clip âm tính, tránh trộn metric mức sự kiện với FAR ở mức clip.



Single-participant pilot; one positive and one negative frozen-test clip per behavior. No aggregate score.

Hình 4-12: Kết quả đánh giá thử nghiệm hành vi trên tập kiểm thử đóng băng

Bảng 4-5 trình bày đầy đủ metric của năm hành vi. FP và Precision được tính ở mức sự kiện, trong khi FAR được tính theo clip âm tính như định nghĩa tại mục 4.6.2.

Chương 4: Kết quả thực nghiệm

Bảng 4-5: Kết quả từng hành vi trên tập kiểm thử đóng băng

Hành vi	TP/FP/TN/FN	DR/FAR	Precision/Recall/F1	Độ trễ trung vị (s)
Intrusion	1/5/1/0	1.000/0.000	0.166667/1.000/0.285714	0.457
Loitering	1/0/1/0	1.000/0.000	1.000/1.000/1.000	0.725
Suspicious behavior	1/0/1/0	1.000/0.000	1.000/1.000/1.000	0.186
Theft	0/1/0/1	0.000/1.000	0.000/0.000/0.000	N/A
Line crossing	1/2/0/0	1.000/1.000	0.333333/1.000/0.500000	4.084

Không tính điểm tổng hợp cho pilot vì mỗi hành vi chỉ có một sự kiện dương tính và một clip âm tính trong frozen test; kết quả phải được diễn giải riêng cho từng hành vi.

4.6.4 Phân tích kết quả

- Intrusion phát hiện đúng lần cắt vạch theo hướng IN với độ trễ 0.457 giây và clip OUT âm tính không phát cảnh báo intrusion. Tuy nhiên, hệ thống sinh thêm năm sự kiện duplicate/unmatched trên chính clip dương tính, làm Precision chỉ đạt 0.166667.
- Loitering hoạt động đúng trên frozen test: phát hiện clip dương tính sau ngưỡng dwell đóng băng và không phát cảnh báo trên clip âm tính.
- Suspicious behavior hoạt động đúng và ổn định trong phạm vi pilot: clip dương tính được phát hiện, clip âm tính sạch và không xuất hiện FP hay FN.
- Theft bỏ sót positive case và phát một false alert trên negative case. Bằng chứng của cảnh báo sai cho thấy pose_push_contact=1 trong khi vehicle_started_moving=0, nên tín hiệu tiếp xúc tự thế có thể bị kích hoạt sai dù xe không dịch chuyển.
- Line crossing bắt đúng event mục tiêu với độ trễ 4.084 giây, nhưng còn một early/unmatched event trên clip dương tính và một false event trên clip âm tính.

4.6.5 Hạn chế của pilot

Pilot chỉ có một participant; mỗi hành vi trong frozen test chỉ có một clip dương tính và một clip âm tính. Development/test được tách theo repetition và session-group nhưng không participant-disjoint. Với sample size nhỏ như vậy, kết quả chỉ dùng để kiểm tra quy trình và nhận diện lỗi, không chứng minh khả năng generalization theo người, theo bối cảnh hoặc hiệu năng validation quy mô đầy đủ.

4.7 Đánh giá pilot ngưỡng nhận diện khuôn mặt

4.7.1 Thiết kế và giao thức pilot

Dataset gồm 24 ảnh của hai danh tính Nguyen (11 ảnh) và Tuan (13 ảnh). Các ảnh sai orientation được chuẩn hóa trước khi trích embedding nhưng ảnh gốc không bị ghi đè. Split được khóa trước khi xem metric: mỗi danh tính dùng ba ảnh enrollment, tổng cộng 10 ảnh development và tám ảnh frozen test (Nguyen: 5; Tuan: 3). Development images không được dùng để tính kết quả frozen test.

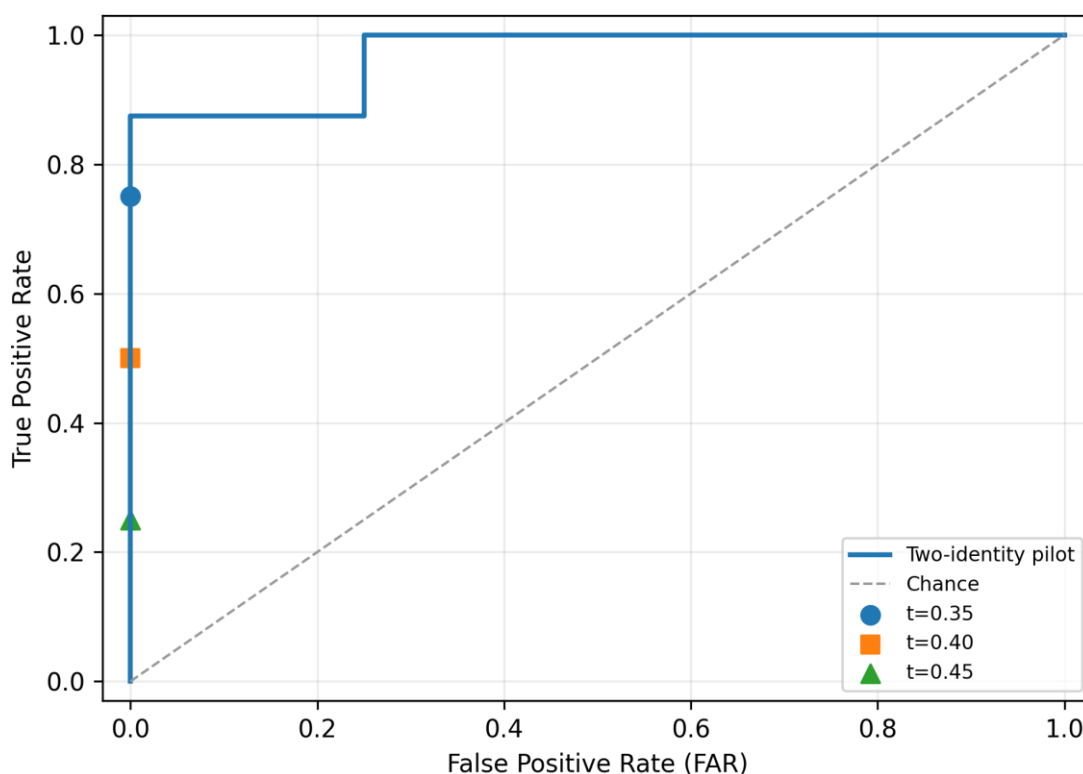
Pilot dùng đúng stack production InsightFace buffalo_sc với detection size 320, cùng bước detect/align của FaceAnalysis. Embedding được chuẩn hóa L2 và cosine similarity được tính bằng tích vô hướng. Mỗi ảnh test được so với ba enrollment references của từng identity; score của identity là giá trị lớn nhất, đúng cơ chế max similarity của runtime. Tất cả 14 ảnh enrollment/frozen-test đều detect được khuôn mặt và trích được embedding.

4.7.2 Tiêu chí đánh giá

Tám ảnh frozen test tạo tám genuine scores và tám impostor scores ở mức identity sau aggregation, tương ứng 48 phép so sánh ảnh-tham chiếu trước khi lấy max. FAR là tỷ lệ impostor score được chấp nhận nhầm; FRR là tỷ lệ genuine score bị từ chối. ROC biểu diễn TPR theo FPR khi thay đổi ngưỡng. EER được lấy tại điểm gần nhất nơi FAR và FRR bằng nhau trên lưới ngưỡng 0,20-0,70 với bước 0,005.

4.7.3 Kết quả ROC, EER và FAR/FRR

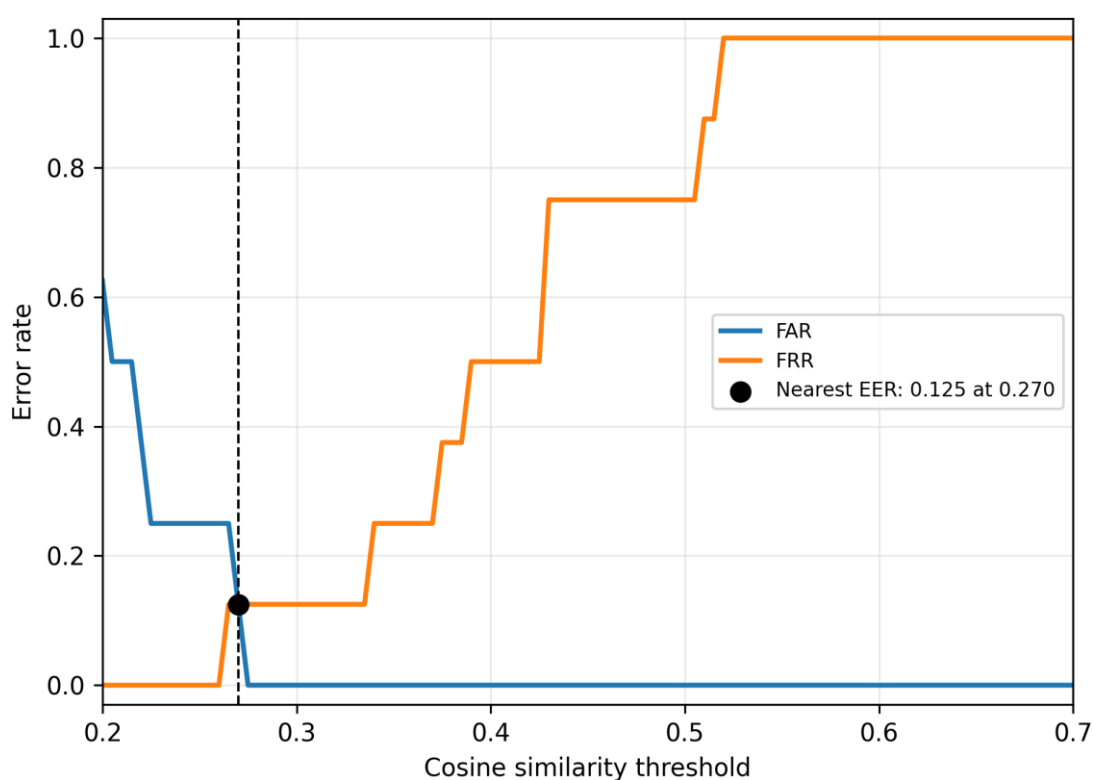
Hình 36 trình bày đường ROC và ba operating point 0,35; 0,40; 0,45. Do chỉ có tám genuine và tám impostor scores, các mức TPR/FPR thay đổi theo bước 0,125; đường ROC vì vậy được diễn giải như kết quả pilot, không phải ước lượng hiệu năng trên dân số lớn.



Hình 4-13: Đường ROC của pilot nhận diện khuôn mặt hai danh tính

Hình 37 cho thấy EER gần đúng bằng 0,125 tại ngưỡng 0,270, trong đó FAR=0,125 và FRR=0,125. Đây là điểm gần nhất trên lưới bước 0,005, không phải một ngưỡng tối ưu đã được chứng minh cho triển khai thực tế.

Chương 4: Kết quả thực nghiệm



Hình 4-14: FAR và FRR theo ngưỡng cosine similarity trong pilot

Bảng 4-6 báo cáo riêng ba ngưỡng đang được quan tâm trong hệ thống. Quy tắc chấp nhận là cosine similarity lớn hơn hoặc bằng threshold.

Bảng 4-6: Kết quả pilot nhận diện khuôn mặt tại ba ngưỡng khảo sát

Threshold	F A R	F R R	T P R	F P R	TP	FP	TN	FN
0,35	0,000	0,250	0,750	0,000	6	0	8	2
0,40	0,000	0,500	0,500	0,000	4	0	8	4
0,45	0,000	0,750	0,250	0,000	2	0	8	6

Trong ba operating point 0,35; 0,40; 0,45, ngưỡng 0,35 cho kết quả tốt nhất với FAR=0,000 và FRR=0,250. Khi tăng lên 0,40 và 0,45, FAR vẫn bằng 0 nhưng FRR tăng lần lượt lên 0,500 và 0,750. Trên toàn lưới sweep, threshold 0,335 đạt balanced accuracy 0,9375 với FAR=0,000 và FRR=0,125; đây chỉ là best-performing threshold trong two-identity pilot này.

Chương 4: Kết quả thực nghiệm

4.7.4 Hạn chế của pilot khuôn mặt

Pilot chỉ có hai identities và sample size nhỏ; không participant-disjoint ngoài hai danh tính này và không đại diện cho dân số lớn. Kết quả ROC/EER/FAR/FRR chỉ kiểm tra tính hợp lý của dải ngưỡng hiện tại, không chứng minh full-scale validation hoặc khả năng generalization theo người, camera, session và điều kiện sáng.

4.8 Đánh giá độ ổn định hệ thống

Độ ổn định được đánh giá bằng hai phiên chạy độc lập, mỗi phiên bật đồng thời bốn camera trong 8 giờ và ghi 2.880 mẫu. Bảng 4-7 so sánh trạng thái đầu và cuối phiên trên hai cấu hình.

Bảng 4-7: Kết quả kiểm thử độ ổn định bốn camera trong 8 giờ

Cấu hình	Số mẫu	AI FPS TB (30 mẫu đầu→cuối)	RAM MB (30 mẫu đầu→cuối)	VRAM cuối / Tmax	Drop / Online
RTX 3050 Laptop	2.880	24,05 (22,37→24,24)	2.029,0→1.143,9	334 / 57 °C	0 / 4/4
RTX 5060 Ti	2.880	29,49 (29,51→29,74)	2.746,8→2.746,8	1.061,9 / 52 °C	0 / 4/4

RTX 3050 Laptop có AI FPS trung bình 24,05 và xu hướng +0,116 FPS/giờ; RSS giảm sau khi giải phóng cache thay vì tăng dần. RTX 5060 Ti đạt 29,49 AI FPS, xu hướng +0,007 FPS/giờ và RAM gần như giữ nguyên giữa 30 mẫu đầu/cuối. Nhiệt độ cực đại lần lượt là 57 °C và 52 °C. Logger của phiên RTX 5060 Ti có một khoảng gián đoạn 35,5 giây, nhưng tại hai phía của khoảng trống hệ thống vẫn báo 4/4 camera online, không drop và thông lượng không giảm bất thường. Vì vậy, trong phạm vi hai phiên 8 giờ này, không ghi nhận dấu hiệu rò rỉ bộ nhớ, drift thông lượng hay thermal throttling; kết luận này không thay thế kiểm thử dài hơn hoặc trên cấu hình khác.

4.9 Kết quả đạt được và hạn chế còn tồn tại

4.9.1 Kết quả đạt được

- Hệ thống đã được đo định lượng trên hai cấu hình GPU với từ một đến bốn camera; RTX 5060 Ti duy trì xu hướng scaling gần tuyến tính đến bốn camera, còn RTX 3050 Laptop bộc lộ điểm nghẽn ở mức bốn camera.
- Chi phí tính toán của tầng phân tích hành vi (dựa trên hình học và mô hình học máy nhẹ) gần như không đáng kể so với các mô hình mạng nơ-ron sâu.
- Bảy luật hành vi hoạt động độc lập và có thể cấu hình riêng theo từng camera đã được kiểm thử tự động. Pilot định lượng năm hành vi cho thấy loitering và suspicious behavior đạt TP=1, FP=0, FN=0 trên tập test nhỏ; intrusion còn năm sự kiện duplicate/unmatched, theft còn một miss và một false alert, còn line crossing còn early/unmatched event và false event.
- Pilot nhận diện khuôn mặt hai danh tính đã bổ sung ROC, EER, FAR và FRR: EER gần đúng 0,125 tại threshold 0,270; trong ba mức 0,35/0,40/0,45, mức 0,35 đạt FAR=0,000 và FRR=0,250. Kết quả chỉ có ý nghĩa trong pilot nhỏ.
- Hai phiên kiểm thử bốn camera liên tục trong 8 giờ đều duy trì 4/4 camera online, không ghi nhận khung bị bỏ và không xuất hiện xu hướng tăng bộ nhớ không kiểm soát.
- Cơ chế phát hiện ngã hiện hỗ trợ cảnh báo hai mức, duy trì candidate và vị trí cuối khi che khuất tối đa 60 giây, xác nhận hồi phục sau 3 giây và xóa trạng thái khi camera khởi động lại; nhóm hồi quy mục tiêu đạt 72/72 và được tính trong tổng số 206 testcase.

4.9.2 Hạn chế còn tồn tại

- Lưu trữ lịch sử cảnh báo hiện chỉ được giữ trong bộ nhớ và sẽ mất khi khởi động lại hệ thống, mặc dù lược đồ cơ sở dữ liệu cho bảng cảnh báo, lượt gửi thông báo, sự kiện hành vi và video minh chứng đã được thiết kế và kiểm thử sẵn.
- Cơ chế ghi video minh chứng (đoạn video trước/sau thời điểm xảy ra cảnh báo) đã có lược đồ dữ liệu tương ứng nhưng chưa được triển khai đầy đủ trong phiên bản hiện tại.
- Do lượng sự kiện đã gán nhãn hiện còn hạn chế, chưa có đủ dữ liệu để báo cáo định lượng mức độ giảm cảnh báo giả nhờ mô hình hồi quy logistic so với chỉ dùng luật nghiệp vụ thuần túy.
- Kết quả chỉ đại diện cho hai cấu hình và bốn video cục bộ; chưa thể suy rộng sang GPU khác, hơn bốn camera hoặc nguồn RTSP.
- ROC/EER và FAR/FRR của ngưỡng khuôn mặt mới chỉ được đánh giá trên pilot nhỏ gồm hai identities, sáu ảnh enrollment và tám ảnh frozen test; kết quả chưa đủ để suy rộng hoặc coi là full-scale face-recognition validation.
- Kết quả scaling phụ thuộc cấu hình: RTX 3050 Laptop giảm ở bốn camera, còn RTX 5060 Ti gần tuyến tính. Khi vượt bốn camera cần đánh giá batching hoặc đa tiến trình.

Chương 5: Kết luận và hướng phát triển

- Pilot hành vi chỉ có P01, mỗi hành vi một positive clip và một negative clip trong frozen test; split không participant-disjoint và sample size nhỏ nên không chứng minh generalization hoặc full-scale validation.
- Fall detection vẫn dựa trên chuỗi tư thế/hình học và chưa được đánh giá trên tập ngã quy mô lớn. Trong giai đoạn che khuất, LAST SEEN chỉ là vị trí cuối cùng; hệ thống không xác nhận tư thế hiện tại và không thay thế kiểm tra trực tiếp hoặc chẩn đoán y tế.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đề tài đã xây dựng thành công một hệ thống cảnh báo thời gian thực cho camera giám sát an ninh dựa trên AI, với kiến trúc pipeline sáu tầng kết hợp phát hiện đối tượng (YOLOv11), theo dõi đa đối tượng (ByteTrack mở rộng thêm cơ chế phục hồi track bị mất và bù chuyển động camera), nhận diện danh tính (InsightFace), phân tích hành vi dựa trên hình học với bảy luật nghiệp vụ độc lập, cơ chế phát hiện ngã safety-critical có xử lý che khuất, vị trí nhìn thấy cuối và xác nhận hồi phục, một lớp học máy giám sát nhẹ (hồi quy logistic) được thiết kế để chấm điểm rủi ro; thành phần Risk này chưa có đủ dữ liệu gán nhãn và model hợp lệ để xác thực định lượng, cùng một hệ thống điều phối cảnh báo đa kênh và giao diện web quản lý trực quan.

Kết quả thực nghiệm ở Chương 4 cho thấy AI FPS tổng hợp trên RTX 3050 Laptop lần lượt là 7,56; 16,35; 21,34 và 16,40 với 1–4 camera, còn RTX 5060 Ti đạt 7,60; 15,31; 21,97 và 29,83 AI FPS. Trong hai phép đo bốn camera kéo dài 8 giờ, thông lượng trung bình tương ứng đạt 24,05 và 29,49 AI FPS; cả hai phiên duy trì 4/4 camera online, không ghi nhận khung bị bỏ và không cho thấy dấu hiệu rò rỉ bộ nhớ, suy giảm thông lượng theo thời gian hoặc thermal throttling trong điều kiện đã thử. Bộ 206 trường hợp kiểm thử tự động tiếp tục xác nhận tính đúng đắn của các thành phần cốt lõi. Pilot ngưỡng khuôn mặt hai danh tính ghi nhận EER gần đúng 0,125 tại threshold 0,270; trong ba mức 0,35/0,40/0,45, mức 0,35 cho FAR=0,000 và FRR=0,250 nhưng chưa đủ để suy rộng. Pilot hành vi đóng băng cho thấy loitering và suspicious behavior hoạt động tốt trong tập test nhỏ, nhưng intrusion còn duplicate event, theft còn miss và false alert, còn line crossing còn false/early event. Các kết quả này chỉ chứng minh tính khả thi trong phạm vi phần cứng và dữ liệu đã khảo sát; không được suy rộng thành generalization theo người, mọi cấu hình phần cứng, mọi nguồn camera hoặc validation quy mô đầy đủ.

5.2 Ưu điểm và nhược điểm

5.2.1 Ưu điểm

- Kiến trúc phân tầng rõ ràng, mỗi thành phần (phát hiện, theo dõi, nhận diện, phân tích, học hành vi, cảnh báo) độc lập và có thể kiểm thử riêng lẻ.
- Kết hợp cách tiếp cận dựa trên luật nghiệp vụ (dễ diễn giải, cấu hình linh hoạt theo từng camera) với một lớp học máy nhẹ nhằm hỗ trợ giảm cảnh báo giả; hiệu quả của lớp Risk chưa được xác thực định lượng, thay vì phụ thuộc hoàn toàn vào một mô hình học sâu dạng hộp đen.
- Đã chạy được trên GPU laptop 4 GB và GPU rời 16 GB, có số liệu scaling 1–4 camera và hai phiên kiểm thử ổn định 8 giờ để đối chiếu.
- Có bộ 206 testcase tự động bao phủ các thành phần cốt lõi, tạo nền tảng vững chắc để tiếp tục mở rộng hệ thống.
- Đã có pilot định lượng ngưỡng nhận diện khuôn mặt bằng ROC/EER và FAR/FRR trên split đóng băng, thay cho việc chỉ nêu ngưỡng cấu hình mà không có số liệu.
- Cảnh báo ngã được phân tầng từ POSSIBLE FALL đến POSSIBLE EMERGENCY, không biến mất ngay khi mất bbox ngắn hạn và được xóa khi xác nhận người đã hồi phục.

5.2.2 Nhược điểm

- Khả năng mở rộng trên GPU laptop còn hạn chế ở mức bốn camera; kết quả hiện mới đại diện cho hai cấu hình phần cứng và bốn video thử nghiệm cục bộ.
- Lịch sử cảnh báo và video minh chứng chưa được lưu trữ bền vững, chỉ tồn tại trong bộ nhớ tạm thời của tiến trình.

Chương 5: Kết luận và hướng phát triển

- Chưa có đủ dữ liệu gán nhãn để đánh giá định lượng đầy đủ hiệu quả của lớp học máy chấm điểm rủi ro so với chỉ dùng luật nghiệp vụ.
- Pilot hành vi hiện chưa participant-disjoint và đã bộc lộ duplicate event ở intrusion, miss/false alert ở theft và false/early event ở line crossing.
- Pilot khuôn mặt chỉ gồm hai identities và tám ảnh frozen test; chưa đủ để chứng minh khả năng khái quát hoặc lựa chọn threshold tối ưu cho triển khai rộng.

5.3 Hướng phát triển

5.3.1 Hướng phát triển ngắn hạn

- Hoàn thiện việc ghi lịch sử cảnh báo và sự kiện hành vi vào cơ sở dữ liệu theo lược đồ đã thiết kế, thay vì chỉ lưu tạm trong bộ nhớ.
- Triển khai đầy đủ cơ chế ghi video minh chứng (10 giây trước và 20 giây sau thời điểm cảnh báo) gắn kèm với từng cảnh báo.
- Thu thập và gán nhãn thêm dữ liệu sự kiện hành vi thực tế để có thể báo cáo định lượng mức độ cải thiện nhờ mô hình chấm điểm rủi ro.
- Thực hiện thí nghiệm loại bỏ từng thành phần (ablation study) để định lượng đóng góp của từng cải tiến đối với thuật toán ByteTrack gốc.
- Khắc phục cơ chế khử trùng sự kiện intrusion, siết điều kiện bằng chứng chuyển động cho theft và thêm debounce/hysteresis cho line crossing; sau đó đánh giá lại trên một phiên bản thí nghiệm mới mà không thay đổi kết quả đã đóng băng.

5.3.2 Hướng phát triển dài hạn

- Đánh giá hệ thống theo dõi trên các bộ dữ liệu benchmark công khai (ví dụ MOT16/MOT17) để có thể so sánh trực tiếp với các phương pháp theo dõi đã công bố bằng các độ đo MOTA, IDF1, HOTA chuẩn đã trình bày ở mục 2.10.
- Nghiên cứu thay thế hoặc bổ sung mô hình phát hiện bằng các kiến trúc dựa trên Transformer thời gian thực (RT-DETR, DEIM) khi điều kiện phần cứng cho phép.
- Thiết kế lại tầng suy luận theo hướng xử lý theo lô (batched inference) hoặc phân tán trên nhiều tiến trình để cải thiện khả năng mở rộng khi số lượng camera tăng lên.
- Mở rộng đánh giá fall detection trên tập dữ liệu có nhiều người tham gia, nhiều kiểu ngã, hoạt động nằm bình thường và các mức che khuất; báo cáo Detection Rate, FAR, Precision/Recall/F1 và latency trước khi đưa ra kết luận về khả năng khái quát hoặc sử dụng trong bối cảnh y tế.
- Mở rộng tập dữ liệu khuôn mặt với nhiều danh tính, nhiều session, góc quay, điều kiện sáng và thiết bị; chia enrollment/development/test theo người và session độc lập trước khi thực hiện validation quy mô lớn hoặc kết luận về generalization.

TÀI LIỆU THAM KHẢO

- [1] Linh Van Ma, Muhammad Ishfaq Hussain, JongHyun Park, Jeongbae Kim, and Moongu Jeon. 2023. Adaptive Confidence Threshold for ByteTrack in Multi-Object Tracking. arXiv:2312.01650.
- [2] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. 2022. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. In European Conference on Computer Vision (ECCV). Springer, 1–21.
- [3] Shihua Huang, Zhichao Lu, Xiaodong Cun, Yongjun Yu, Xiao Zhou, and Xi Shen. 2025. DEIM: DETR with Improved Matching for Fast Convergence. arXiv:2412.04234.
- [4] Yian Zhao, Wenyu Lv, Shangliang Xu, Jinman Wei, Guanzhong Wang, Qingqing Dang, Yi Liu, and Jie Chen. 2024. DETRs Beat YOLOs on Real-Time Object Detection. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).
- [5] G. Divya Deepak and Subraya Krishna Bhat. 2025. Optimization of deep learning-based Faster R-CNN network for vehicle detection. Scientific Reports 15, 38937.
- [6] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. 2016. Simple Online and Realtime Tracking. In IEEE International Conference on Image Processing (ICIP). 3464–3468.
- [7] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. 2017. Simple Online and Realtime Tracking with a Deep Association Metric. In IEEE International Conference on Image Processing (ICIP). 3645–3649.

- [8] Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. 2021. YOLOX: Exceeding YOLO Series in 2021. arXiv:2107.08430.
- [9] Jocher Glenn. 2024. Ultralytics YOLO11. <https://docs.ultralytics.com/models/yolo11/>.
- [10] Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. 2019. ArcFace: Additive Angular Margin Loss for Deep Face Recognition. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).
- [11] Rudolph E. Kalman. 1960. A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering* 82, 1, 35–45.
- [12] Harold W. Kuhn. 1955. The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly* 2, 1-2, 83–97.
- [13] Bruce D. Lucas and Takeo Kanade. 1981. An Iterative Image Registration Technique with an Application to Stereo Vision. In *Proceedings of the 7th International Joint Conference on Artificial Intelligence (IJCAI)*. 674–679.
- [14] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You Only Look Once: Unified, Real-Time Object Detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 779–788.
- [15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778.
- [16] Anton Milan, Laura Leal-Taixé, Ian Reid, Stefan Roth, and Konrad Schindler. 2016. MOT16: A Benchmark for Multi-Object Tracking. arXiv:1603.00831.
- [17] David W. Hosmer Jr., Stanley Lemeshow, and Rodney X. Sturdivant. 2013. *Applied Logistic Regression* (3rd ed.). John Wiley & Sons.

- [18] Waqas Sultani, Chen Chen, and Mubarak Shah. 2018. Real-World Anomaly Detection in Surveillance Videos. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 6479–6488.
- [19] N. Nielsen, “Comparison between YOLOv10, YOLOv9 and YOLOv8 on Real-world Videos,” 2024.
- [20] Viblo.asia, “Hiểu sâu về You Only Look Once – YOLOv1.
- [21] VNPT AI, “Mô hình YOLO: Giải pháp phát hiện đối tượng theo thời gian thực.” [Online]. Available: <https://vnptai.io/en/blog/detail/mo-hinh-yolo>
- [22] Ultralytics, “Models Supported by Ultralytics,” *Ultralytics Docs*. [Online]
- [23] Wikipedia contributors, “Small object detection,” *Wikipedia, The Free Encyclopedia*.
- [24] MC&TT Co., Ltd, “YOLO (You Only Look Once) là gì? Các tính năng mới của YOLOv8.
- [25] Ultralytics, “YOLO Object Detection & Segmentation,” *Ultralytics Docs*.
- [26] Ultralytics, “YOLOv9 Object Detection with PGI and GELAN,” *Ultralytics Docs*.